

Grupo ARCOS

uc3m | Universidad **Carlos III** de Madrid

Tema 5: Jerarquía de Memoria (II)

Estructura de Computadores

Grado en Ingeniería Informática

Grado en Matemática aplicada y Computación

Doble Grado en Ingeniería Informática y Administración de Empresas

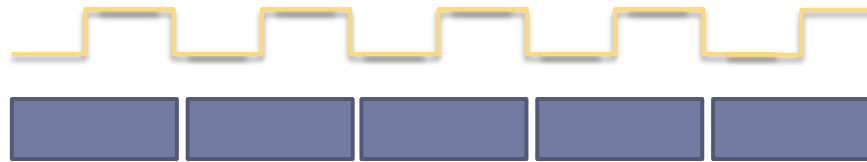


Contenidos

1. Tipos de memoria
2. Jerarquía de memoria
3. Memoria principal
4. Memoria caché
 1. Introducción
 2. Estructura de la memoria caché
 3. Diseño y organización de la memoria caché
5. Memoria virtual

Característica de la memoria principal

- ▶ Se premia el acceso a posiciones consecutivas de memoria
- ▶ Ejemplo 1: acceder a 5 posiciones de memoria **individuales no consecutivas**



- ▶ Ejemplo 2: acceder a 5 posiciones de memoria **consecutivas**



Característica de los accesos a memoria

- ▶ “Principio de proximidad o localidad de referencias”:

Durante la ejecución de un programa,
las referencias (direcciones) a memoria tienden a estar
agrupadas por:

- ▶ **proximidad espacial**
 - ▶ Secuencia de instrucciones
 - ▶ Acceso secuencial a arrays
- ▶ **proximidad temporal**
 - ▶ Bucles

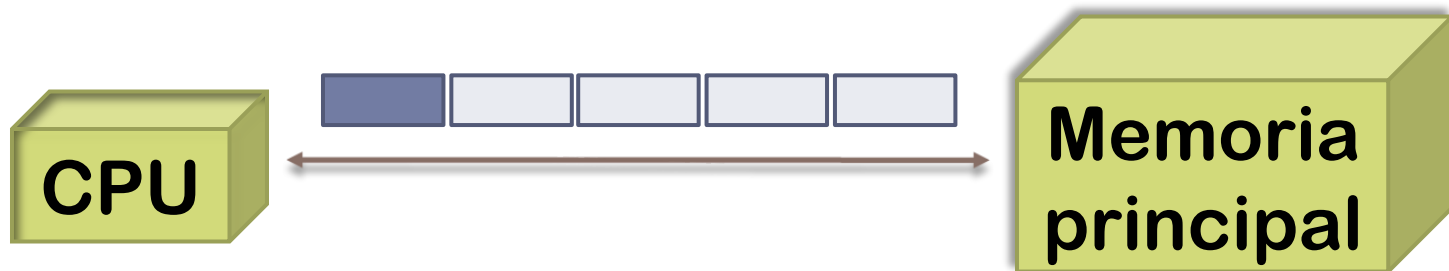
```
.data
vector: .space 4*1024

.text
main: li t0 0
      la t1 vector
      li t3 1024
      li t4 4
b2:   bge t0 t3 fb2
      mul t2 t0 t4
      add t2 t1 t2
      sw t0 0(t2)
      addi t0 t0 1
      j b2
fb2: jr ra
```

Objetivo de la memoria caché:

aprovechar los accesos contiguos

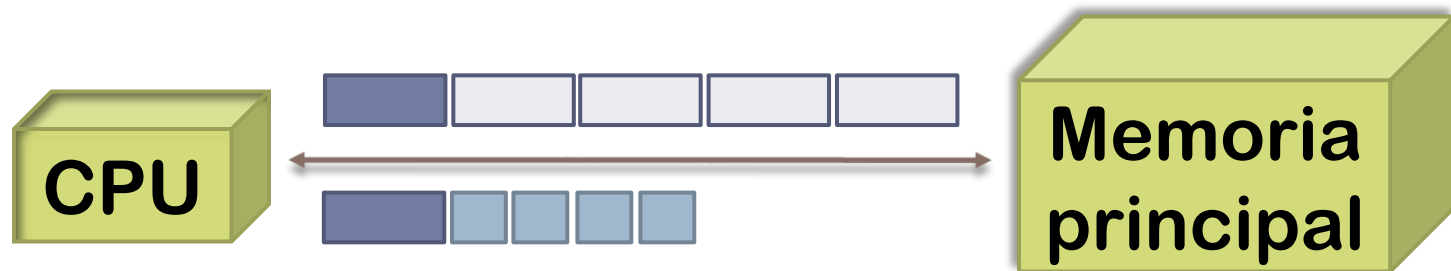
- ▶ Si cuando se accede a una posición de memoria solo se transfieren los datos de esa posición, no se aprovecha los posibles accesos a datos contiguos.



Objetivo de la memoria caché:

aprovechar los accesos contiguos

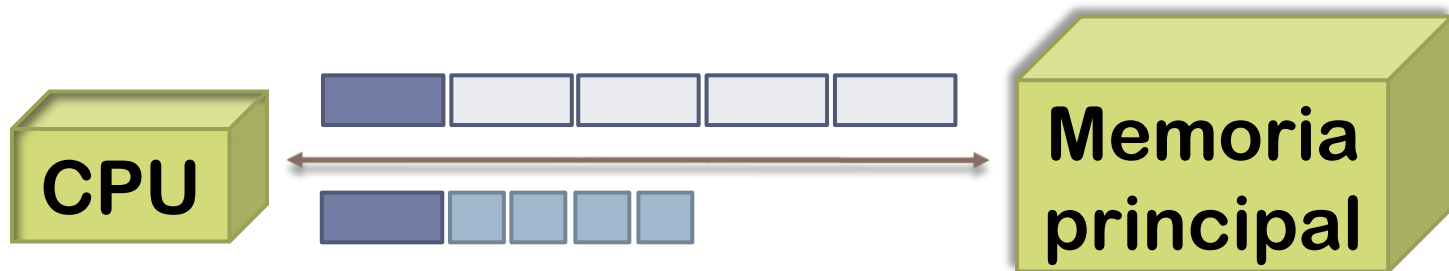
- ▶ Si cuando se accede a una posición de memoria se transfiere esos datos y los contiguos, sí se aprovecha el acceso a datos contiguos



Objetivo de la memoria caché:

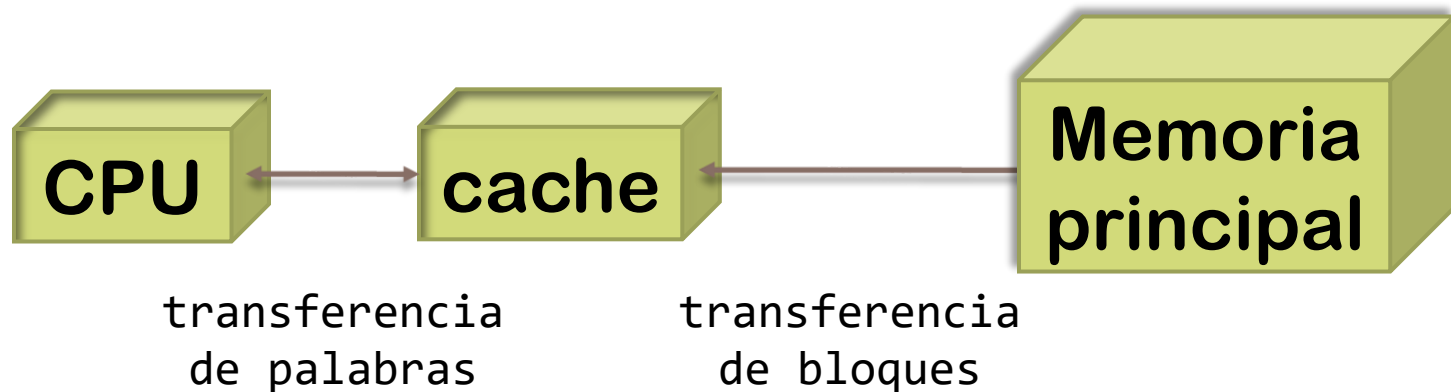
aprovechar los accesos contiguos

- ▶ Si cuando se accede a una posición de memoria se transfiere esos datos y los contiguos, sí se aprovecha el acceso a datos contiguos
 - ▶ Transfiero de la memoria principal un bloque de palabras
 - ▶ ¿Dónde se almacenan las palabras del bloque?



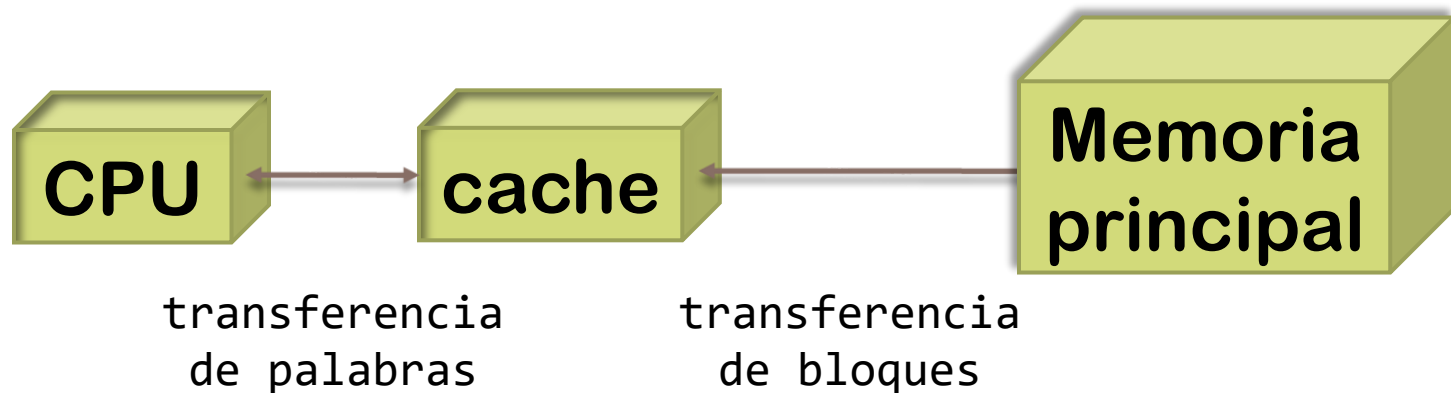
Memoria Cache

- ▶ Cantidad pequeña de memoria rápida SRAM
 - ▶ Más rápida y cara que la memoria principal DRAM
- ▶ Está **entre** la **memoria principal** y el **procesador** (CPU)
 - ▶ Integrada en el mismo procesador normalmente
- ▶ Almacena una **copia** de partes de la memoria principal



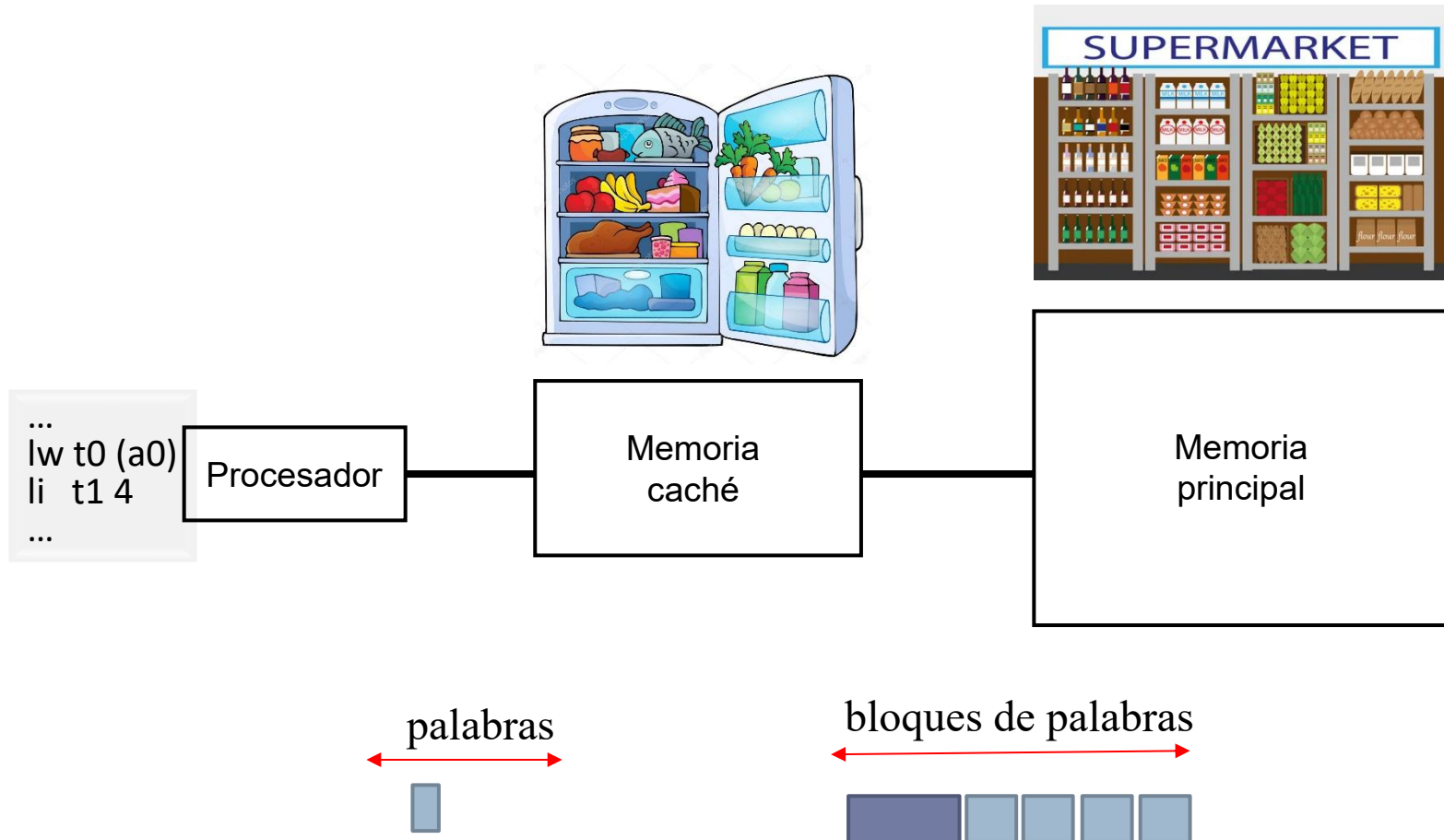
Ejemplo de tiempos de acceso

- ▶ Memoria principal (DRAM o similar)
 - ▶ Tiempo de acceso: entre 30 y 80 ns.
- ▶ Memoria caché (SRAM o similar)
 - ▶ Tiempo de acceso: entre 1 y 2.5 ns.



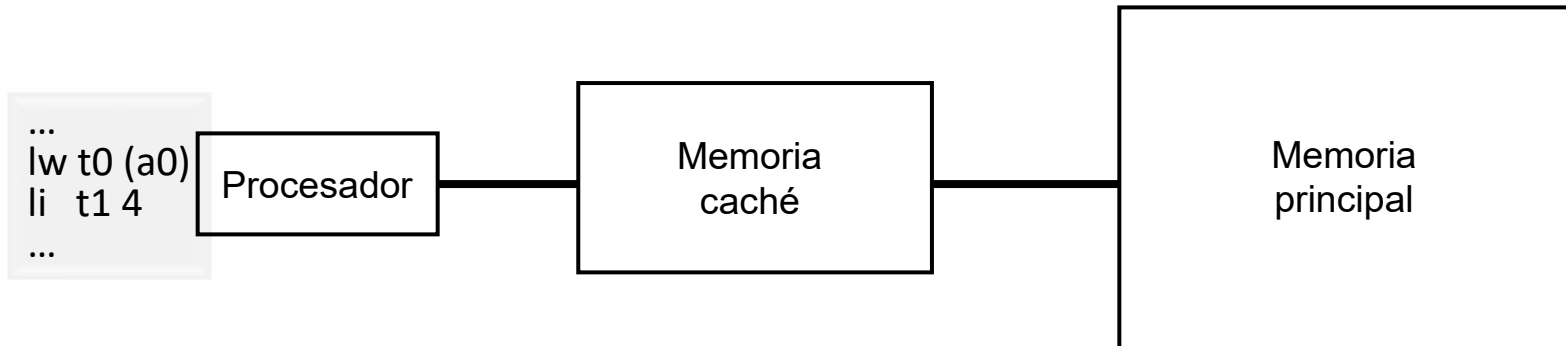
Memoria caché

metáfora de supermercado y el frigorífico



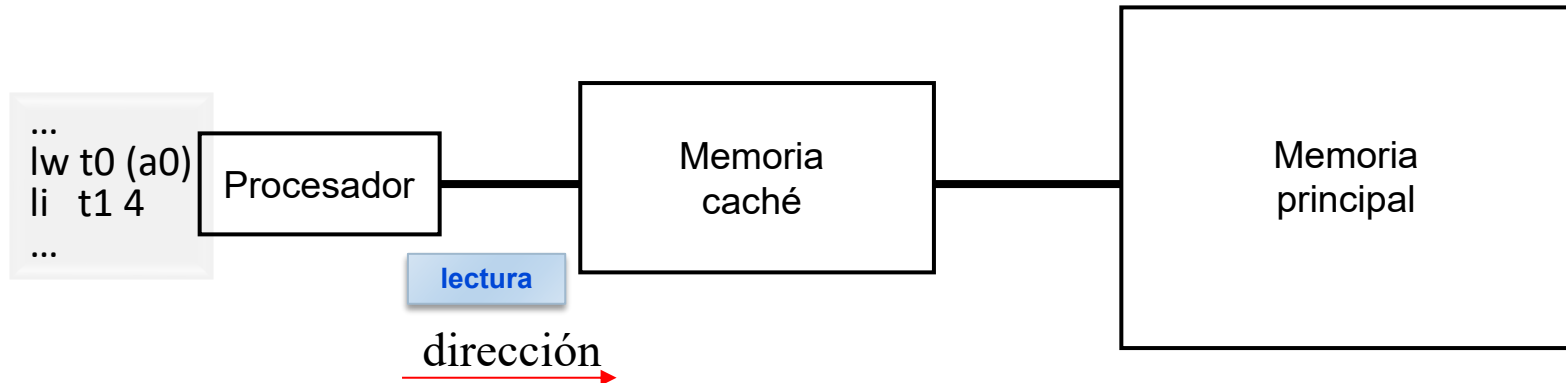
Funcionamiento de la memoria cache

1. El procesador solicita leer/escribir una posición de memoria.
2. La cache comprueba si ya están los datos de esta posición:
 - ▶ Si está (**ACIERTO**)
 - 3.A.1 Se lee de caché y se sirve al procesador / se escribe en cache.
 - ▶ Si no está (**FALLO**)
 - 3.B.1 La cache transfiere de memoria principal el bloque asociado a la posición.
 - 3.B.2 Después, se lee los datos desde caché / se escribe en caché la palabra.



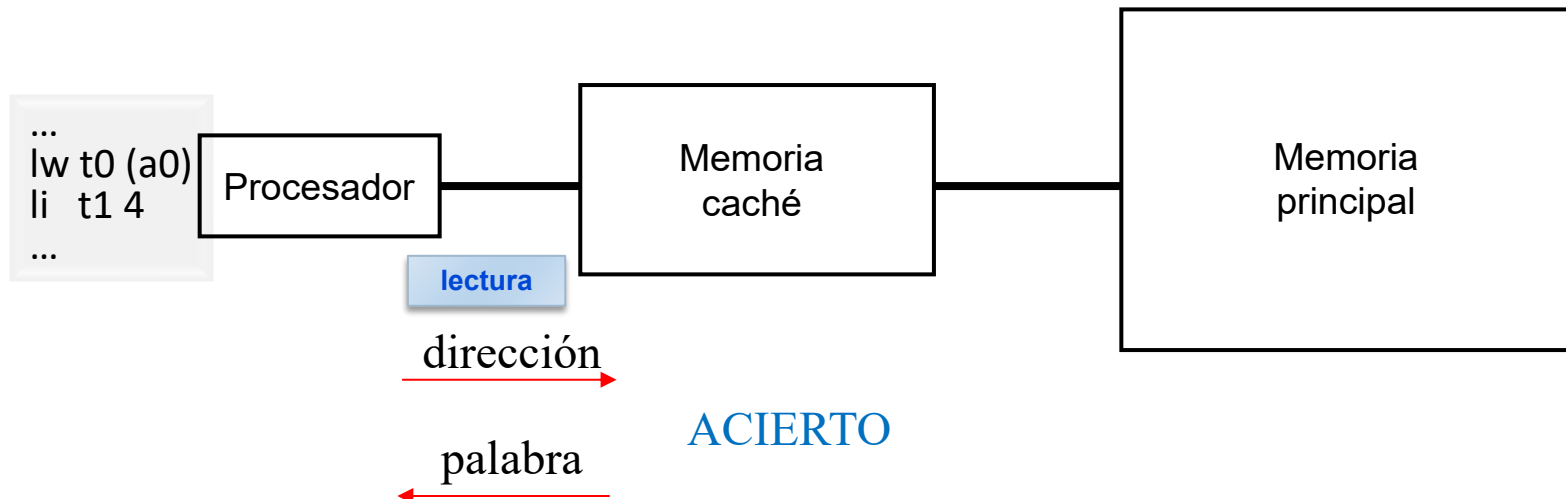
Funcionamiento de la memoria cache

1. El procesador solicita leer/escribir una posición de memoria.
2. La cache comprueba si ya están los datos de esta posición:
 - ▶ Si está (ACIERTO)
 - 3.A.1 Se lee de caché y se sirve al procesador / se escribe en cache.
 - ▶ Si no está (FALLO)
 - 3.B.1 La cache transfiere de memoria principal el bloque asociado a la posición.
 - 3.B.2 Después, se lee los datos desde caché / se escribe en caché la palabra.



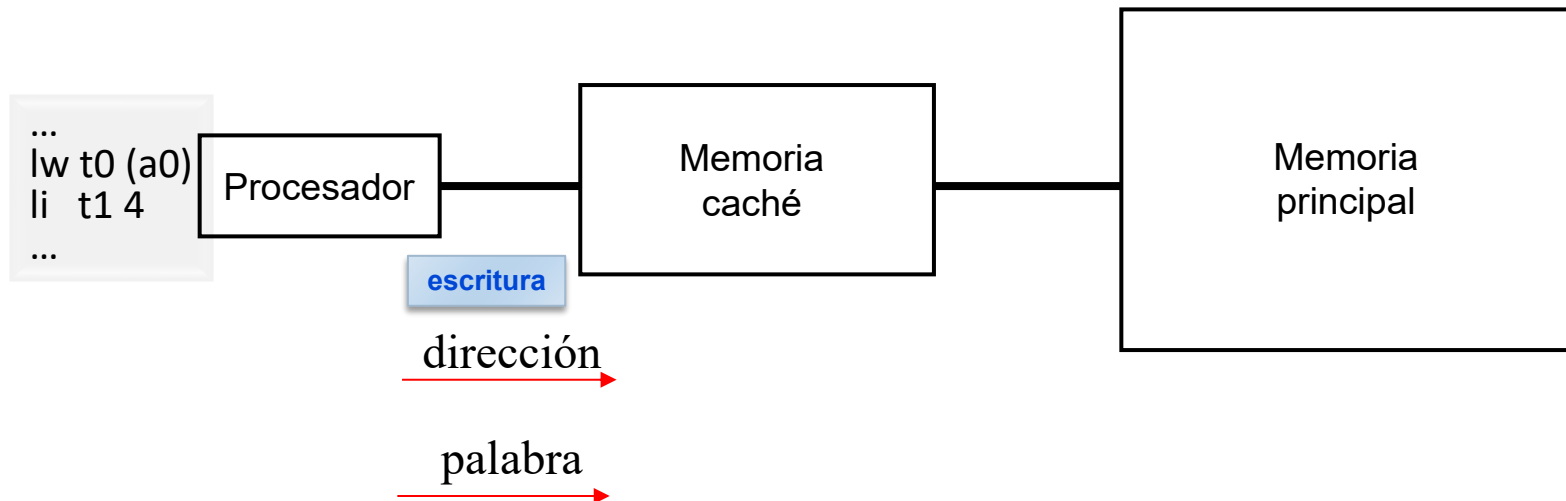
Funcionamiento de la memoria cache

1. El procesador solicita leer/**escribir** una posición de memoria.
2. La cache comprueba si ya están los datos de esta posición:
 - ▶ **Si está (ACIERTO)**
 - 3.A.1 Se lee de caché y se sirve al procesador / **se escribe en cache.**
 - ▶ **Si no está (FALLO)**
 - 3.B.1 La cache transfiere de memoria principal el bloque asociado a la posición.
 - 3.B.2 Después, se lee los datos desde caché / **se escribe en caché la palabra.**



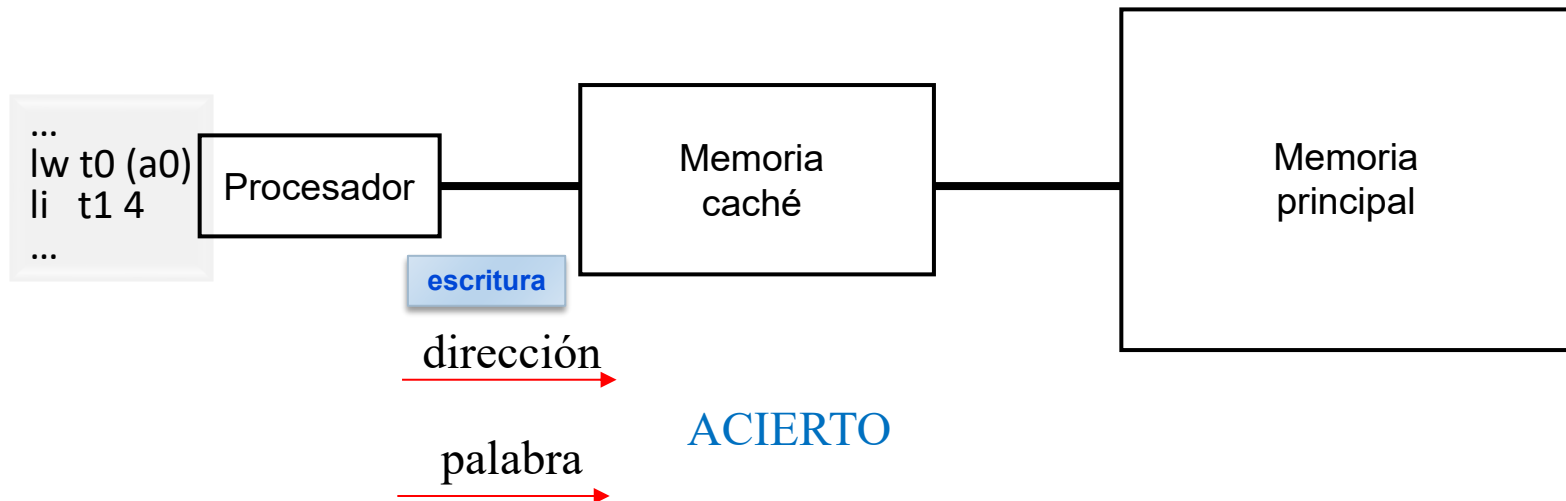
Funcionamiento de la memoria cache

1. El procesador solicita leer/escribir una posición de memoria.
2. La cache comprueba si ya están los datos de esta posición:
 - ▶ Si está (ACIERTO)
 - 3.A.1 Se lee de caché y se sirve al procesador / se escribe en cache.
 - ▶ Si no está (FALLO)
 - 3.B.1 La cache transfiere de memoria principal el bloque asociado a la posición.
 - 3.B.2 Después, se lee los datos desde caché / se escribe en caché la palabra.



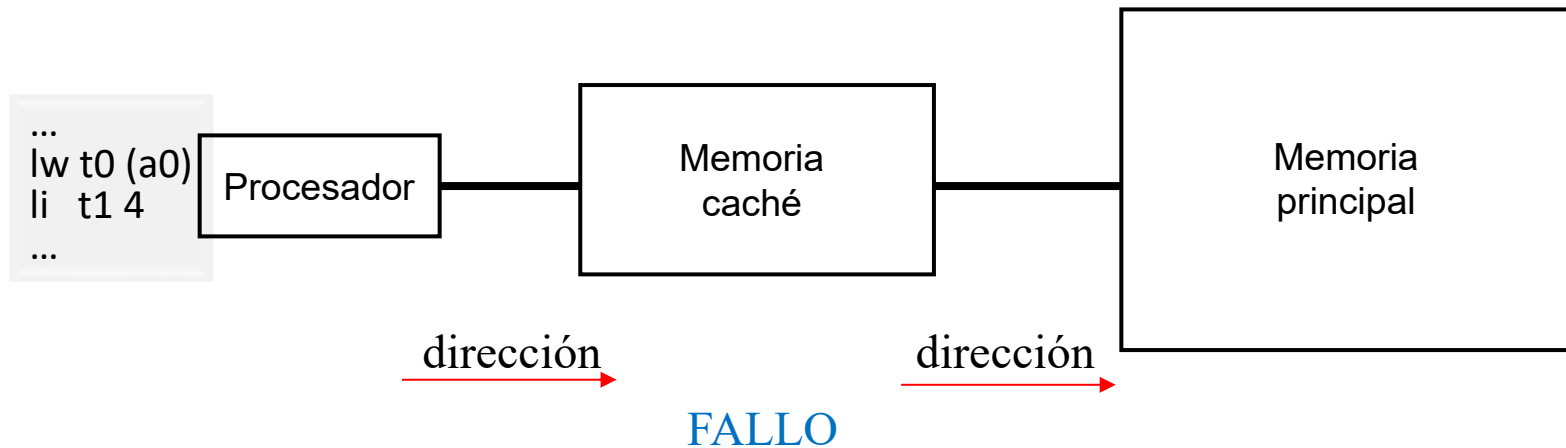
Funcionamiento de la memoria cache

1. El procesador solicita leer/escribir una posición de memoria.
2. La cache comprueba si ya están los datos de esta posición:
 - ▶ **Si está (ACIERTO)**
 - 3.A.1 Se lee de caché y se sirve al procesador / se escribe en cache.
 - ▶ **Si no está (FALLO)**
 - 3.B.1 La cache transfiere de memoria principal el bloque asociado a la posición.
 - 3.B.2 Después, se lee los datos desde caché / se escribe en caché la palabra.



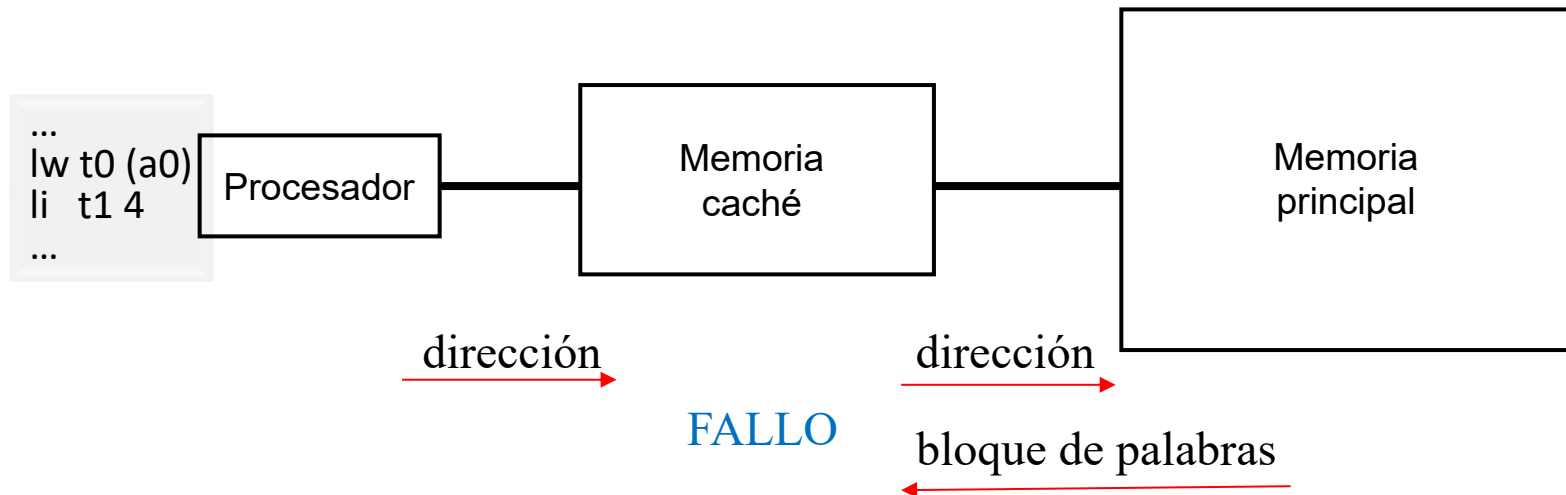
Funcionamiento de la memoria cache

1. El procesador solicita leer/escribir una posición de memoria.
2. La cache comprueba si ya están los datos de esta posición:
 - ▶ Si está (**ACIERTO**)
 - 3.A.1 Se lee de caché y se sirve al procesador / se escribe en cache.
 - ▶ Si no está (**FALLO**)
 - 3.B.1 La cache transfiere de memoria principal el bloque asociado a la posición.
 - 3.B.2 Después, se lee los datos desde caché / se escribe en caché la palabra.



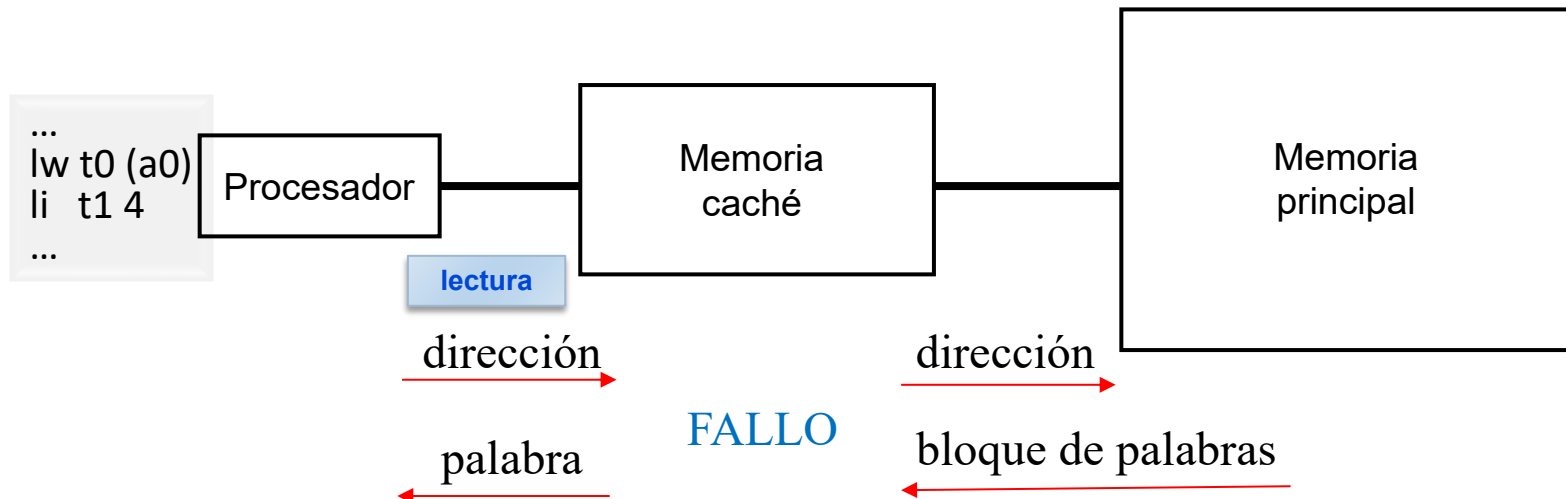
Funcionamiento de la memoria cache

1. El procesador solicita leer/escribir una posición de memoria.
2. La cache comprueba si ya están los datos de esta posición:
 - ▶ Si está (**ACIERTO**)
 - 3.A.1 Se lee de caché y se sirve al procesador / se escribe en cache.
 - ▶ Si no está (**FALLO**)
 - 3.B.1 La cache transfiere de memoria principal el bloque asociado a la posición.
 - 3.B.2 Después, se lee los datos desde caché / se escribe en caché la palabra.



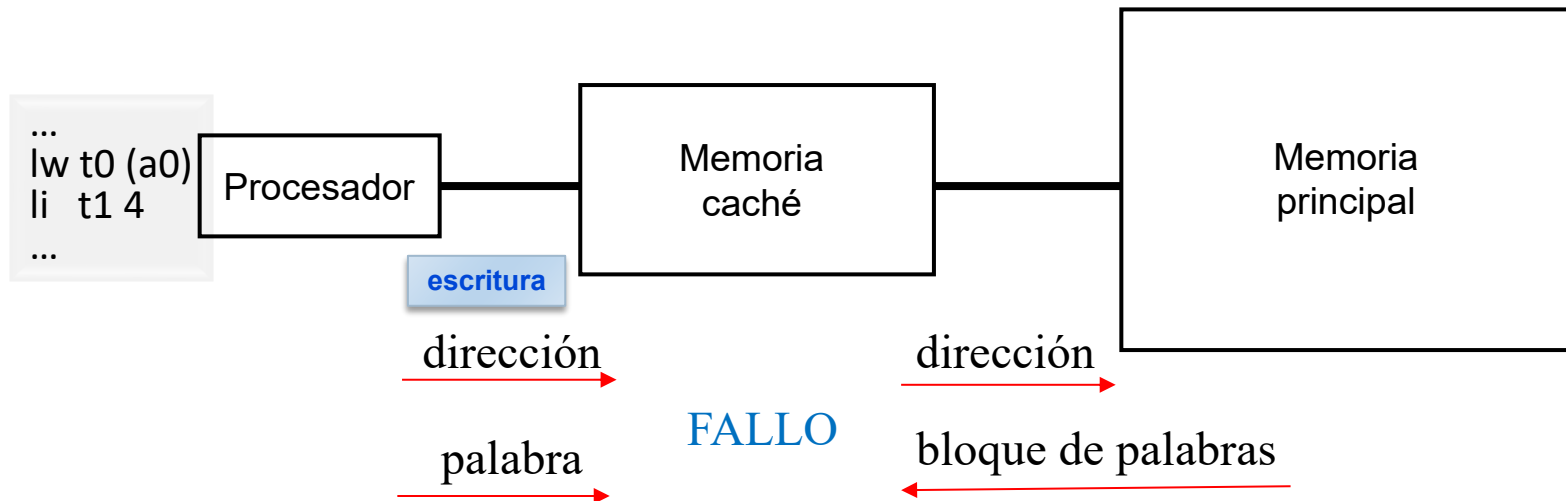
Funcionamiento de la memoria cache

1. El procesador solicita leer/escribir una posición de memoria.
2. La cache comprueba si ya están los datos de esta posición:
 - ▶ Si está (**ACIERTO**)
 - 3.A.1 Se lee de caché y se sirve al procesador / se escribe en cache.
 - ▶ Si no está (**FALLO**)
 - 3.B.1 La cache transfiere de memoria principal el bloque asociado a la posición.
 - 3.B.2 Después, se lee los datos desde caché / se escribe en caché la palabra.



Funcionamiento de la memoria cache

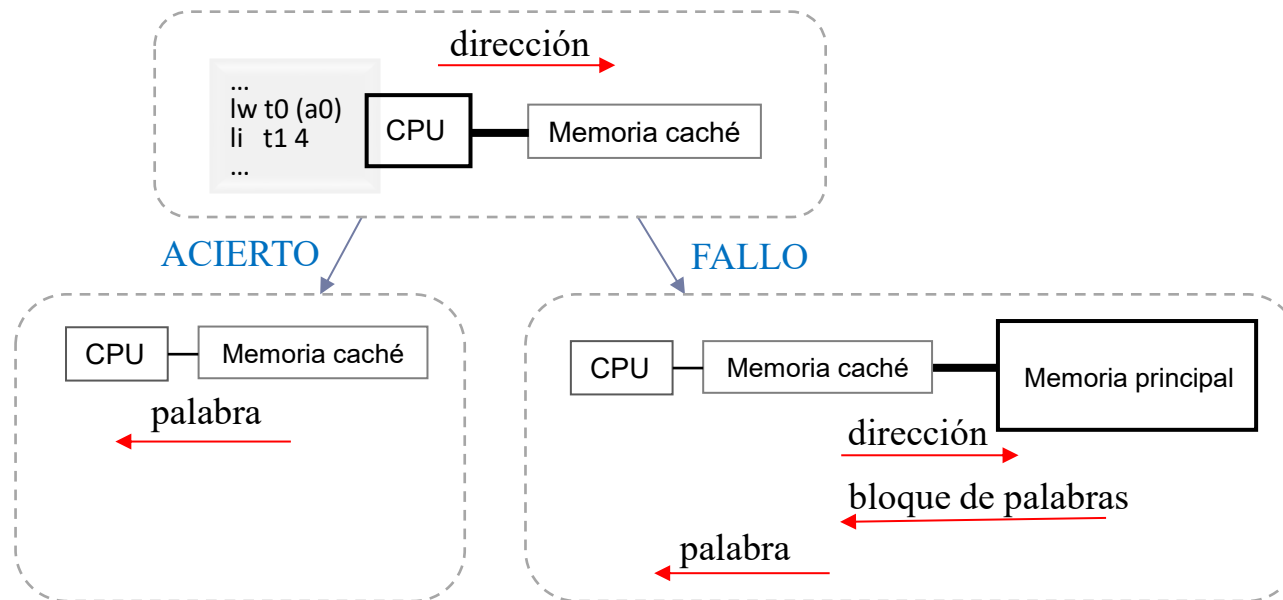
1. El procesador solicita leer/escribir una posición de memoria.
2. La cache comprueba si ya están los datos de esta posición:
 - ▶ Si está (**ACIERTO**)
 - 3.A.1 Se lee de caché y se sirve al procesador / se escribe en cache.
 - ▶ Si no está (**FALLO**)
 - 3.B.1 La cache transfiere de memoria principal el bloque asociado a la posición.
 - 3.B.2 Después, se lee los datos desde caché / se escribe en caché la palabra.



Funcionamiento de la memoria caché

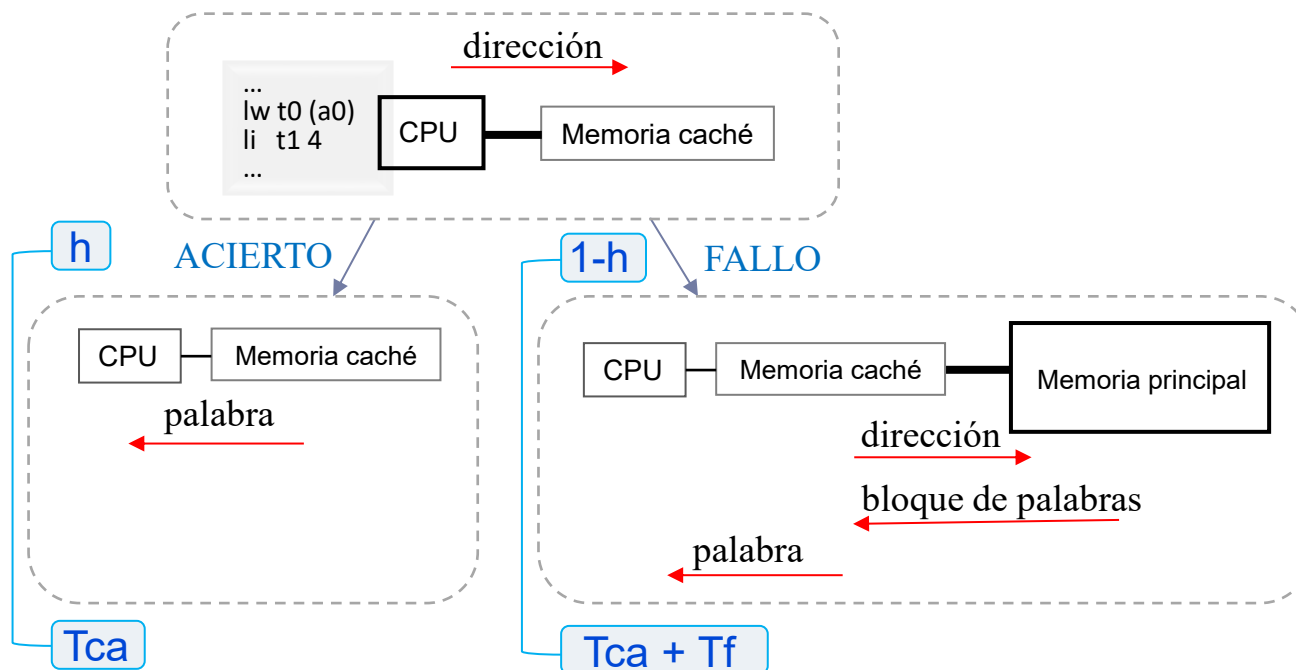
Resumen

1. El procesador solicita leer/escribir una posición de memoria.
2. La cache comprueba si ya están los datos de esta posición:
 - ▶ **Si está (ACIERTO)**
 - 3.A.1** Se lee de caché y se sirve al procesador / se escribe en cache.
 - ▶ **Si no está (FALLO)**
 - 3.B.1** La cache transfiere de memoria principal el bloque asociado a la posición.
 - 3.B.2** Después, se lee los datos desde caché / se escribe en caché la palabra.



Funcionamiento de la memoria caché

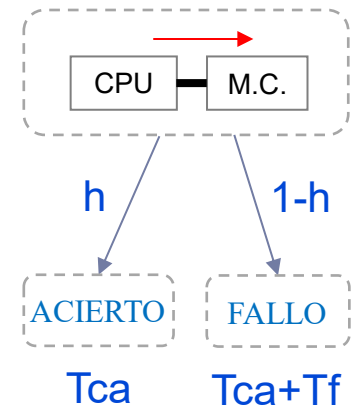
1. El procesador solicita leer/escribir una posición de memoria.
2. La cache comprueba si ya están los datos de esta posición:
 - ▶ **Si está (ACIERTO)**
 - 3.A.1** Se lee de caché y se sirve al procesador / se escribe en cache.
 - ▶ **Si no está (FALLO)**
 - 3.B.1** La cache transfiere de memoria principal el bloque asociado a la posición.
 - 3.B.2** Después, se lee los datos desde caché / se escribe en caché la palabra.



Tiempo medio de acceso a caché

- ▶ Tiempo medio de acceso a un sistema de memoria con dos niveles:

$$\begin{aligned} T_m &= h \cdot T_a + (1-h) \cdot (T_a + T_f) \\ &= T_a + (1-h) \cdot T_f \end{aligned}$$



- ▶ **Ta**: tiempo de acceso a la caché
- ▶ **Tf**: tiempo en tratar el fallo
 - ▶ Incluye el tiempo en remplazar un bloque, traer un bloque de memoria principal a caché, etc.
- ▶ **h**: tasa de aciertos

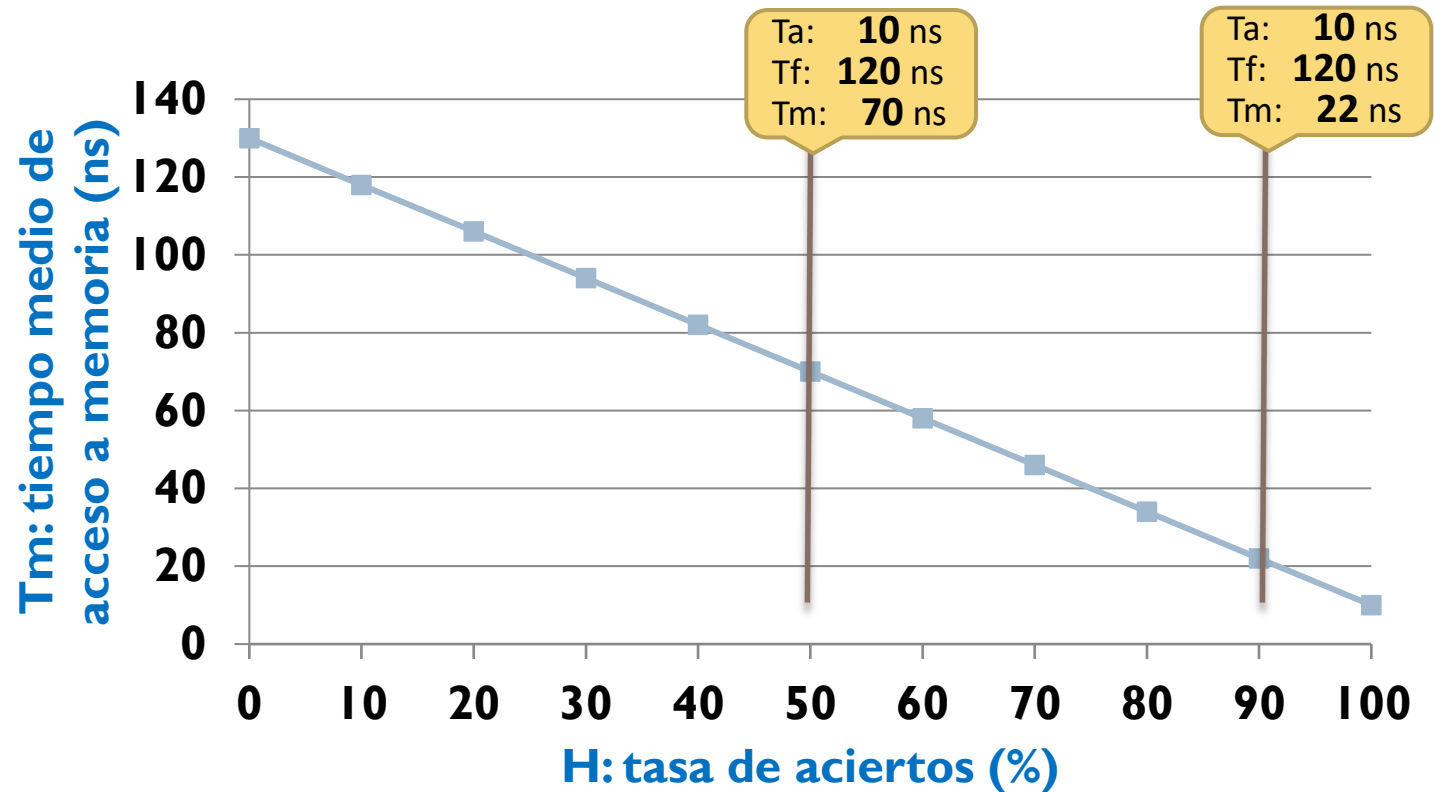
Ejemplo

$$\begin{aligned} T_m &= h \cdot T_a + (1-h) \cdot (T_a + T_f) \\ &= T_a + (1-h) \cdot T_f \end{aligned}$$

1. T_a : Tiempo de acceso a caché = **10 ns**
2. T_f : Tiempo de acceso a memoria principal = **120 ns**
3. h : tasa de aciertos $\rightarrow X = 0.1, 0.2, \dots, 0.9, 1.0$
10%, 20%, ..., 90%, 100%

Ejemplo

$$\begin{aligned} T_m &= h \cdot T_a + (1-h) \cdot (T_a + T_f) \\ &= T_a + (1-h) \cdot T_f \end{aligned}$$



Ejercicio

- ▶ **Computador:**
 - ▶ Tiempo de acceso a caché: 4 ns
 - ▶ Tiempo de acceso a bloque de MP: 120 ns.
- ▶ Si se tiene una tasa de aciertos del 90%,
¿Cuál es el tiempo medio de acceso?
- ▶ Tasas de acierto necesarias para que el tiempo medio de acceso sea menor de 5 ns.

Ejercicio (solución)

- ▶ **Computador:**

- ▶ Tiempo de acceso a caché: 4 ns
- ▶ Tiempo de acceso a bloque de MP: 120 ns.

- ▶ Si se tiene una tasa de aciertos del 90%,
¿Cuál es el tiempo medio de acceso?

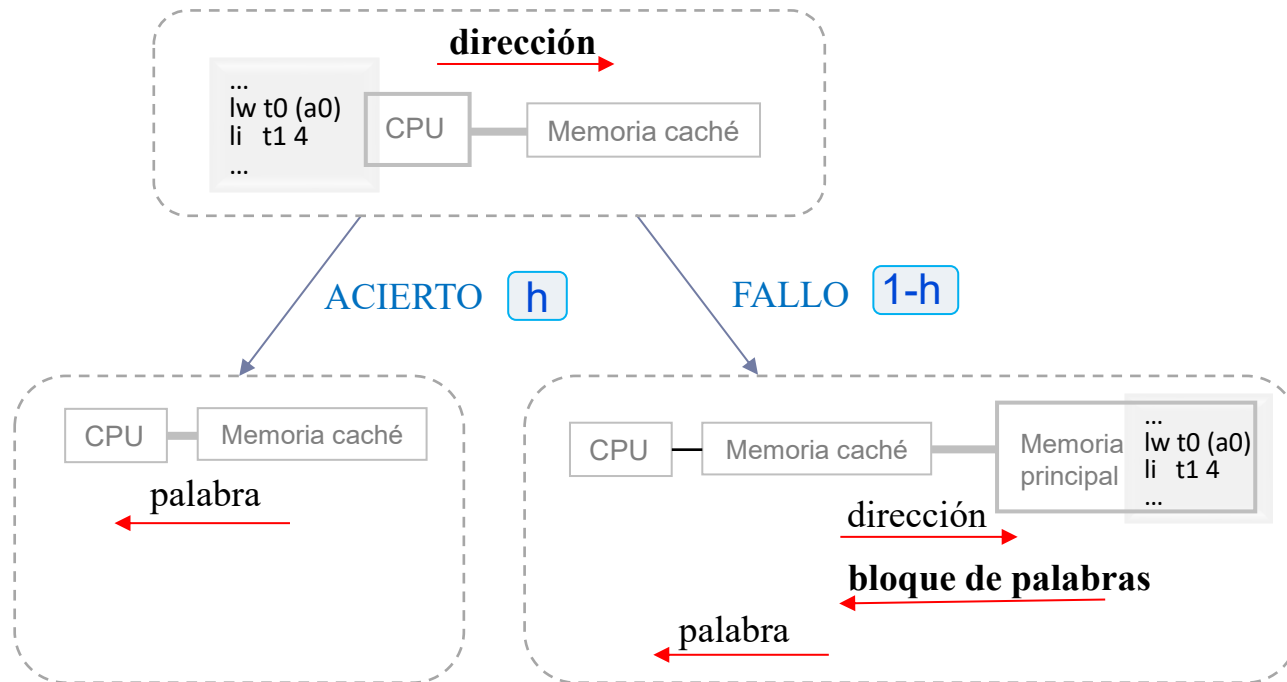
$$T_m = 4 \times 0.9 + (120 + 4) \times 0.1 = 16 \text{ ns}$$

- ▶ Tasas de acierto necesarias para que el tiempo medio de acceso sea menor de 5 ns.

$$\begin{aligned} 5 &= 4 \times h + (120 + 4) \times (1 - h) \\ \Rightarrow h &> 0.9916 \end{aligned}$$

Tasa de aciertos de un fragmento de código

- ▶ La tasa de aciertos h depende de:
 - ▶ Disposición en memoria ppal. y caché (bloques afectados).
 - ▶ Traza de accesos (lista de direcciones) que genera al ejecutarse.
 - ▶ El comportamiento de la memoria caché (tiempo de búsqueda, remplazo, etc.)



Ejemplo de funcionamiento

```
int i;  
int s = 0;  
for (i=0; i < 1000; i++)  
    s = s + i;
```

```
li    t0, 0      # s  
li    t1, 0      # i  
li    t2, 1000  
bucle: bge    t1, t2, fin  
add   t0, t0, t1  
addi  t1, t1, 1  
j     bucle  
fin:  ...
```

- ▶ Ejemplo:
 - ▶ Acceso a M.caché: 2 ns
 - ▶ Acceso a M.P.: 120 ns
 - ▶ Bloque de caché: 4 palabras
 - ▶ Transferencia de un bloque entre memoria principal y caché: 200 ns

Ejemplo de funcionamiento

```
int i;  
int s = 0;  
for (i=0; i < 1000; i++)  
    s = s + i;
```

```
li    t0, 0    # s  
li    t1, 0    # i  
li    t2, 1000  
bucle: bge    t1, t2, fin  
add   t0, t0, t1  
addi  t1, t1, 1  
j     bucle  
fin:  ...
```

► Sin memoria caché:

- Número de accesos a memoria = $3 + 4 \times 1000 + 1 = 4004$ accesos
- Tiempo de acceso a memoria = $4004 \times 120 = 480480$ ns = 0,480 ms

Ejemplo de funcionamiento

```
int i;  
int s = 0;  
for (i=0; i < 1000; i++)  
    s = s + i;
```

```
li    t0, 0      # s  
li    t1, 0      # i  
li    t2, 1000  
bucle: bge    t1, t2, fin  
add   t0, t0, t1  
addi  t1, t1, 1  
j     bucle  
fin:  ...
```

- ▶ **Con** memoria caché (bloque de 4 palabras):
 - ▶ Número de accesos = 4004 accesos
 - ▶ Número de bloques = ?
 - ▶ Número de fallos = ?
 - ▶ Tiempo de acceso = ?

Ejemplo de funcionamiento

```
int i;  
int s = 0;  
for (i=0; i < 1000; i++)  
    s = s + i;
```

bucle:

```
li    t0, 0      # s  
li    t1, 0      # i  
li    t2, 1000  
bge   t1, t2, fin
```

fin:

```
add   t0, t0, t1  
addi  t1, t1, 1  
j     bucle  
...
```

► Con memoria caché (bloque de 4 palabras):

- Número de accesos = 4004 accesos
- Número de bloques = 2
- Número de fallos = ?
- Tiempo de acceso = ?

(1/2) estudio de bloques:
análisis de bloques de datos y
código afectados

Ejemplo de funcionamiento

```
int i;  
int s = 0;  
for (i=0; i < 1000; i++)  
    s = s + i;
```

bucle:

```
li    t0, 0    # s  
li    t1, 0    # i  
li    t2, 1000  
bge   t1, t2, fin
```

fin:

```
add   t0, t0, t1  
addi  t1, t1, 1  
j     bucle  
...
```

► **Con** memoria caché (bloque de 4 palabras):

- Número de accesos = 4004 accesos
- Número de bloques = 2
- Número de fallos = ?
- Tiempo de acceso = ?

(2/2) estudio de referencias
generadas por ejecución:
accesos a código (fetch) y datos

Ejemplo de funcionamiento

(2/2) estudio de referencias generadas

procesador

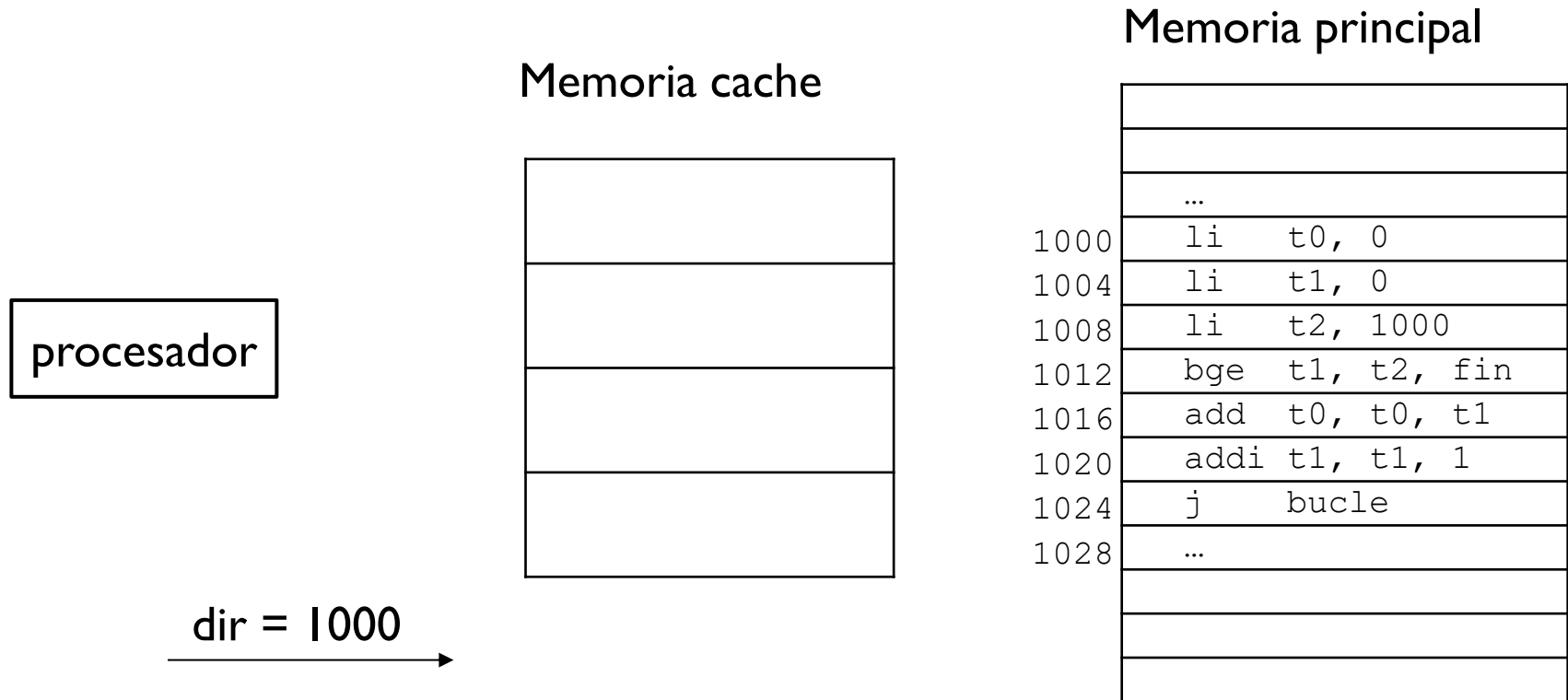
Memoria cache

Memoria principal

	...
1000	li t0, 0
1004	li t1, 0
1008	li t2, 1000
1012	bge t1, t2, fin
1016	add t0, t0, t1
1020	addi t1, t1, 1
1024	j bucle
1028	...

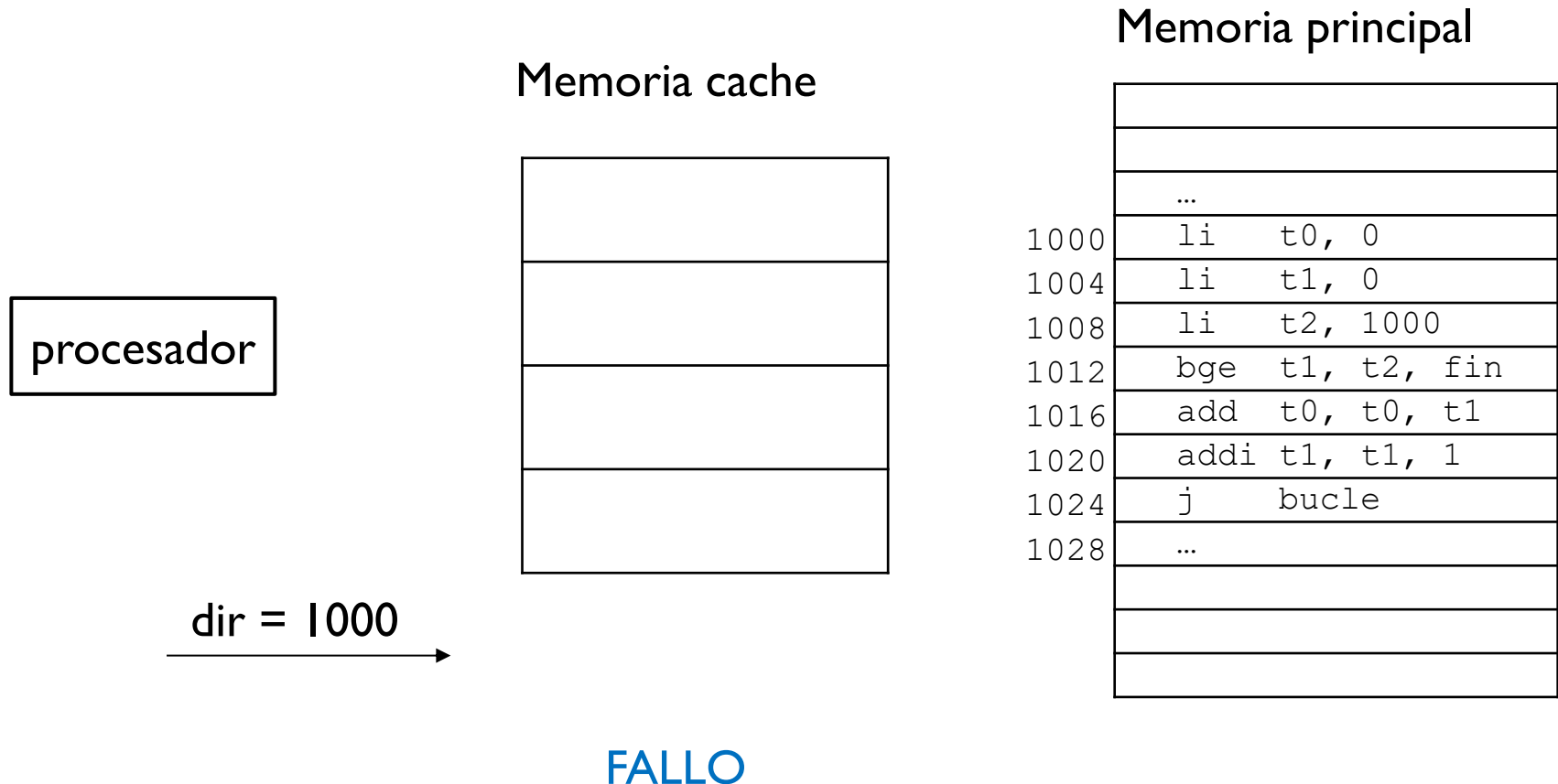
Ejemplo de funcionamiento

(2/2) estudio de referencias generadas



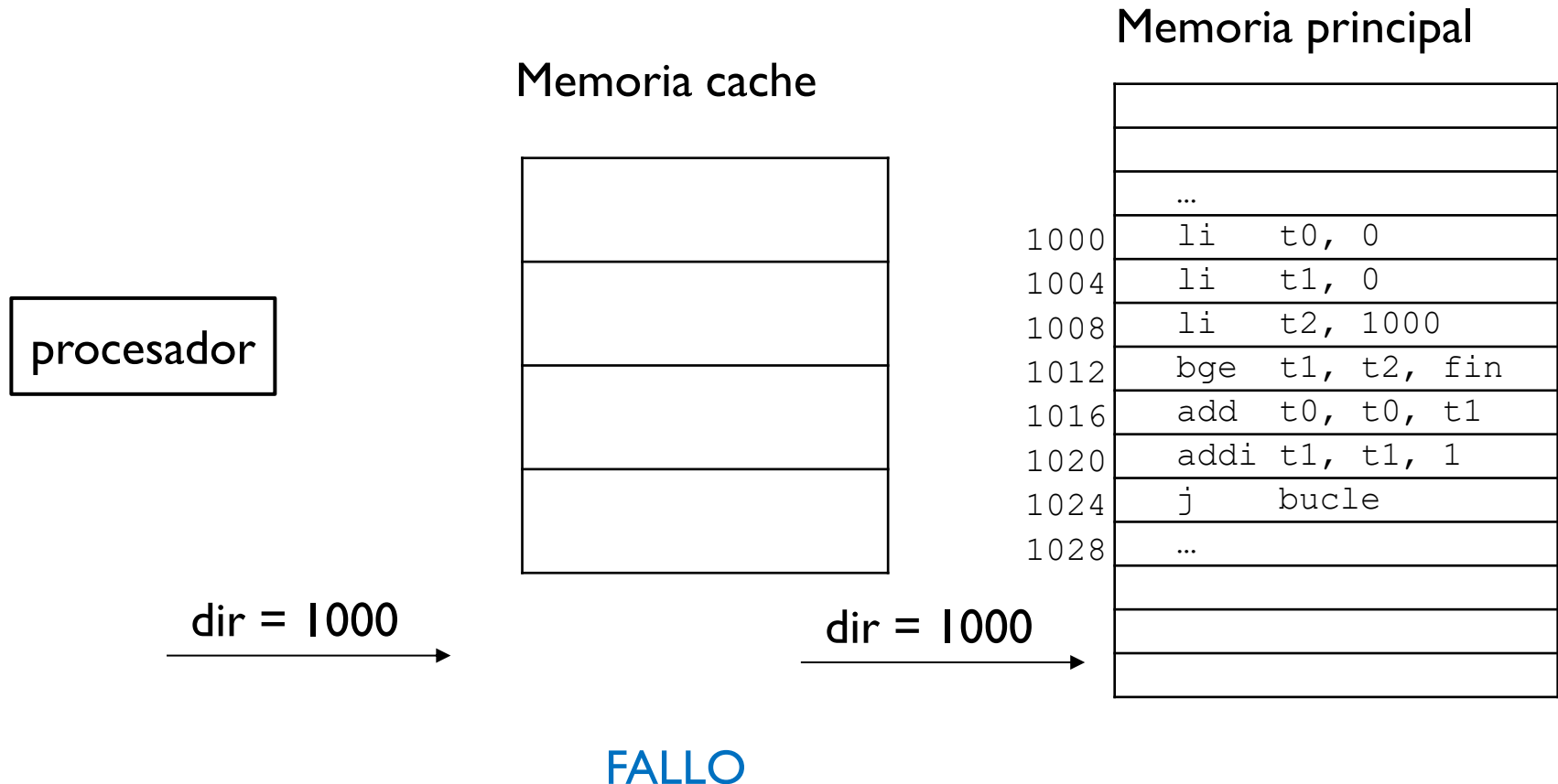
Ejemplo de funcionamiento

(2/2) estudio de referencias generadas



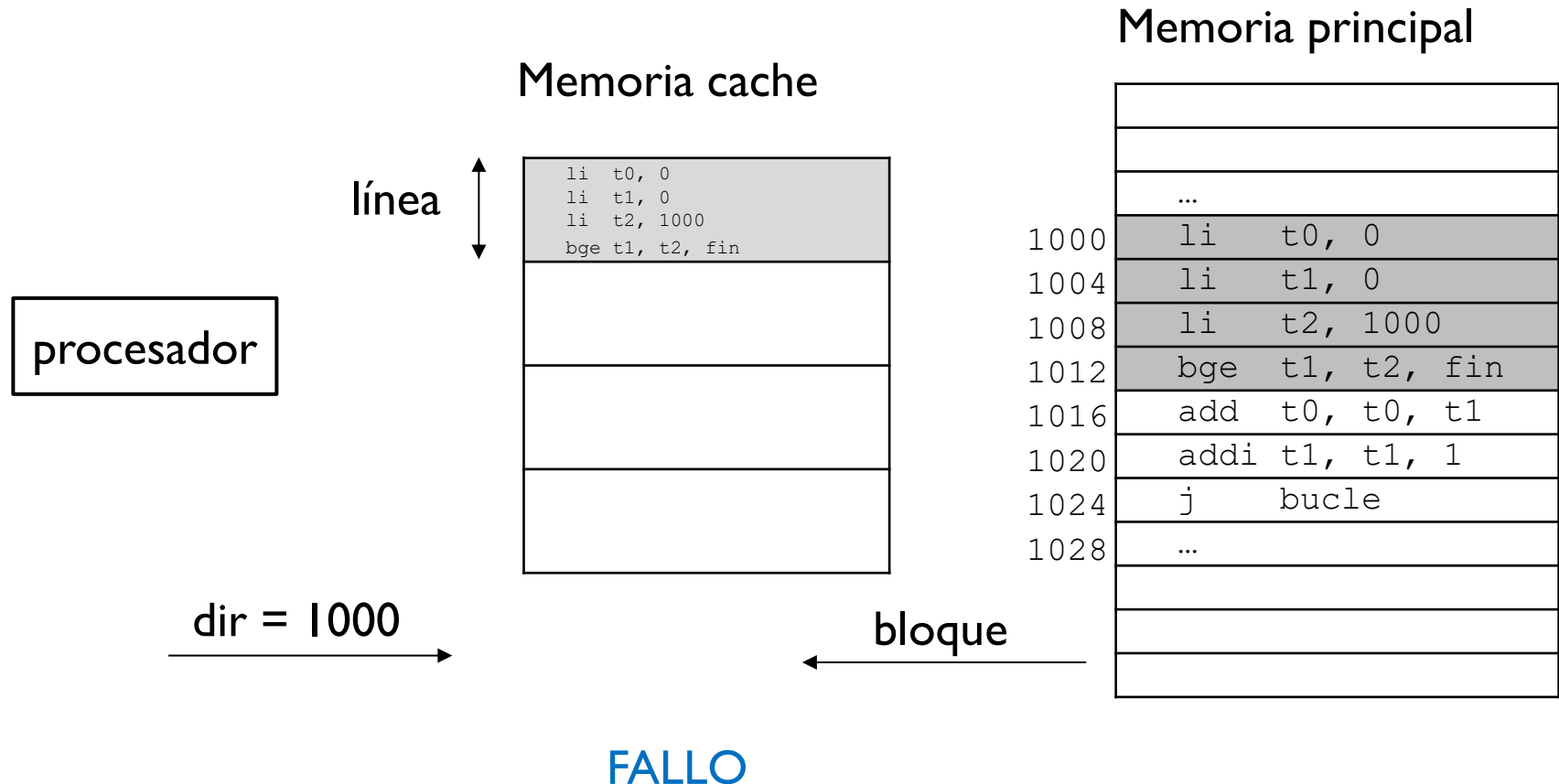
Ejemplo de funcionamiento

(2/2) estudio de referencias generadas



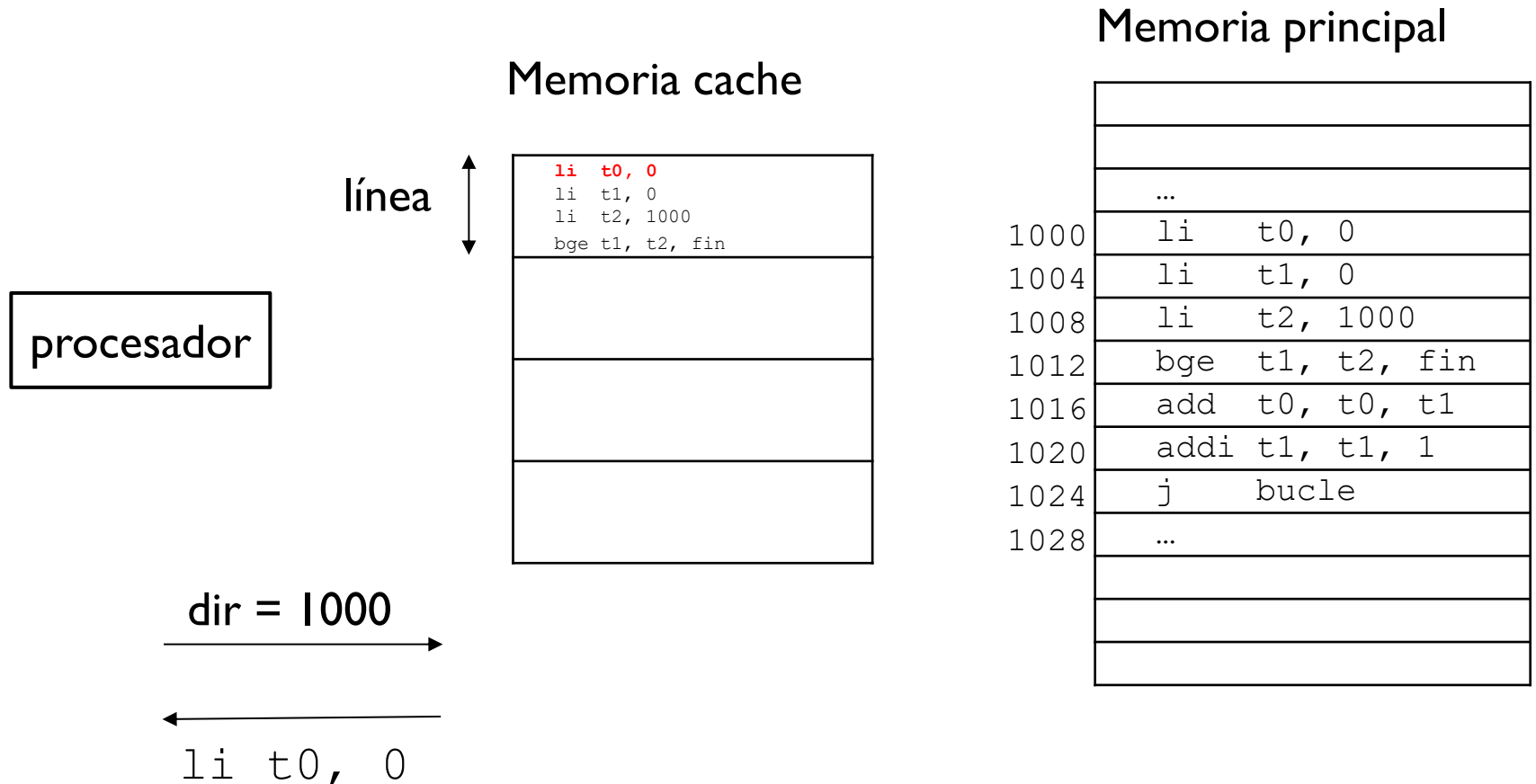
Ejemplo de funcionamiento

(2/2) estudio de referencias generadas



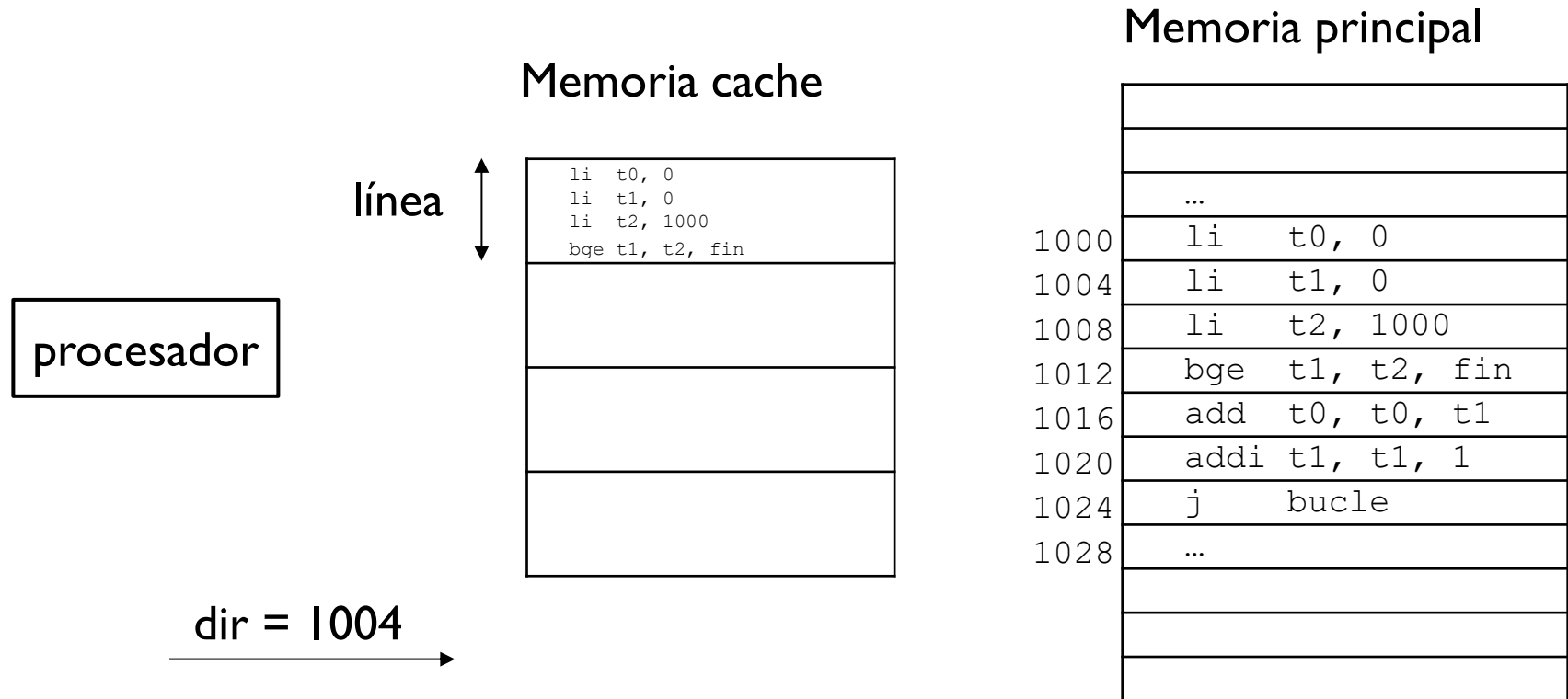
Ejemplo de funcionamiento

(2/2) estudio de referencias generadas



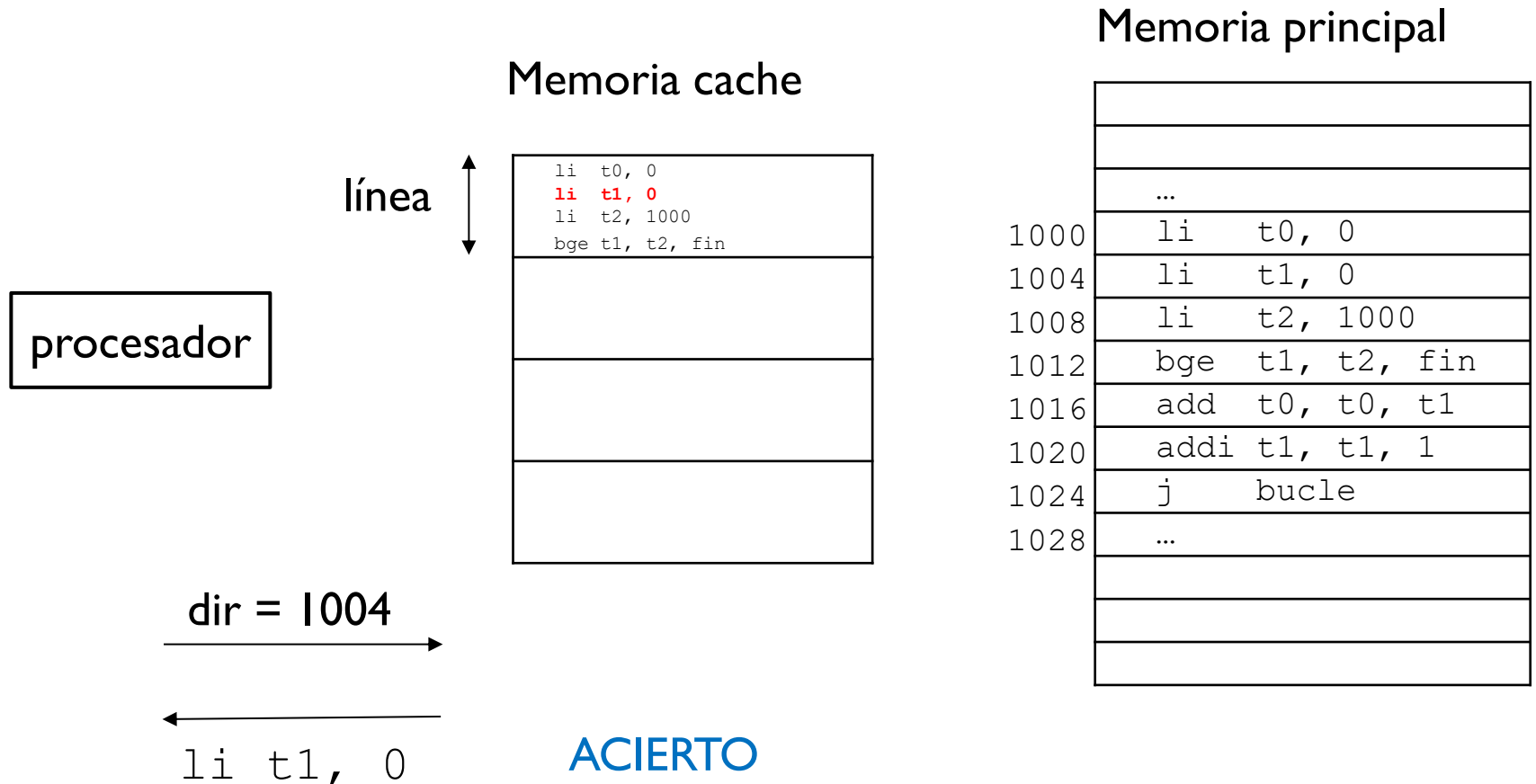
Ejemplo de funcionamiento

(2/2) estudio de referencias generadas



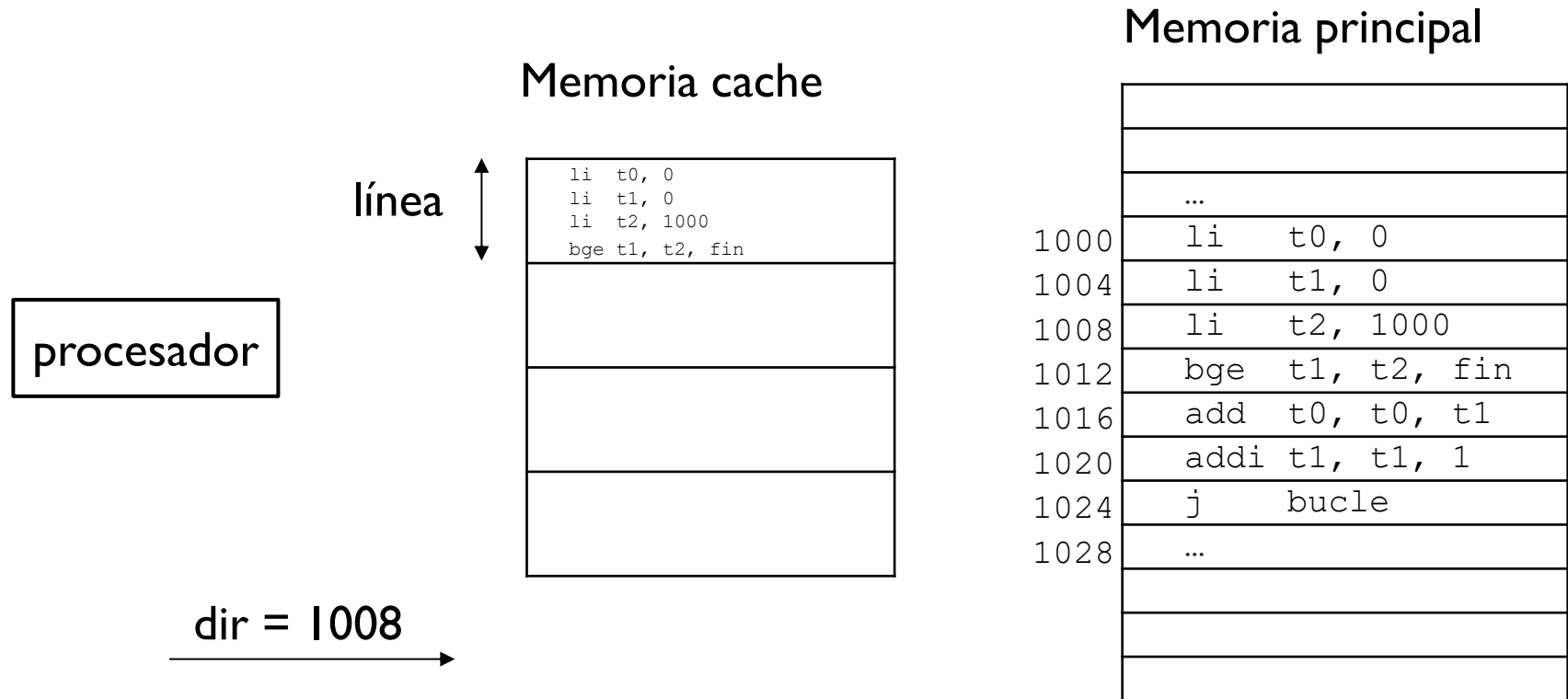
Ejemplo de funcionamiento

(2/2) estudio de referencias generadas



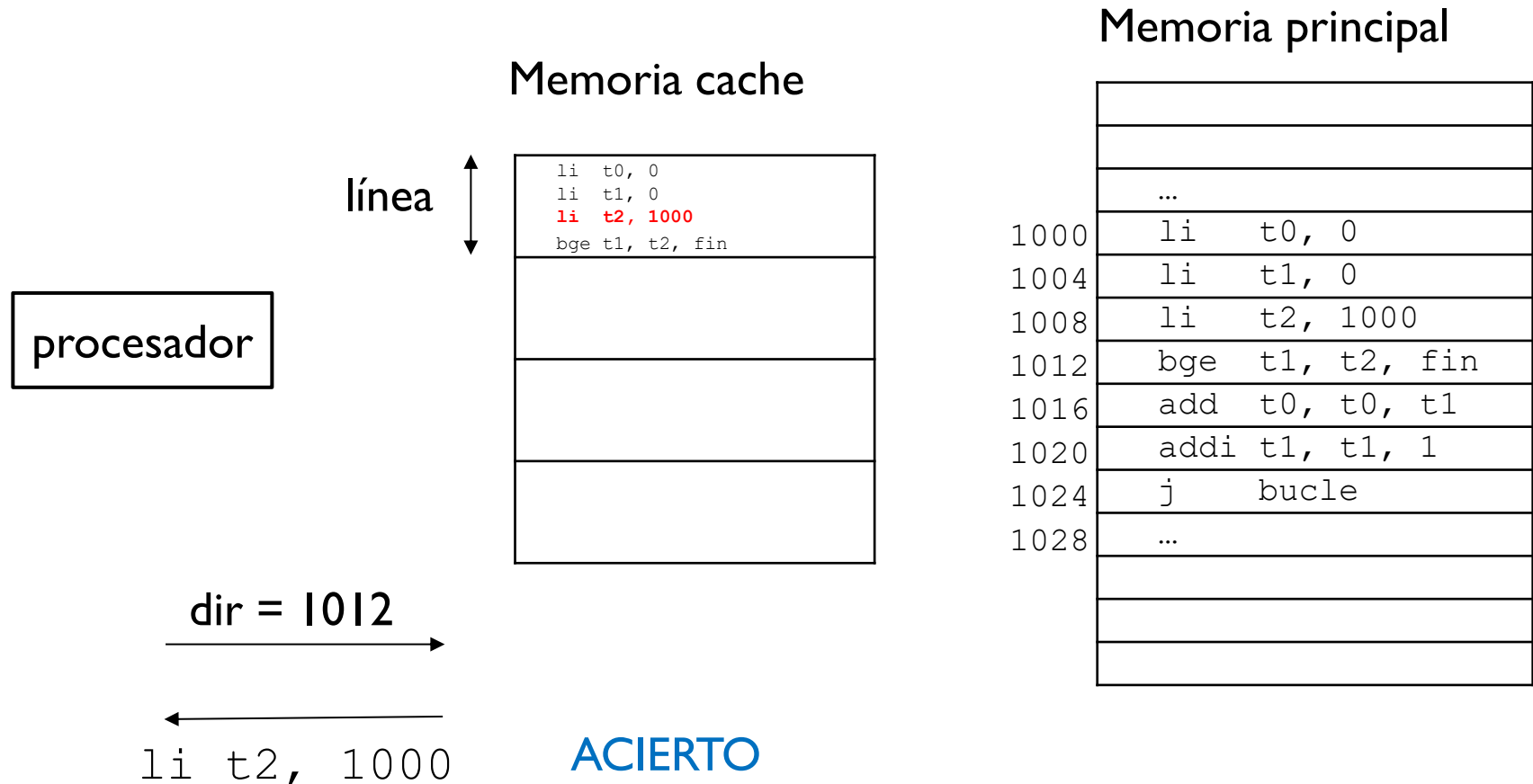
Ejemplo de funcionamiento

(2/2) estudio de referencias generadas



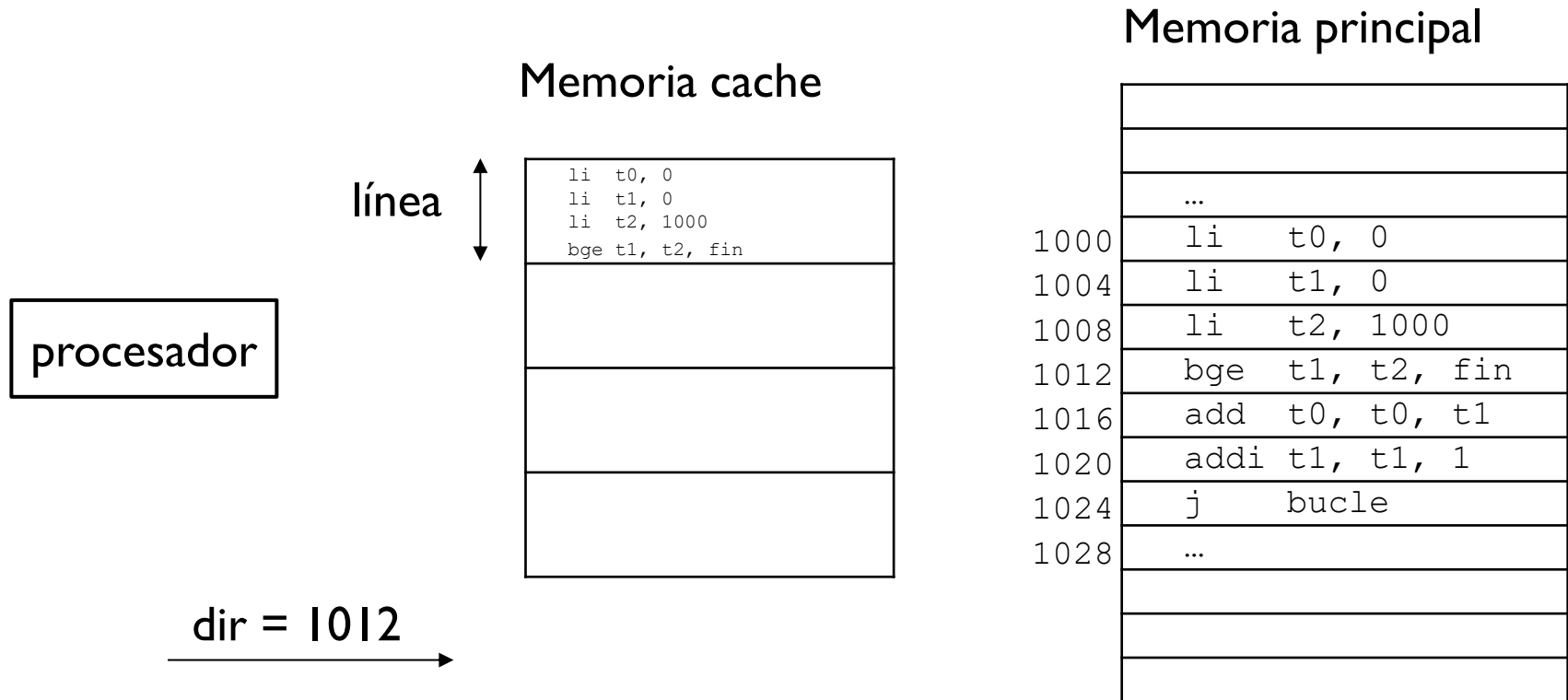
Ejemplo de funcionamiento

(2/2) estudio de referencias generadas



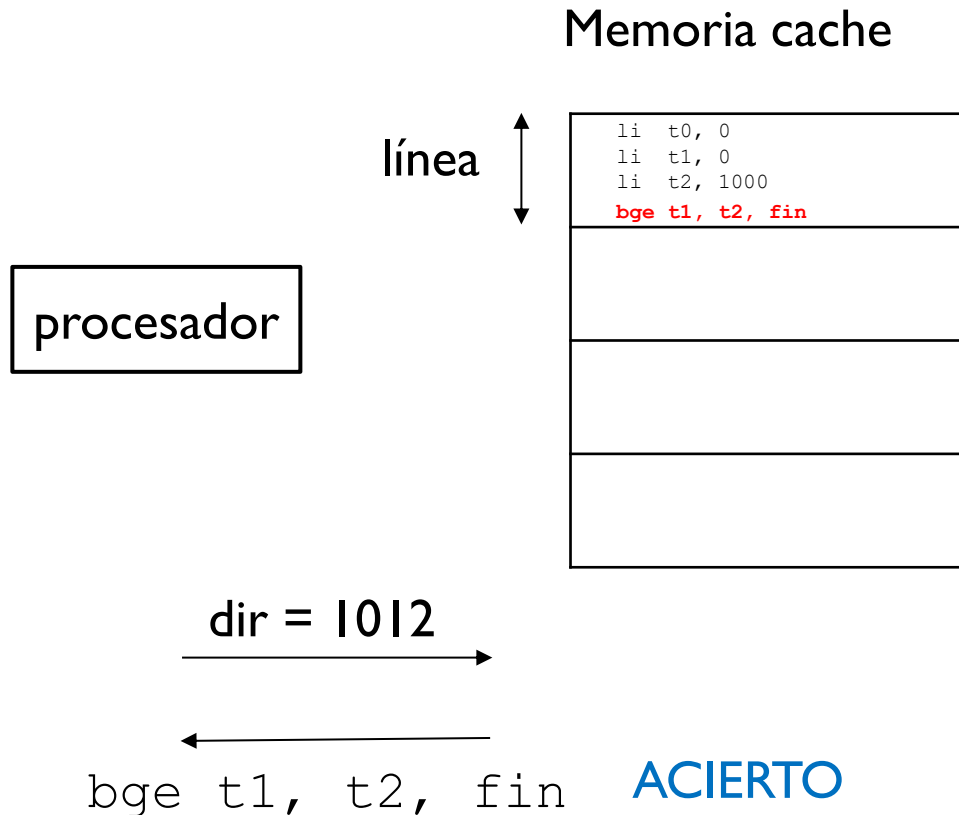
Ejemplo de funcionamiento

(2/2) estudio de referencias generadas



Ejemplo de funcionamiento

(2/2) estudio de referencias generadas

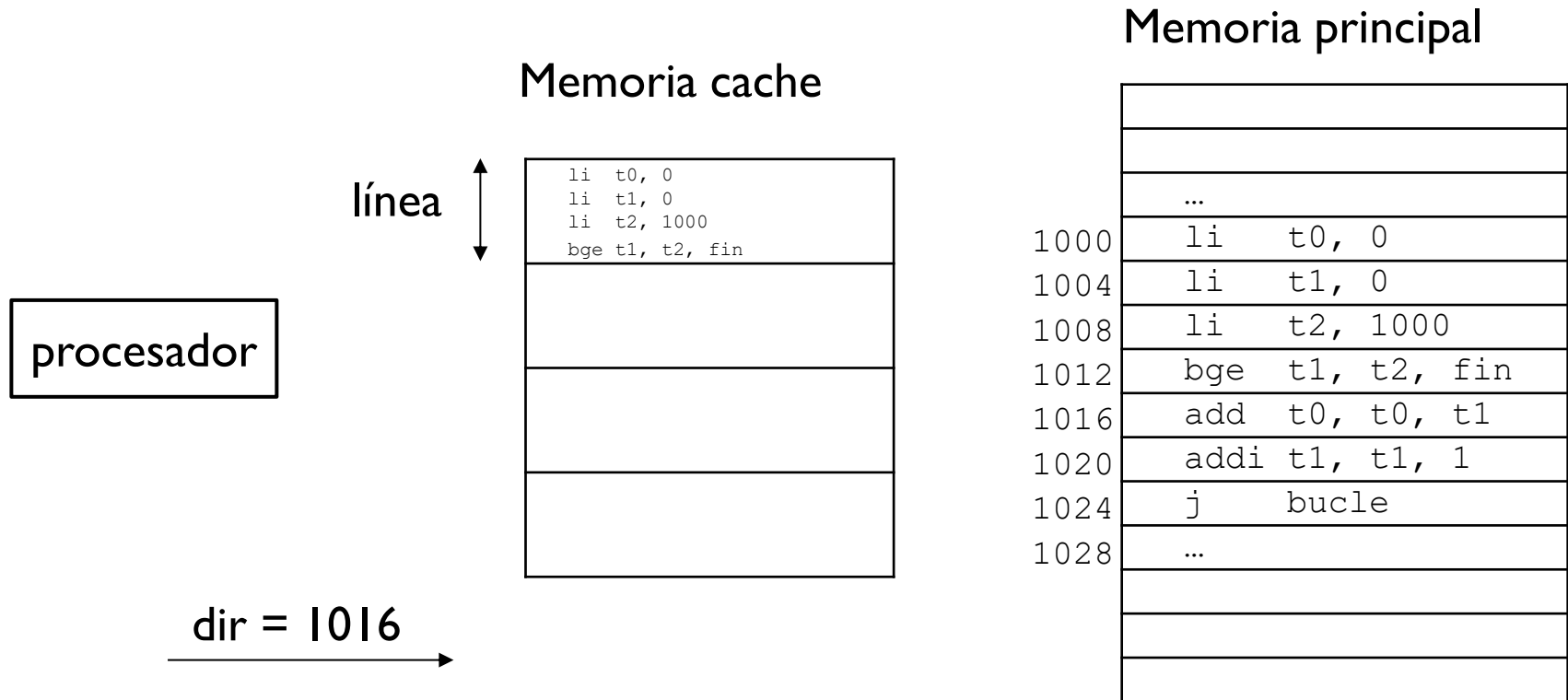


Memoria principal

	...
1000	li t0, 0
1004	li t1, 0
1008	li t2, 1000
1012	bge t1, t2, fin
1016	add t0, t0, t1
1020	addi t1, t1, 1
1024	j bucle
1028	...

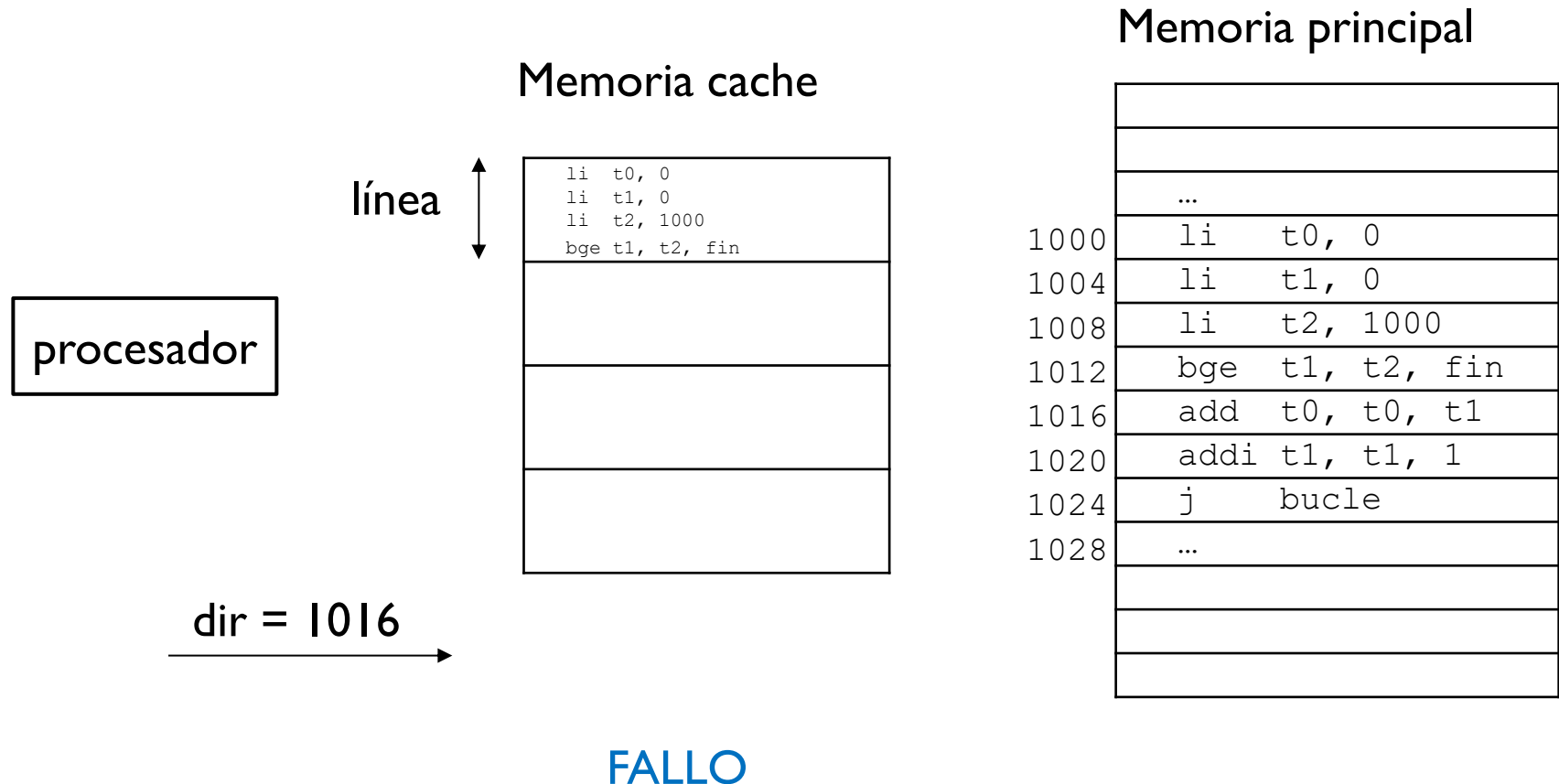
Ejemplo de funcionamiento

(2/2) estudio de referencias generadas



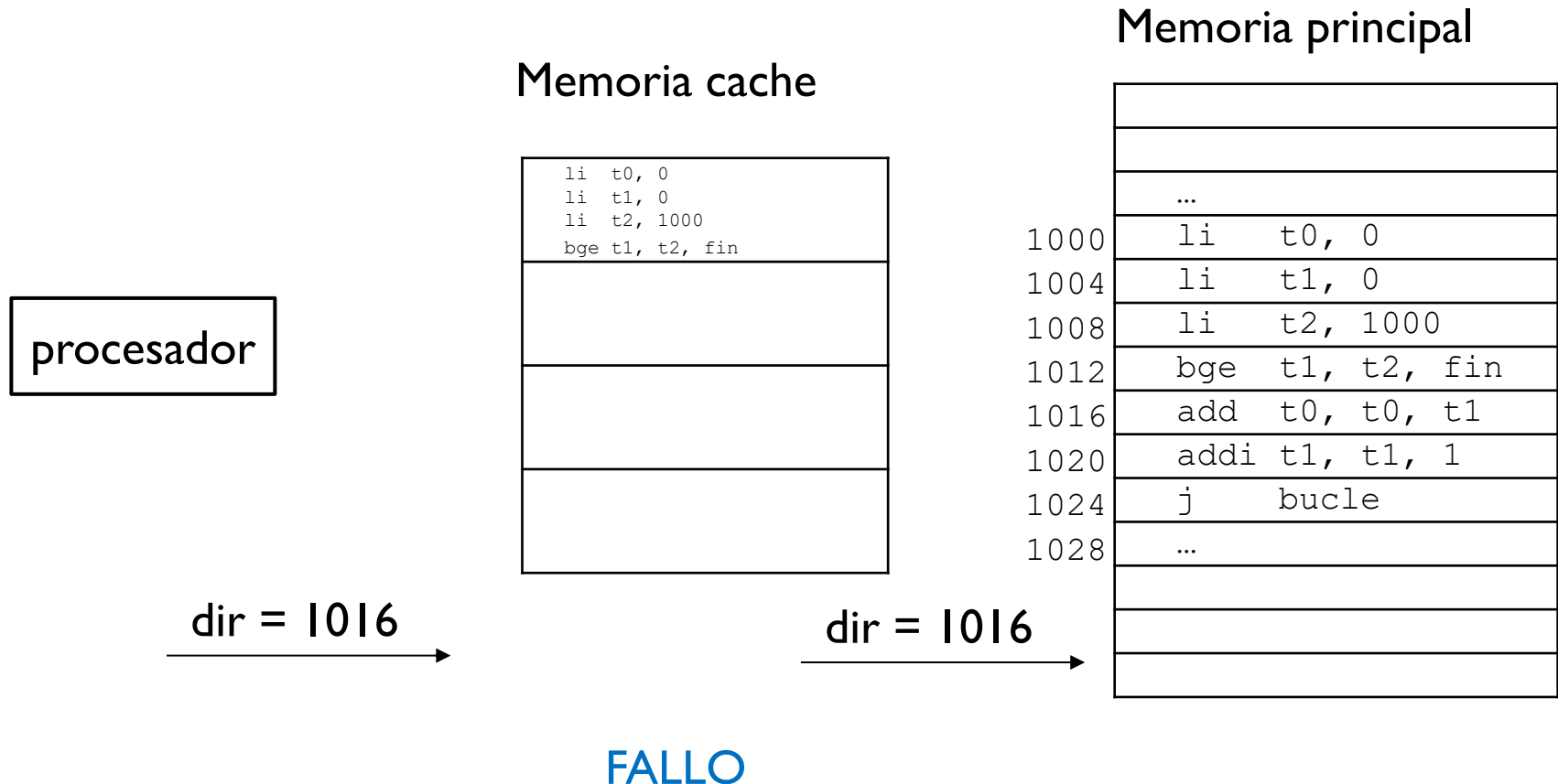
Ejemplo de funcionamiento

(2/2) estudio de referencias generadas



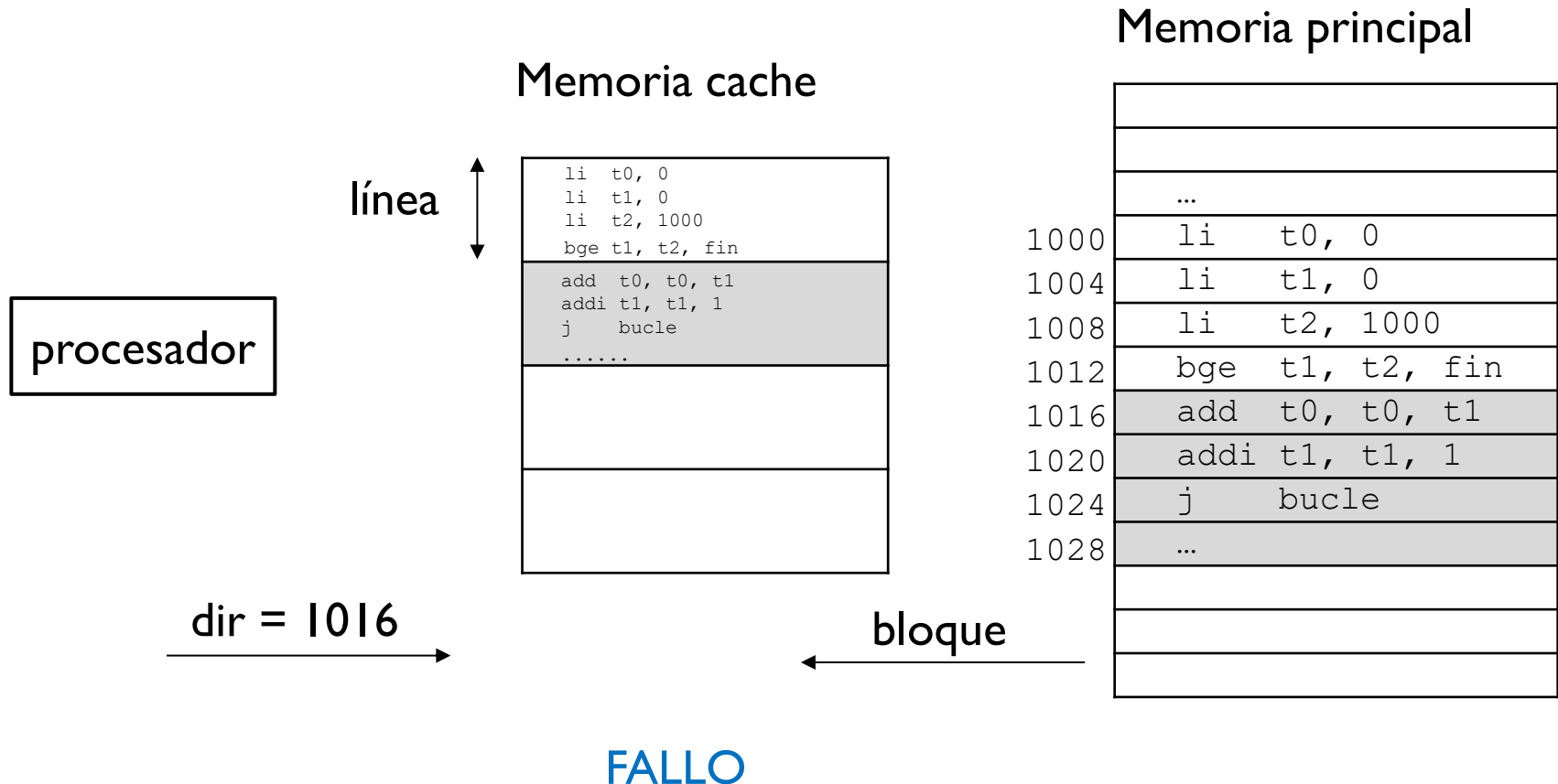
Ejemplo de funcionamiento

(2/2) estudio de referencias generadas



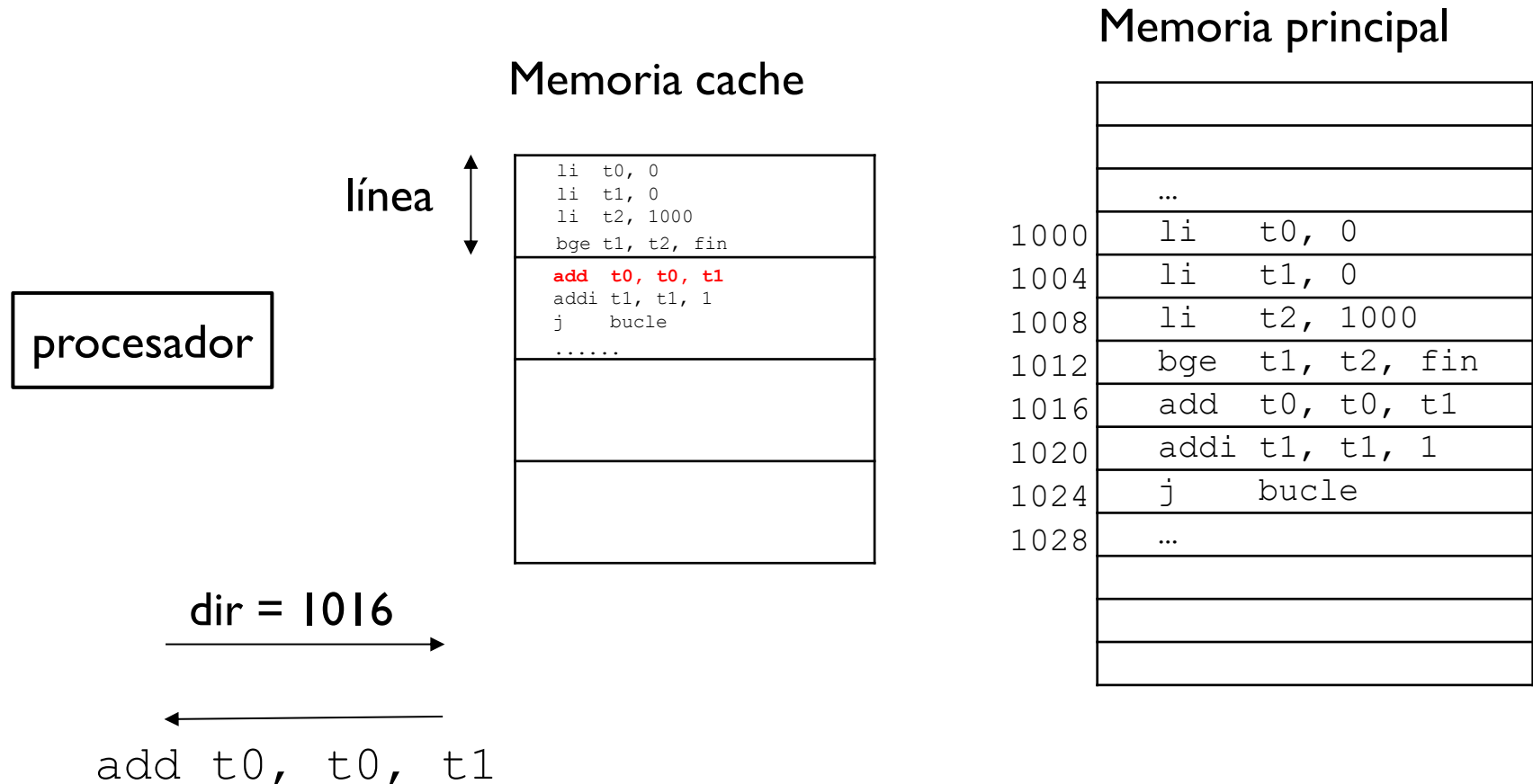
Ejemplo de funcionamiento

(2/2) estudio de referencias generadas



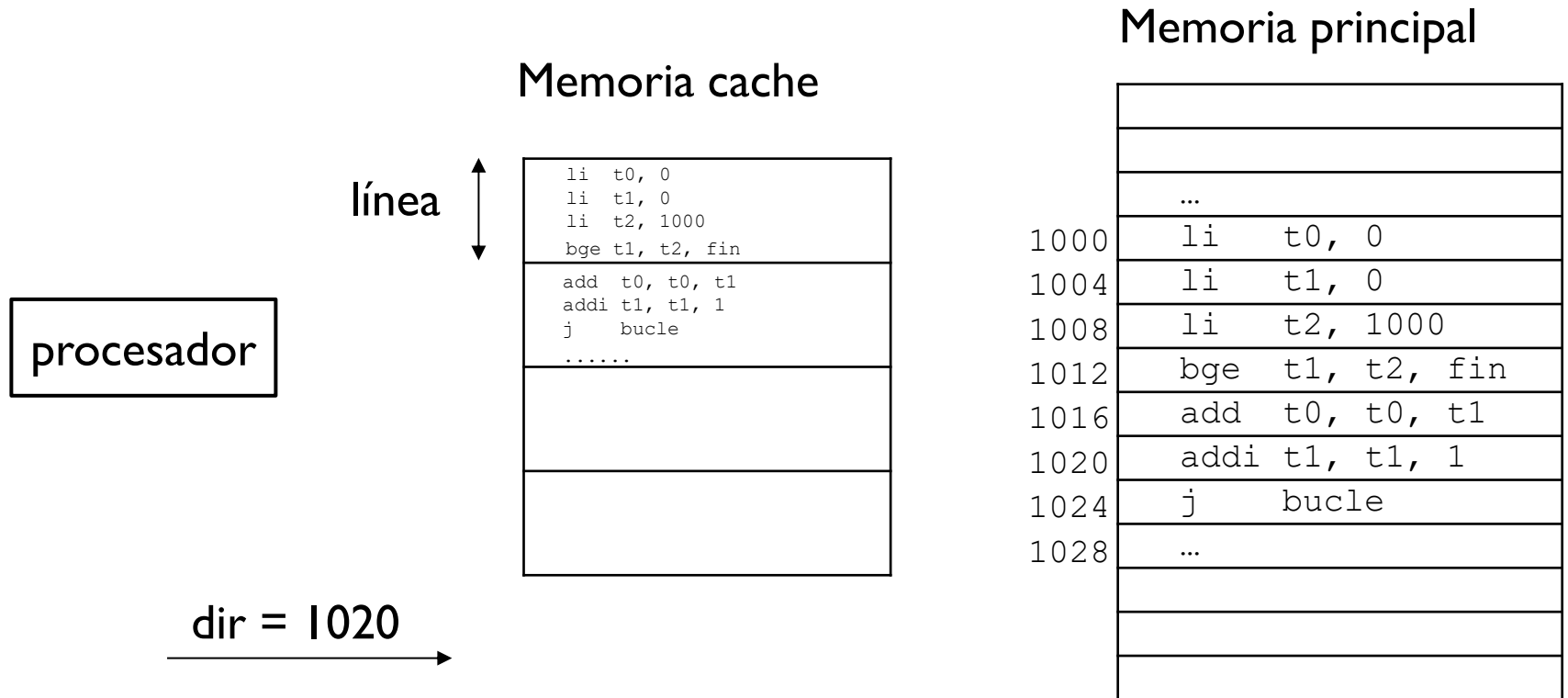
Ejemplo de funcionamiento

(2/2) estudio de referencias generadas



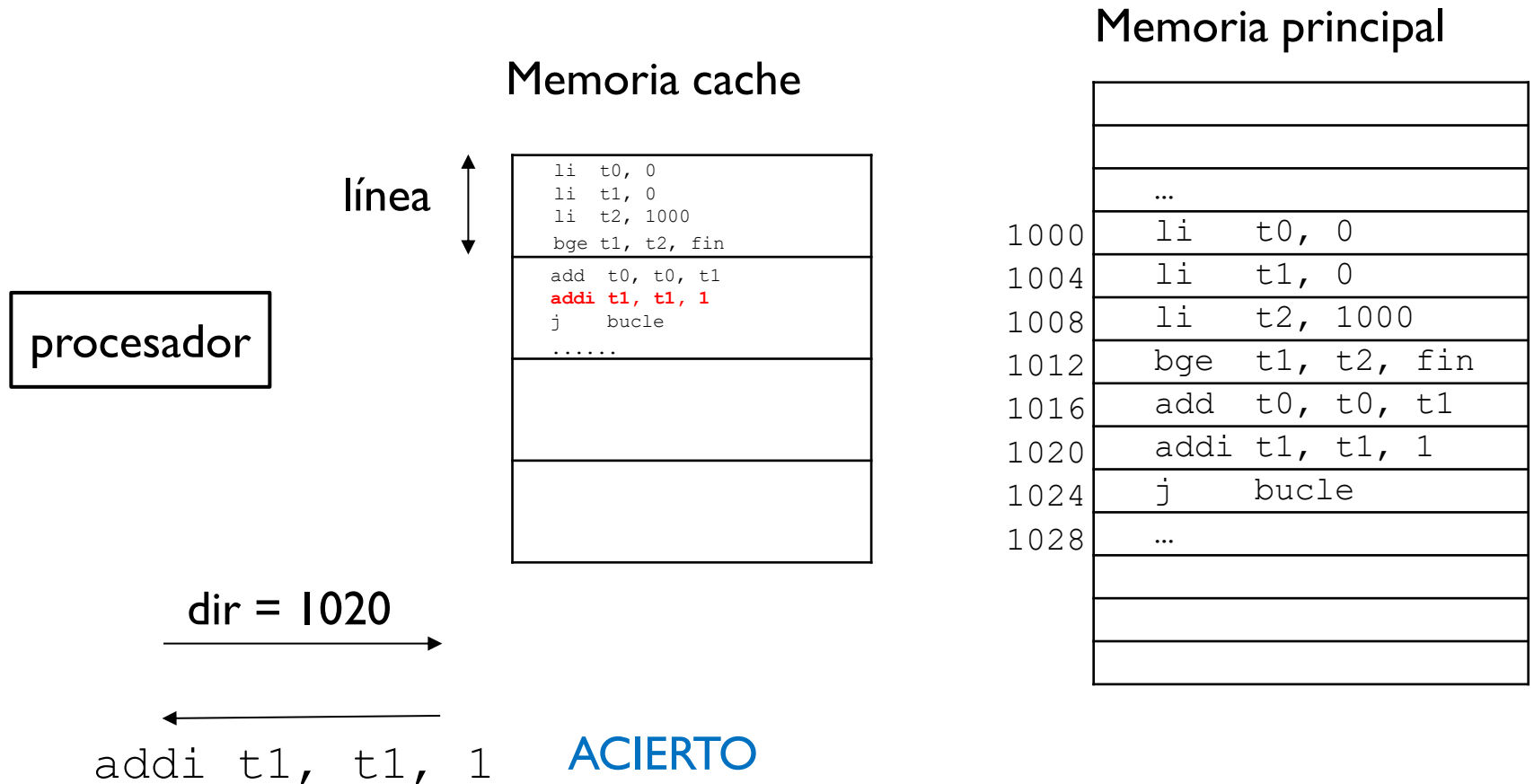
Ejemplo de funcionamiento

(2/2) estudio de referencias generadas



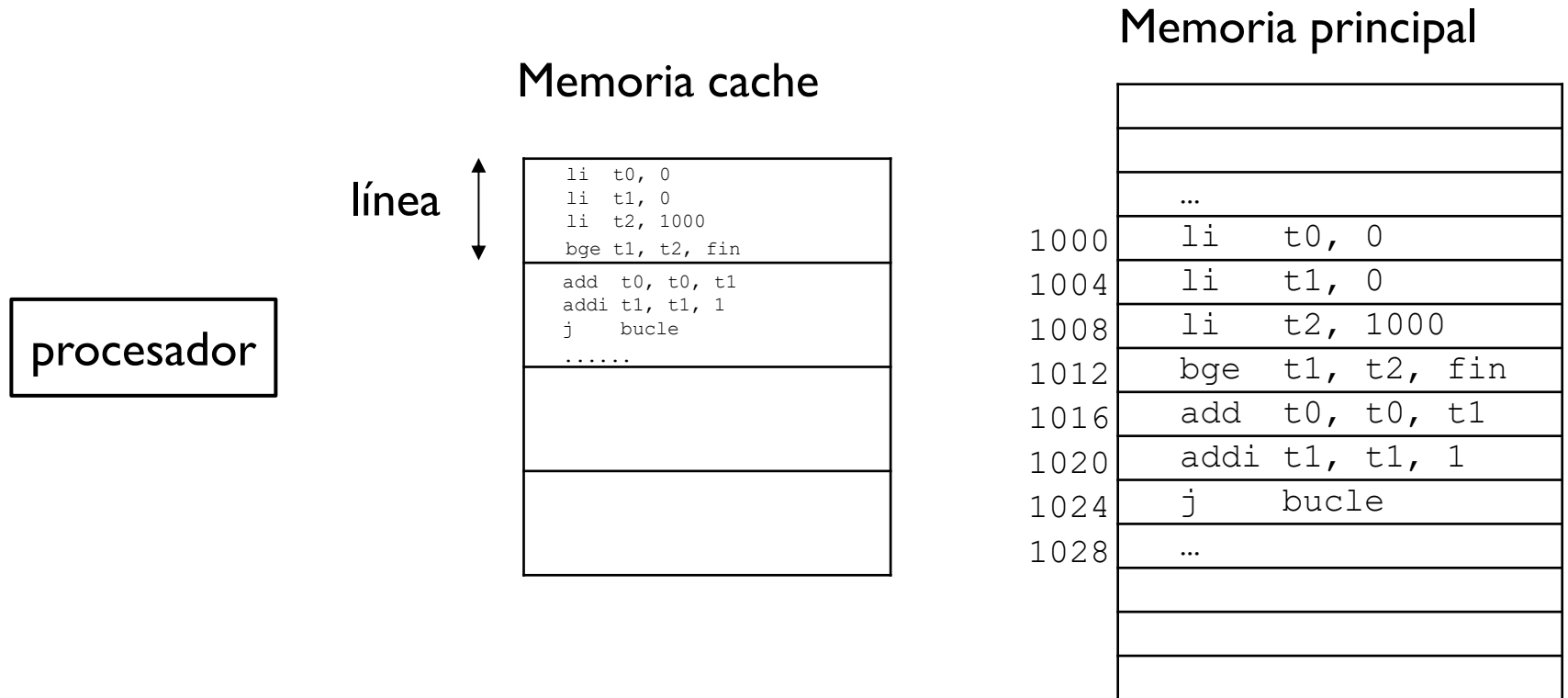
Ejemplo de funcionamiento

(2/2) estudio de referencias generadas



Ejemplo de funcionamiento

(2/2) estudio de referencias generadas



El resto de los accesos son ACIERTOS

Ejemplo de funcionamiento

```
int i;  
int s = 0;  
for (i=0; i < 1000; i++)  
    s = s + i;
```

bucle:

```
li    t0, 0    # s  
li    t1, 0    # i  
li    t2, 1000  
bge   t1, t2, fin
```

fin:

```
add   t0, t0, t1  
addi  t1, t1, 1  
j     bucle  
...
```

► **Con** memoria caché (bloque de 4 palabras):

- Número de accesos = 4004 accesos
- Número de bloques = 2
- Número de fallos = 2
- Tiempo de acceso = ?

(2/2) estudio de referencias
generadas por ejecución:
accesos a código (fetch) y datos

Ejemplo de funcionamiento

```
int i;  
int s = 0;  
for (i=0; i < 1000; i++)  
    s = s + i;
```

bucle:

```
li    t0, 0      # s  
li    t1, 0      # i  
li    t2, 1000  
bge   t1, t2, fin
```

fin:

```
add   t0, t0, t1  
addi  t1, t1, 1  
j     bucle  
...
```

► Con memoria caché (bloque de 4 palabras):

- Número de accesos = 4004 accesos
- Número de bloques = 2
- Número de fallos = 2
- **Tiempo de acceso = 8408 ns**

- Tiempo para transferir 2 bloques = $200 \times 2 = 400$ ns

- Tiempo de acceso a la caché = $4004 \times 2 = 8008$ ns

- Acceso a M.caché: 2 ns
- Acceso a M.P.: 120 ns
- Bloque de caché: 4 palabras
- Transferencia de un bloque entre memoria principal y caché: 200 ns

Ejemplo de funcionamiento

```
int i;
int s = 0;
for (i=0; i < 1000; i++)
    s = s + i;

li    t0, 0    # s
li    t1, 0    # i
li    t2, 1000
bucle: bge    t1, t2, fin
add    t0, t0, t1
addi   t1, t1, 1
j      bucle

fin:   ...
```

- ▶ Sin memoria caché = 480480 ns
- ▶ Con memoria caché = 8408 ns
- ▶ Tasa de aciertos a la caché = $4002 / 4004 \Rightarrow 99,95 \%$

¡ x57 !

Ejercicio

Calcular tasa de aciertos

```
int v[1000]; // global
...
int i;
int s;
for (i=0; i < 1000; i++)
    s = s + v[i];
```

- ▶ **Ejemplo:**
 - ▶ Acceso a caché: 2 ns
 - ▶ Acceso a MP: 120 ns
 - ▶ Bloque de MP: 4 palabras
 - ▶ Transferencia de un bloque entre memoria principal y caché: 200 ns

```
.data
    v: .space 4000 # 4*1000

.text
main:
    li    t0, 0      # i
    li    t1, 0      # i de v
    li    t2, 1000   # num. eltos.
    li    t3, 0      # s
bucle: bge    t0, t2, fin
    lw     t4, v(t1)
    add    t3, t3, t4
    addi   t0, t0, 1
    addi   t1, t1, 4
    j      bucle
fin:     ...
```

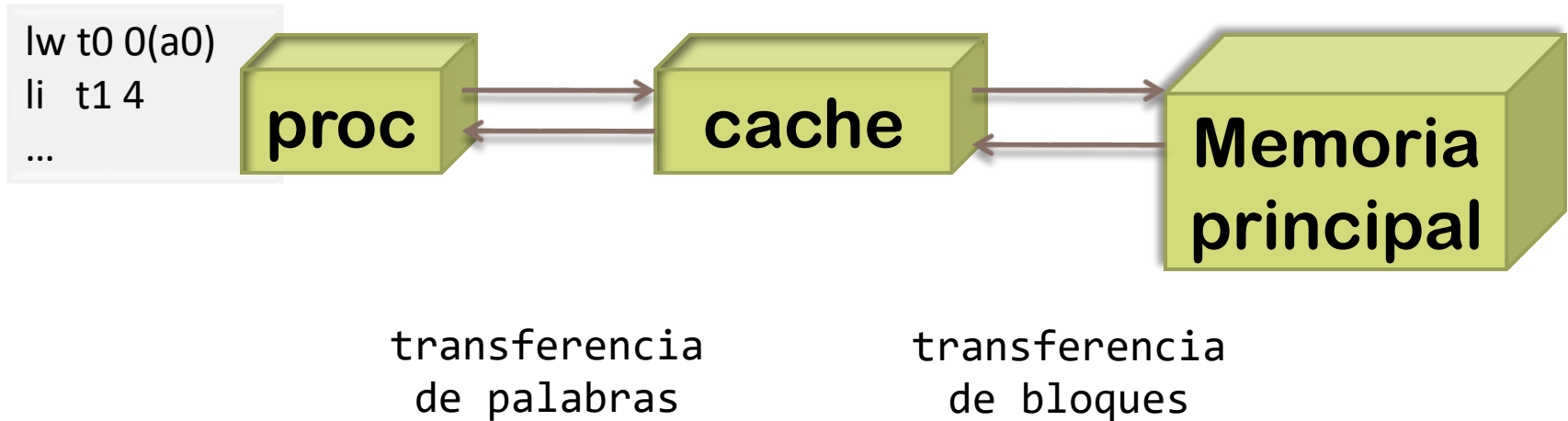
¿Por qué funciona?

- ▶ Tiempo de acceso a caché mucho menor que tiempo de acceso a memoria principal.
- ▶ A la memoria principal se accede por bloques.
- ▶ Cuando un programa accede a una dirección, es probable que vuelva a acceder a ella en el futuro cercano.
 - ▶ Proximidad o Localidad temporal.
- ▶ Cuando un programa accede a una dirección, es probable que en el futuro cercano acceda a posiciones cercanas.
 - ▶ Proximidad o Localidad espacial.
- ▶ Tasa de aciertos: probabilidad de que un dato accedido esté en la caché

Tasa de aciertos elevada

Funcionamiento general

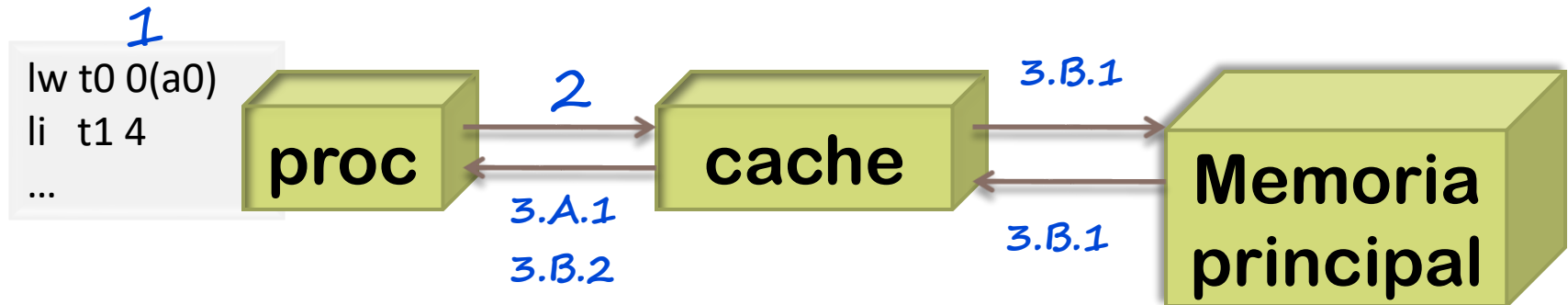
recordatorio



- ▶ Se construye con tecnología SRAM
 - ▶ Integrada en el mismo procesador.
 - ▶ Más rápida y más cara que la memoria DRAM.
- ▶ Mantiene una **copia** de partes de la memoria principal.

Funcionamiento general

recordatorio



1. El procesador solicita contenidos de una posición (dirección) de memoria.

2. La cache comprueba si ya están los datos de esta posición:

Si está (ACIERTO),

3.A.1 Se la sirve al procesador desde la cache (rápidamente).

Si no está (FALLO),

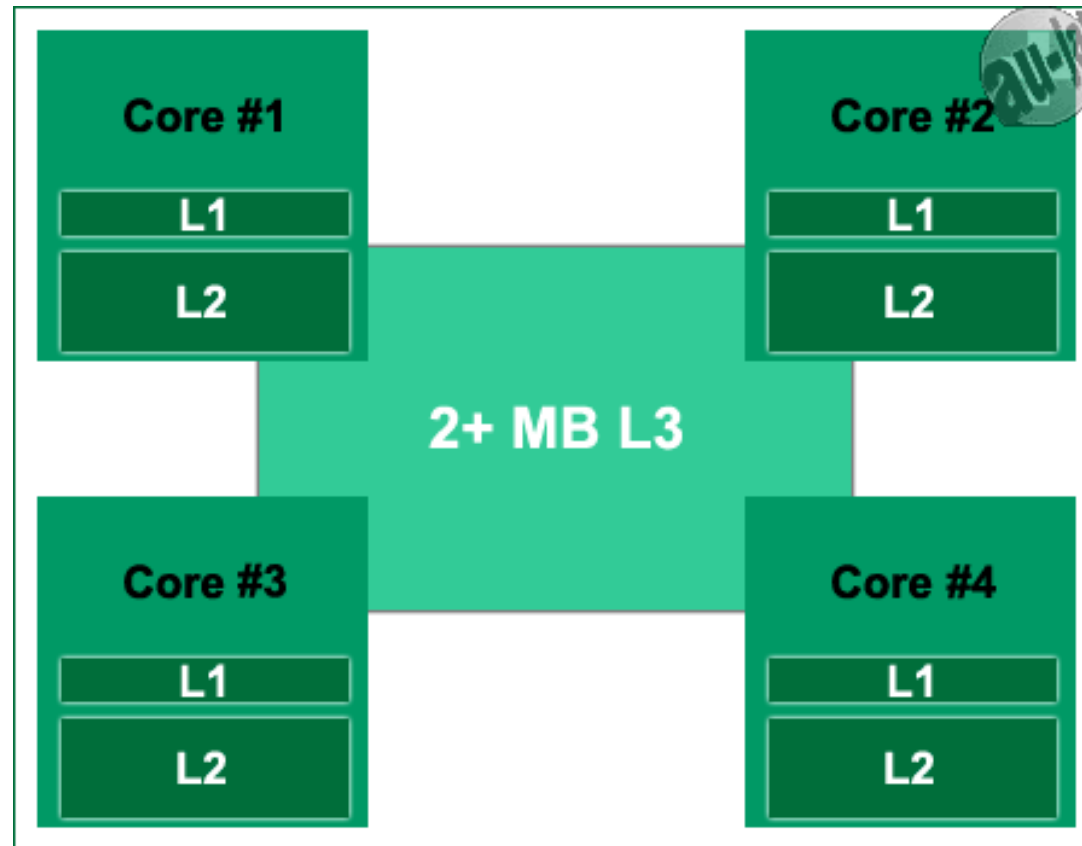
3.B.1 La cache transfiere de memoria principal el bloque de palabras asociado a la posición.

3.B.2 Después, la cache entrega los datos pedidos al procesador.

Niveles de memoria caché

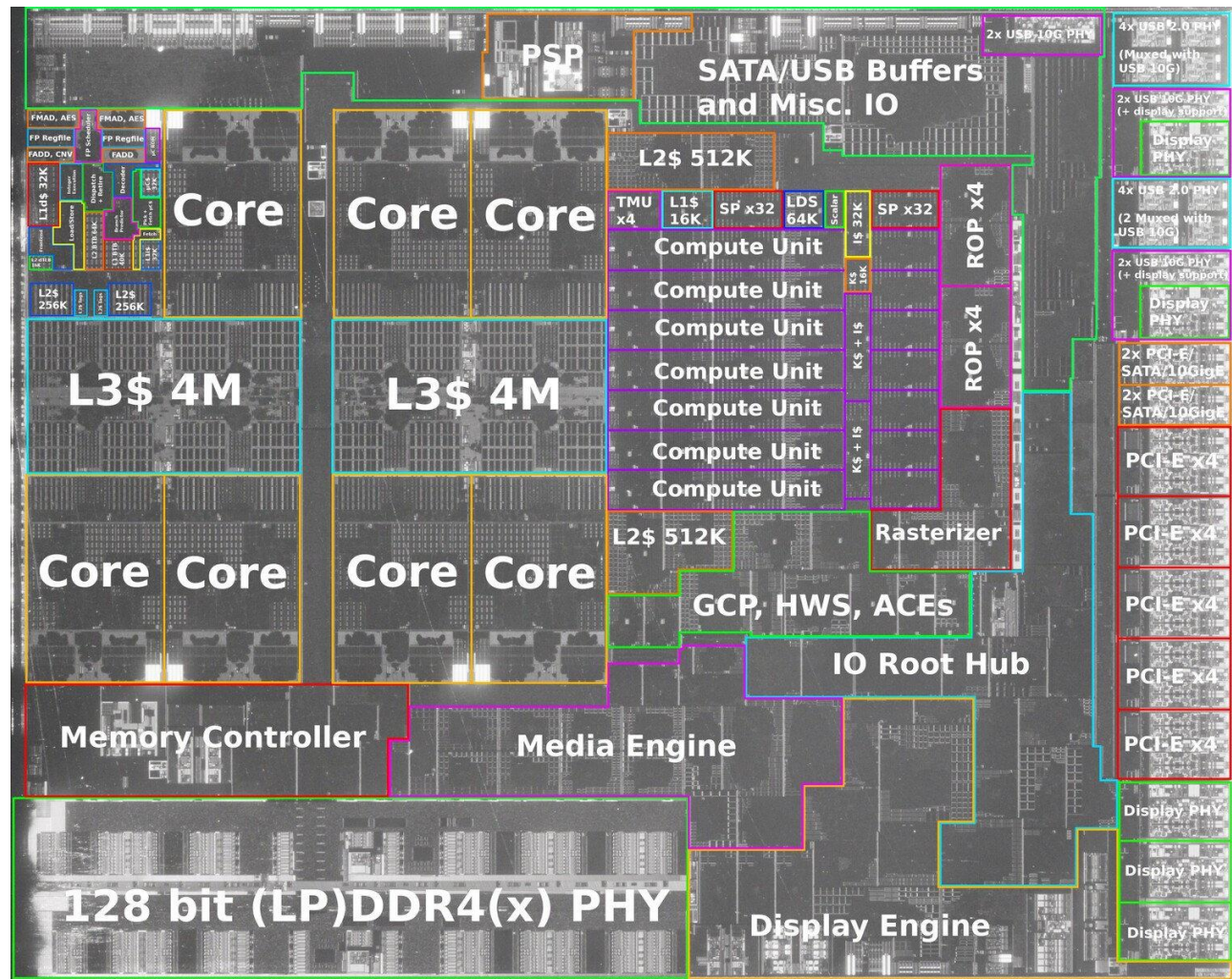
- ▶ Es habitual encontrar tres niveles:
 - ▶ **L1 o nivel 1:**
 - ▶ Caché interna: la más cercana al procesador
 - ▶ Tamaño pequeño (8KB-128KB) y máxima velocidad
 - ▶ Pueden separarse para instrucciones y datos
 - ▶ **L2 o nivel 2:**
 - ▶ Caché interna
 - ▶ Entre L1 y L3 (o entre L1 y M. Principal)
 - ▶ Tamaño mediano (256KB – 4MB) y menor velocidad que L1
 - ▶ **L3 o nivel 3:**
 - ▶ Típicamente último nivel antes de M. Principal
 - ▶ Tamaño mayor y menor velocidad que L2
 - ▶ Interna o externa al procesador

Ejemplo: AMD quad-core

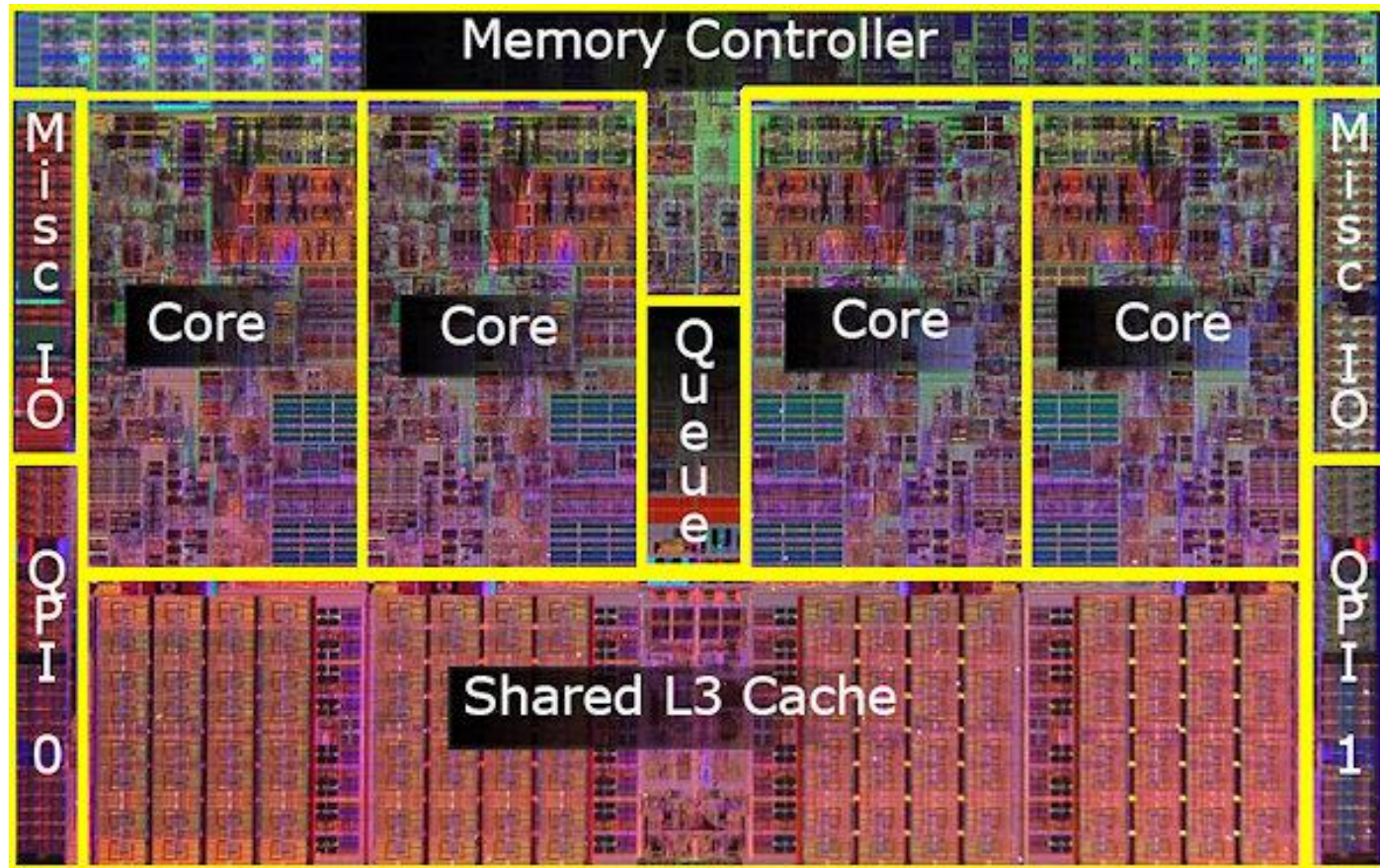


Quad-Core CPU mit gemeinsamen L3-Cache

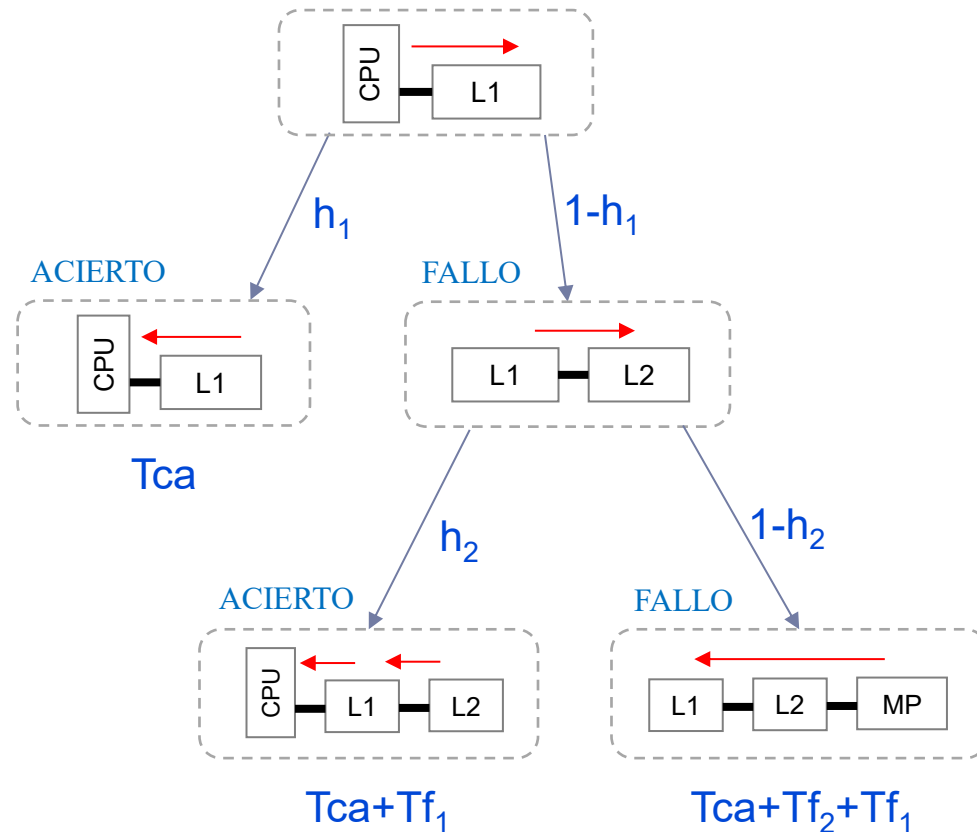
Ejemplo: AMD Ryzen 4000



Ejemplo: Intel Core i7



Ejemplo de accesos con varios niveles...



Contenidos

1. Tipos de memoria
2. Jerarquía de memoria
3. Memoria principal
4. Memoria caché
 1. Introducción
 2. Estructura de la memoria caché
 3. Diseño y organización de la memoria caché
5. Memoria virtual

Acceso a una palabra en m. principal

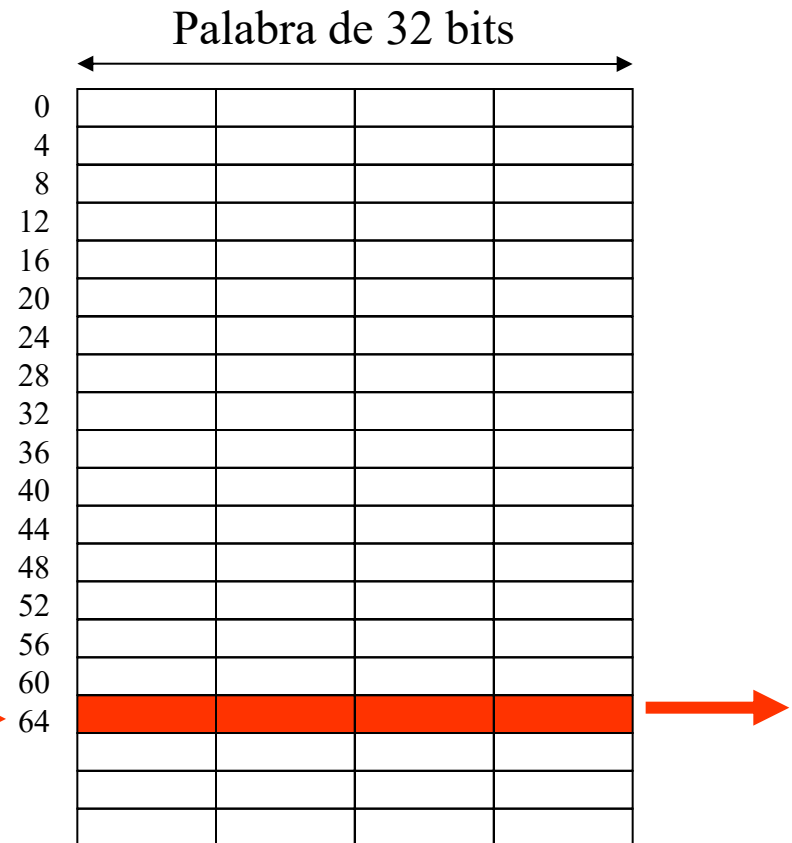
Recordatorio

- ▶ Ejemplo:
 - ▶ Computador de 32 bits
 - ▶ Memoria direccionada por bytes
 - ▶ Acceso a memoria principal por palabras
 - ▶ Acceso a la palabra con dirección 64 en decimal

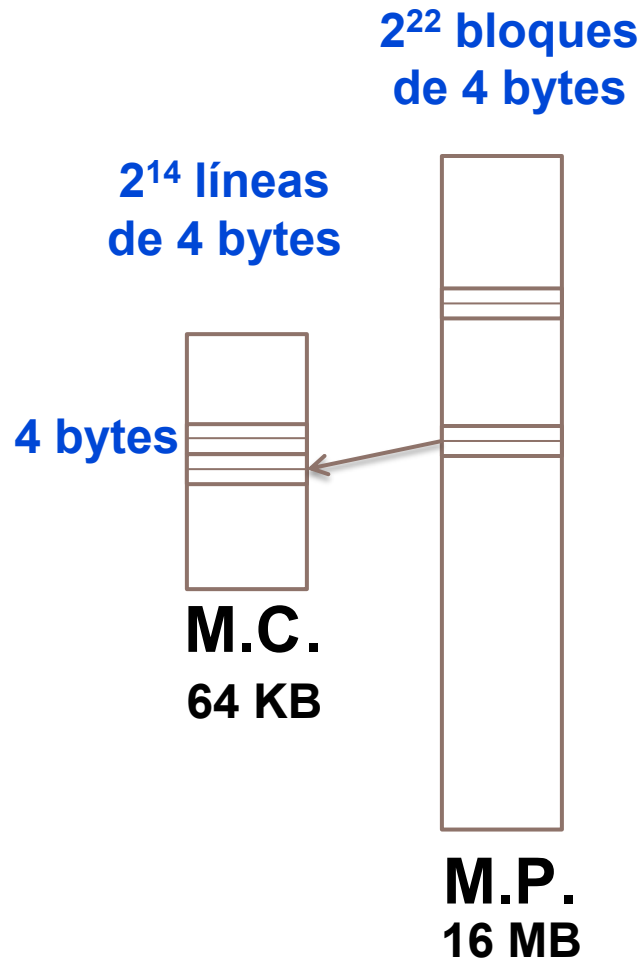
Palabra de 32 bits

0				
4				
8				
12				
16				
20				
24				
28				
32				
36				
40				
44				
48				
52				
56				
60				
64				

- ▶ Computador de 32 bits
- ▶ Memoria direccionada por bytes
- ▶ Acceso a memoria principal por palabras
- ▶ Acceso a la palabra con dirección 64 en decimal

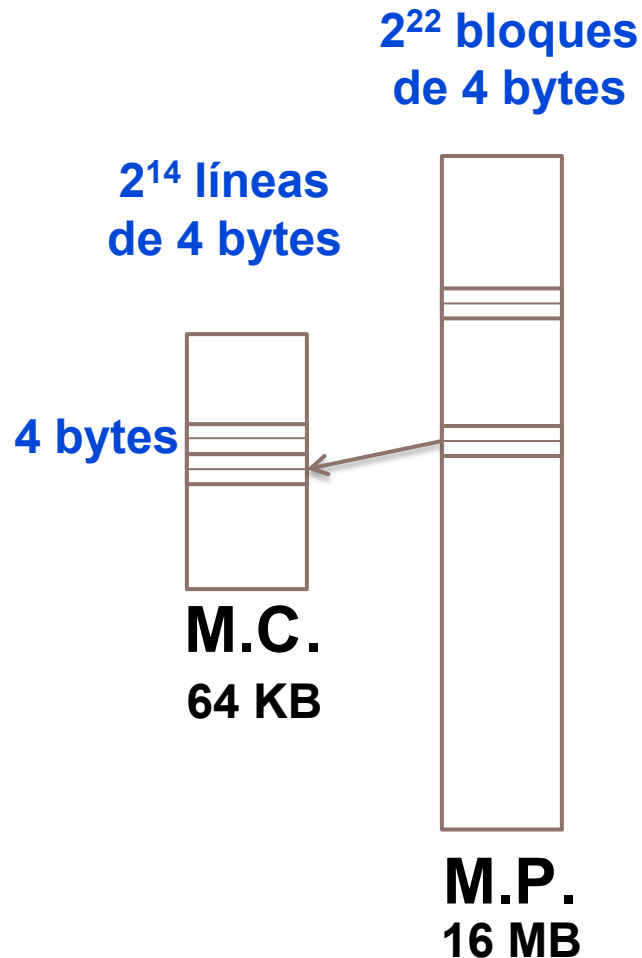


Estructura de la memoria caché



- ▶ Se divide la M.P. y la M.C. (de forma lógica) en bloques de igual tamaño
 - ▶ El bloque en caché se le llama **línea**
- ▶ A cada bloque de M.P. le corresponderá **una línea de M.C.** (bloque en caché)
- ▶ El tamaño de la M.C. es menor:
 - ▶ El número de bloques en la memoria caché es pequeño.

Estructura de la memoria caché

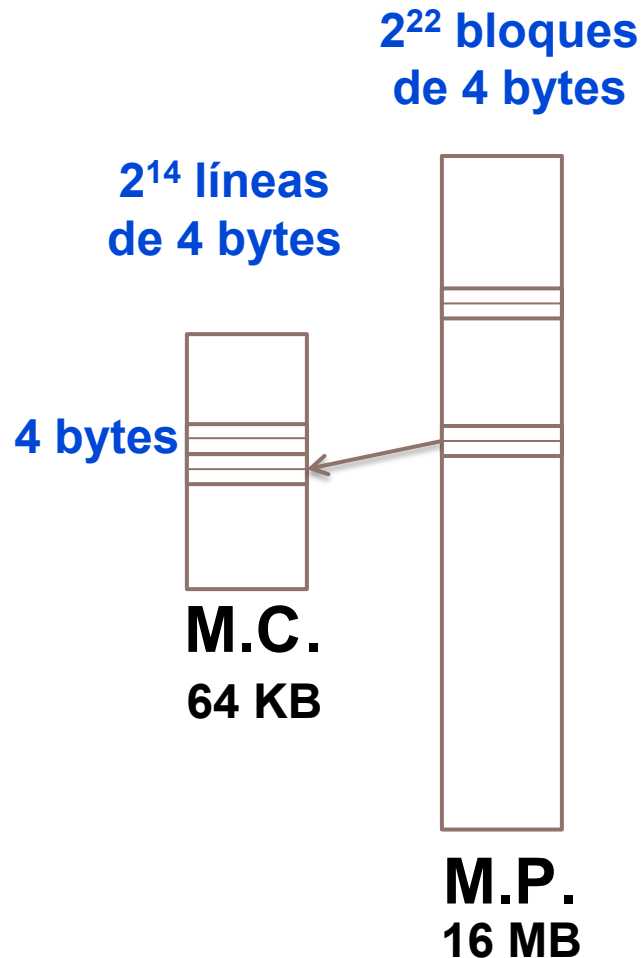


- ▶ Se divide la M.P. y la M.C. (de forma lógica) en bloques de igual tamaño
 - ▶ El bloque en caché se le llama **línea**
- ▶ A cada bloque de M.P. le corresponderá **una línea de M.C.** (bloque en caché)
- ▶ El tamaño de la M.C. es menor:
 - ▶ El número de bloques en la memoria caché es pequeño.

¿Cuántos bloques de 4 palabras caben en una m. caché de 64 KiB para una CPU de 32 bits?

- ▶ Solución:
 $2^6 \cdot 2^{10} \text{ bytes} / 2^4 \text{ bytes} = 2^{11} \text{ bloques} = 2048 \text{ bloques} = 2048 \text{ líneas}$

Estructura de la memoria caché



- ▶ Se divide la M.P. y la M.C. (de forma lógica) en bloques de igual tamaño
 - ▶ El bloque en caché se le llama **línea**
- ▶ A cada bloque de M.P. le corresponderá **una línea de M.C.** (bloque en caché)
- ▶ El tamaño de la M.C. es menor:
 - ▶ El número de bloques en la memoria caché es pequeño.

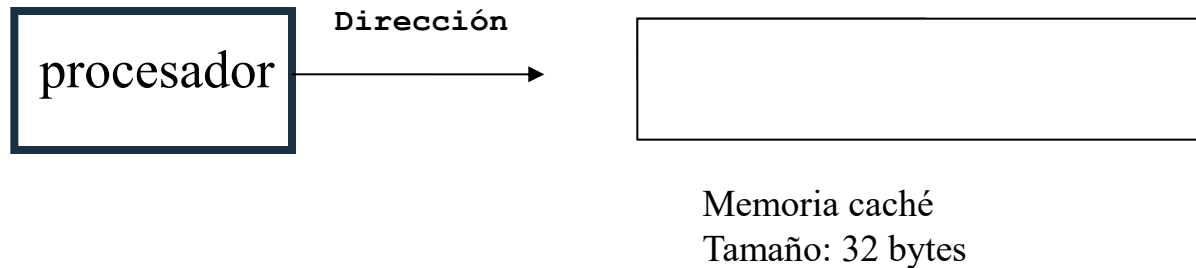
¿Cuántos bloques de 4 palabras de 4 bytes hay en una memoria de 1 GiB?

- ▶ Solución:

$$2^{30} \text{ b} / 2^4 \text{ b} = 2^{30-4} \text{ b} = 2^{26} \text{ b} = 64 \text{ megabloques} = \sim 64 \text{ millones}$$

Ejemplo de estructura...

Con líneas de dos palabras (8 bytes),
¿cuántas líneas tiene la caché y la m. ppal.?

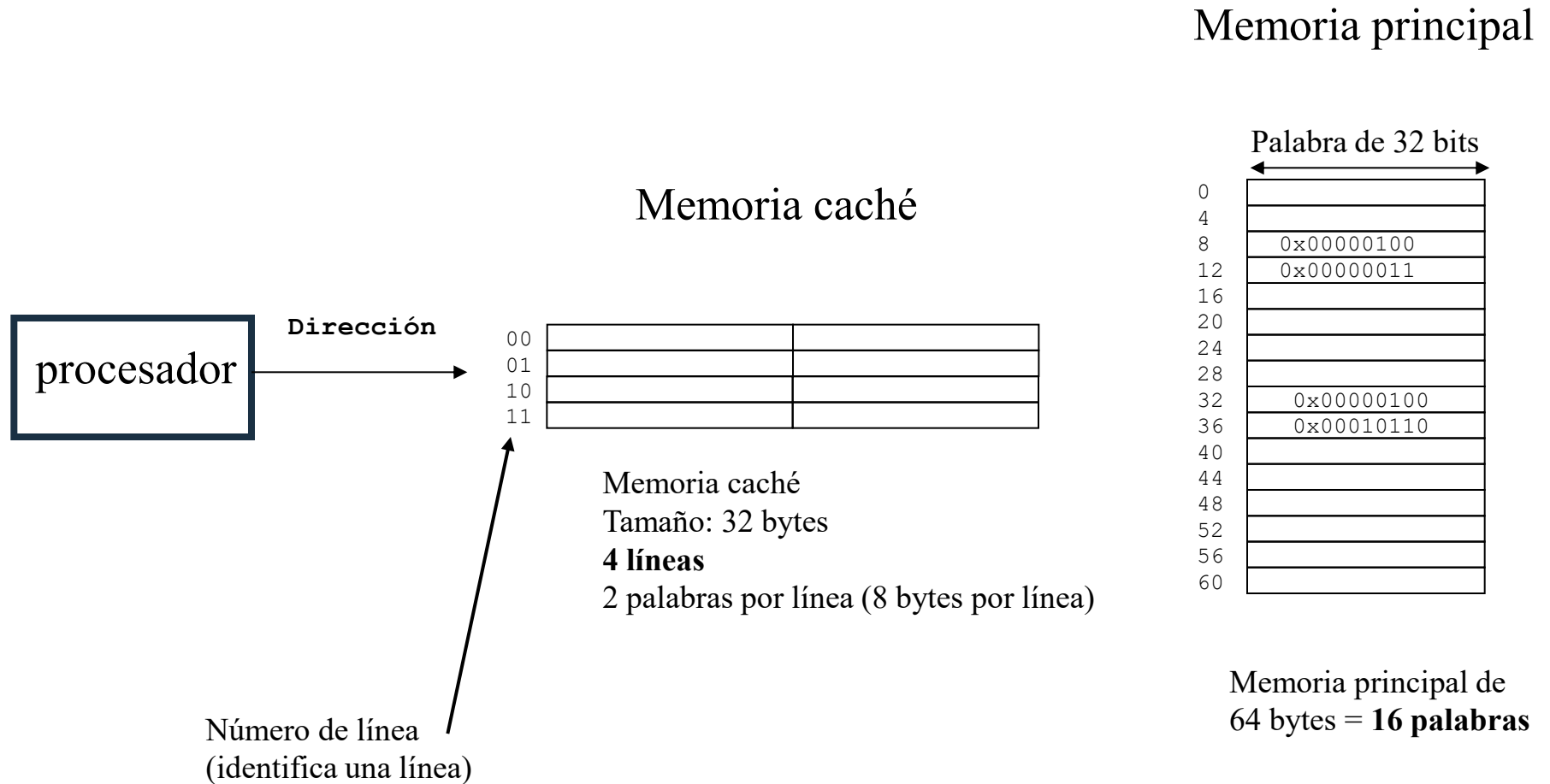


Memoria principal

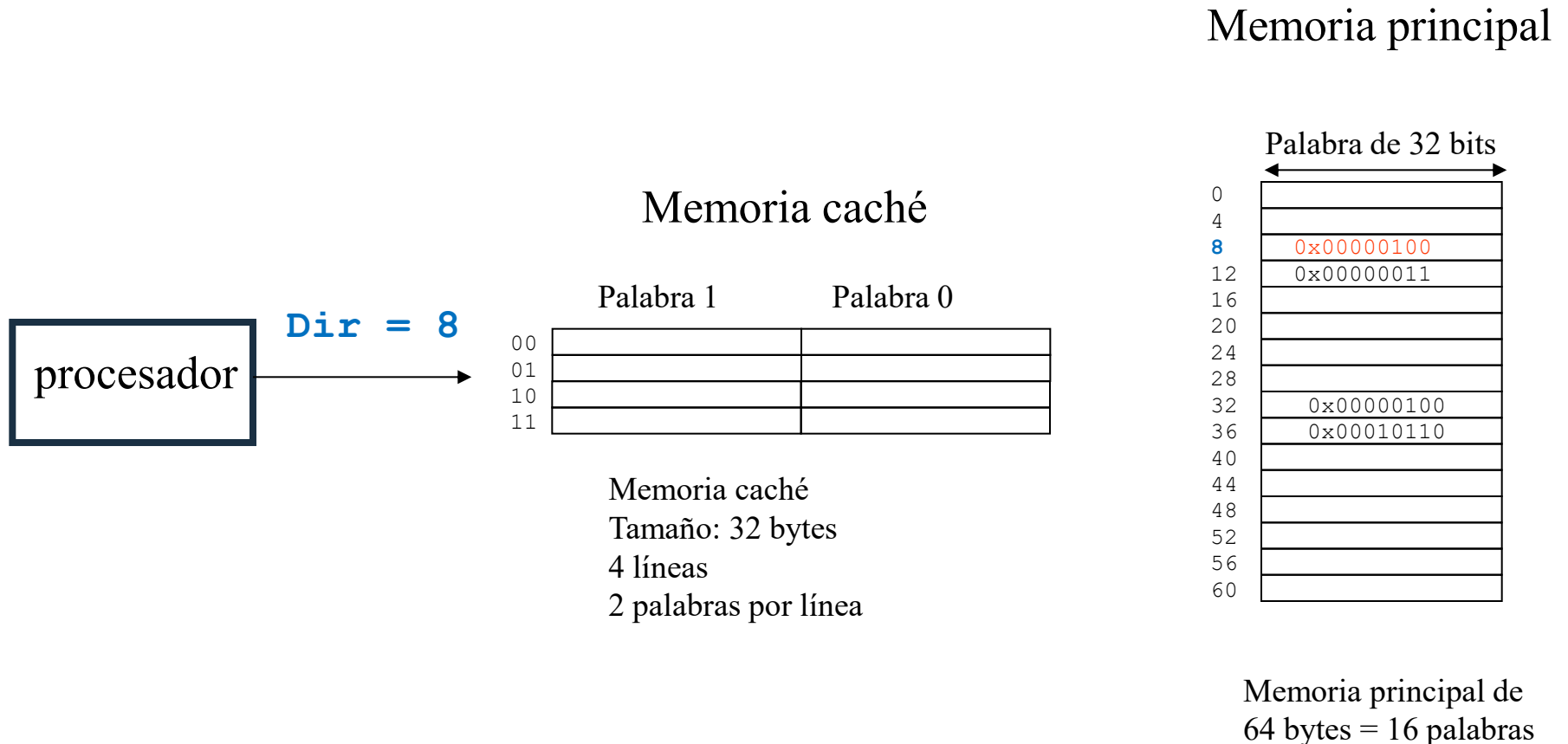
	Palabra de 32 bits
0	
4	
8	0x00000100
12	0x00000011
16	
20	
24	
28	
32	0x00000100
36	0x00010110
40	
44	
48	
52	
56	
60	

Memoria principal de
Tamaño: 64 bytes

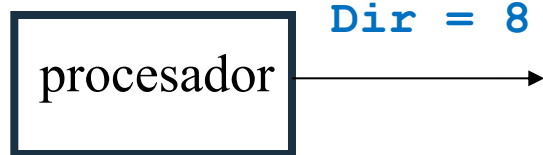
Ejemplo de estructura...



Qué no sabemos todavía: ¿cómo...?



¿Cómo buscar/...?



Memoria caché		
	Palabra 1	Palabra 0
00		
01		
10		
11		

Memoria caché
Tamaño: 32 bytes
4 líneas
2 palabras por línea

Memoria principal

Palabra de 32 bits	
0	
4	
8	0x00000100
12	0x00000011
16	
20	
24	
28	
32	0x00000100
36	0x00010110
40	
44	
48	
52	
56	
60	

Memoria principal de
64 bytes = 16 palabras

¿Cómo se localiza un bloque de
memoria principal?

¿Cómo buscar/...?

procesador

Dir = 8

Memoria caché		
	Palabra 1	Palabra 0
00		
01		
10		
11		

Memoria caché
Tamaño: 32 bytes
4 líneas
2 palabras por línea

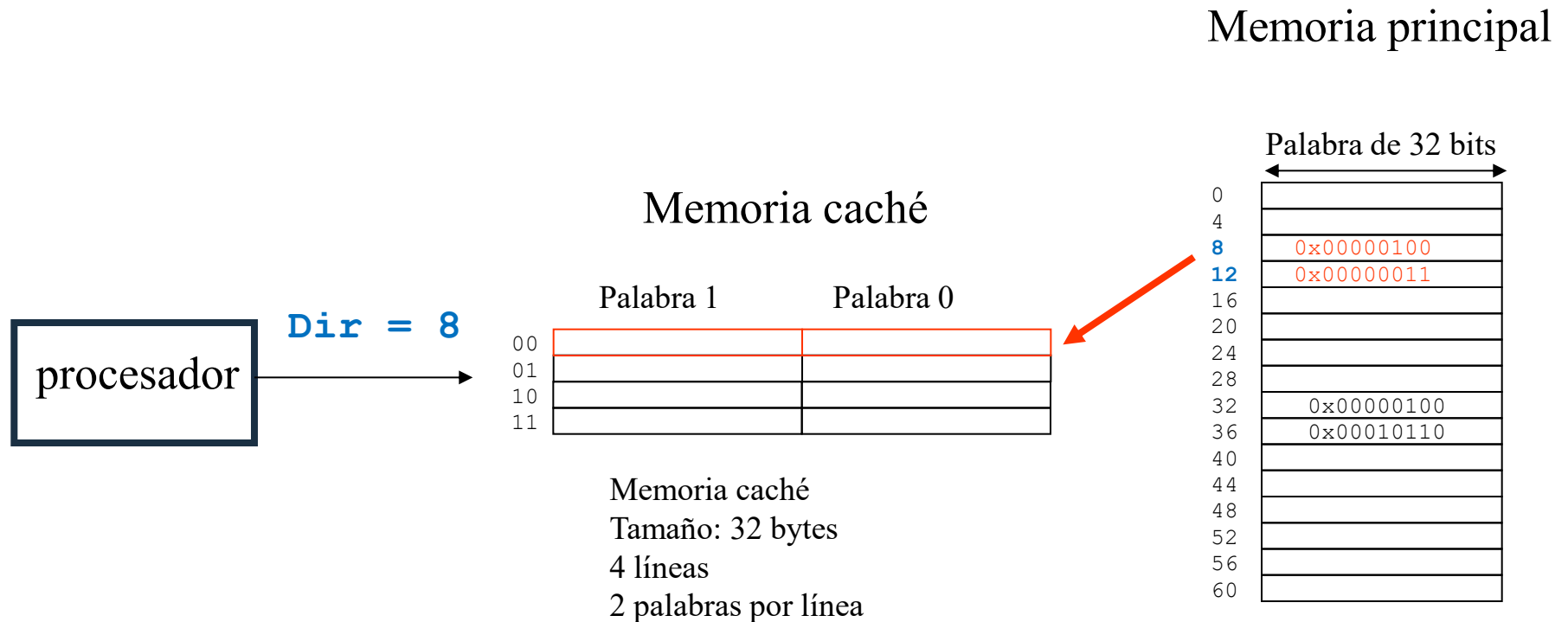
FALLO

Memoria principal

Palabra de 32 bits	
0	
4	
8	0x00000100
12	0x00000011
16	
20	
24	
28	
32	0x00000100
36	0x00010110
40	
44	
48	
52	
56	
60	

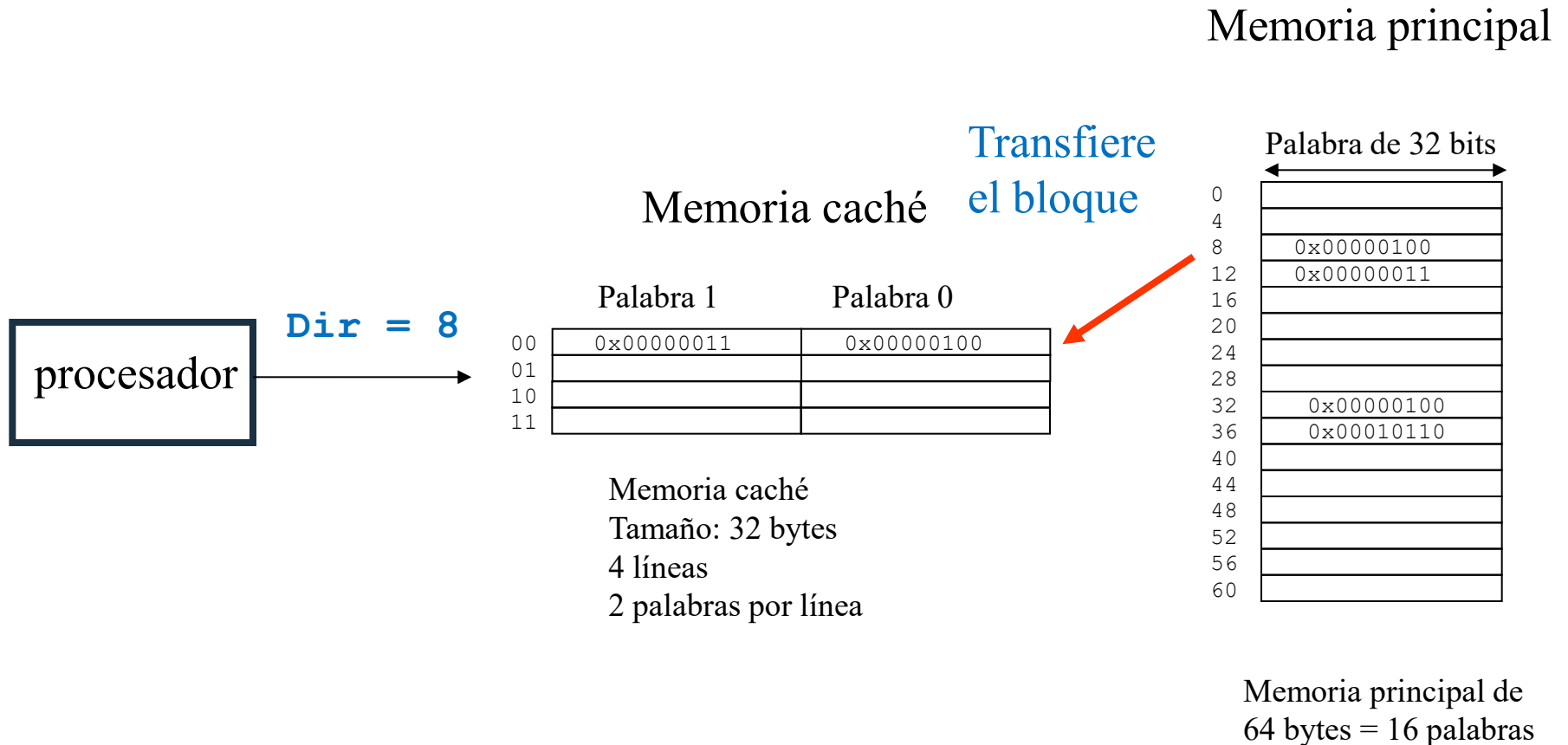
Memoria principal de
64 bytes = 16 palabras

¿Cómo asignar/...?

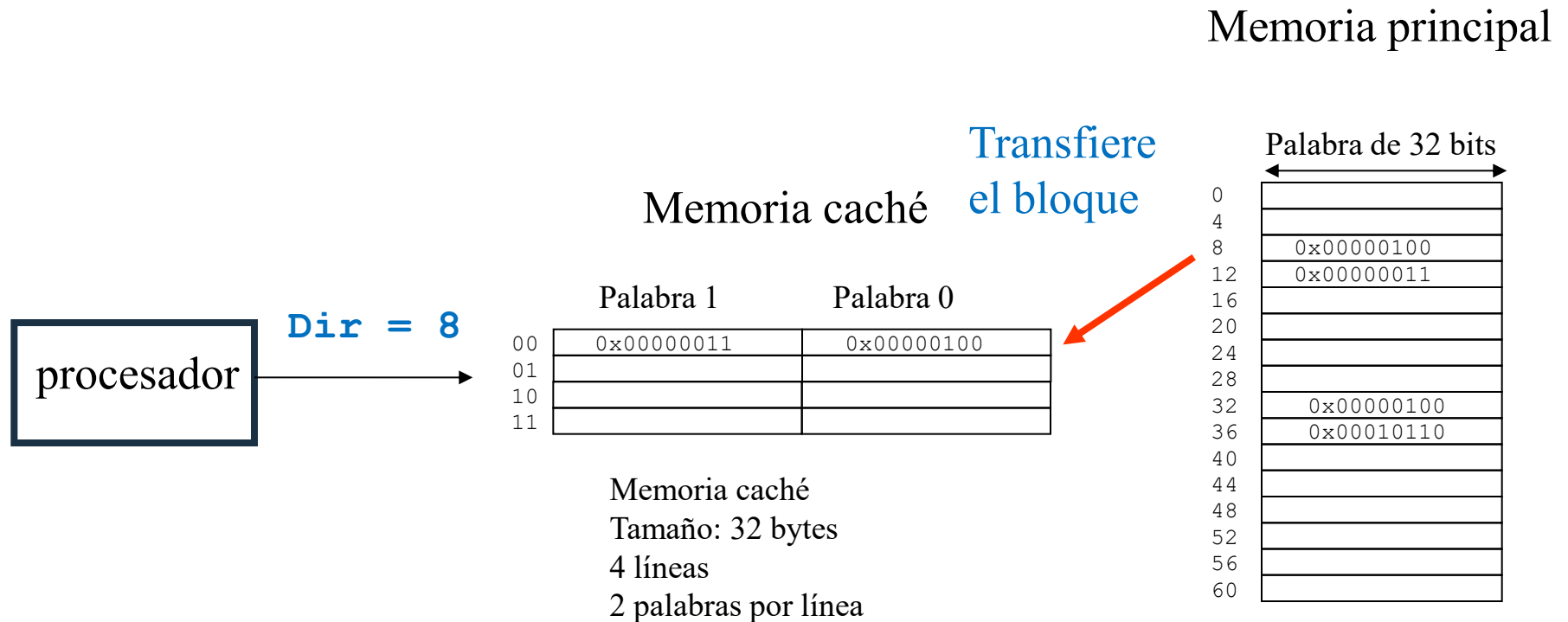


¿Dónde se ubica un bloque de memoria principal?

¿Cómo...?



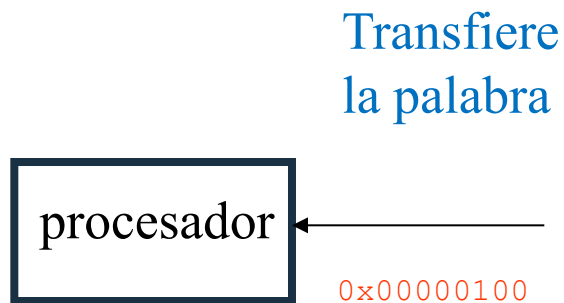
¿Cómo remplazar/...?



¿Qué bloque debe remplazarse en caso de fallo con remplazo?

Memoria principal de
64 bytes = 16 palabras

¿Cómo...?



	Palabra 1	Palabra 0
00	0x00000011	0x00000100
01		
10		
11		

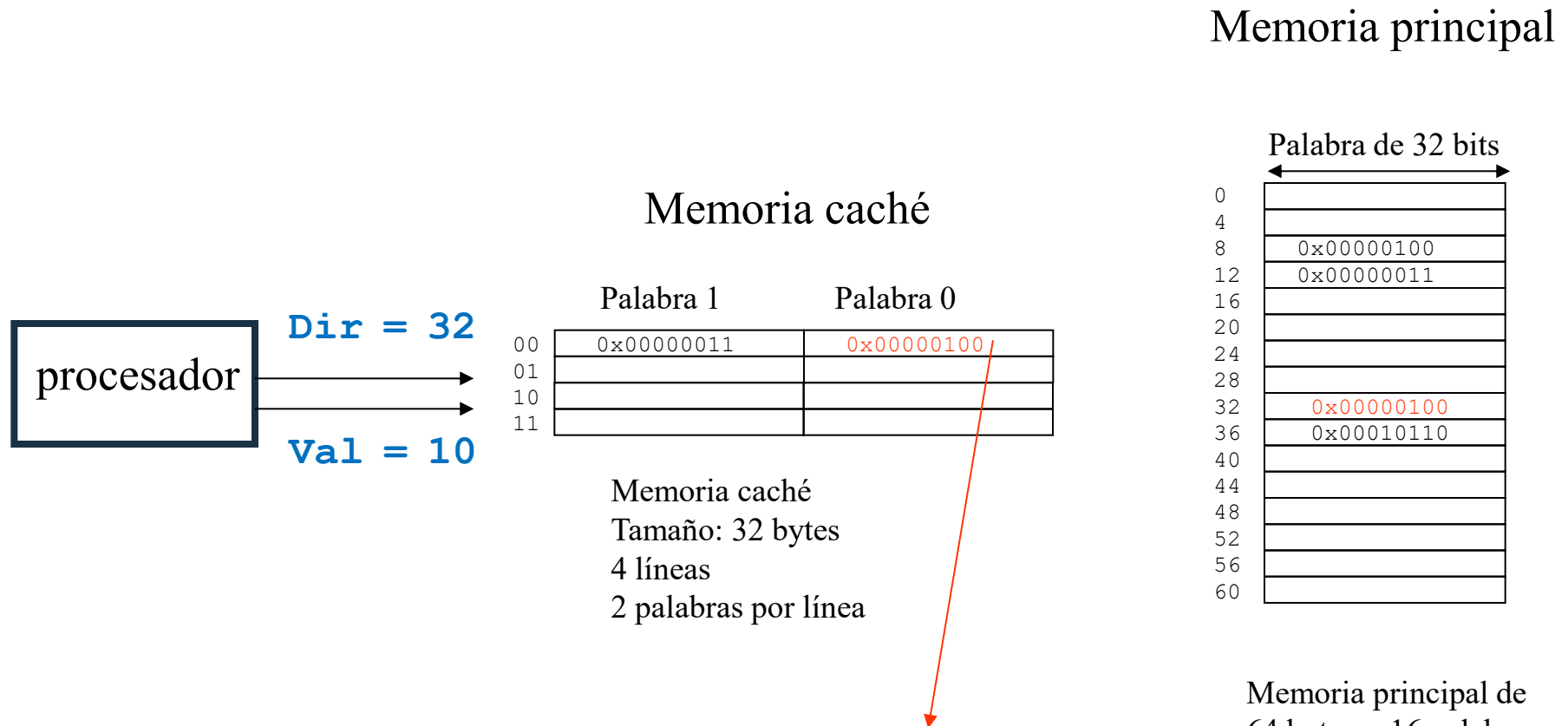
Memoria caché
Tamaño: 32 bytes
4 líneas
2 palabras por línea

Memoria principal

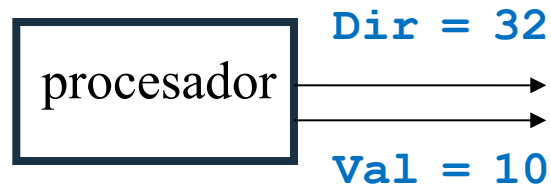
	Palabra de 32 bits
0	
4	
8	0x00000100
12	0x00000011
16	
20	
24	
28	
32	0x00000100
36	0x00010110
40	
44	
48	
52	
56	
60	

Memoria principal de
64 bytes = 16 palabras

¿Cómo buscar/asignar/remplazar/...?



¿Cómo escribir/...?



Memoria caché

	Palabra 1	Palabra 0
00	0x00000011	0x00000100
01		
10		
11		

Memoria caché
Tamaño: 32 bytes
4 líneas
2 palabras por línea

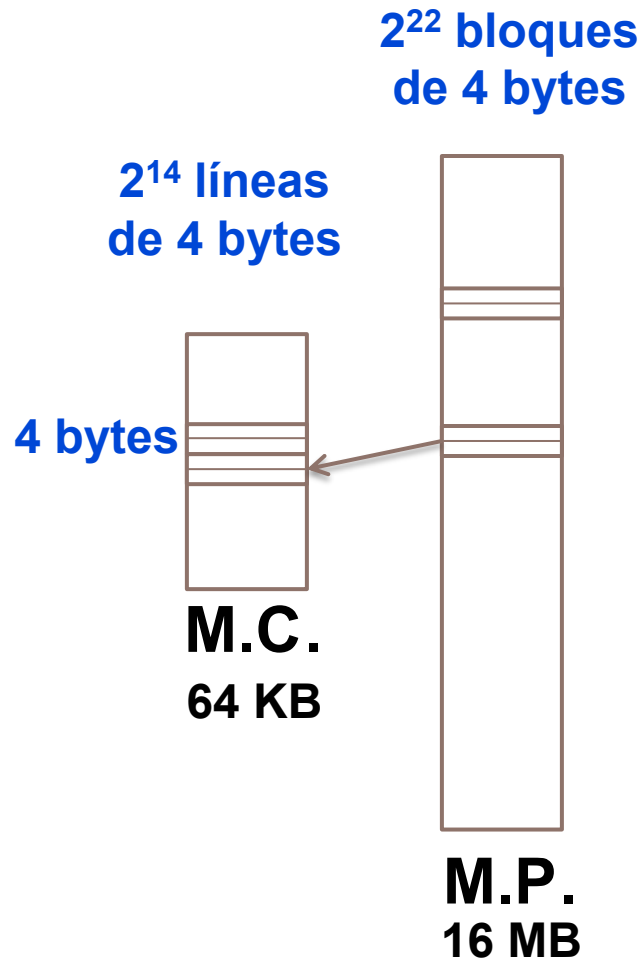
Memoria principal

	Palabra de 32 bits
0	
4	
8	0x00000100
12	0x00000011
16	
20	
24	
28	
32	0x00000100
36	0x00010110
40	
44	
48	
52	
56	
60	

Memoria principal de
64 bytes = 16 palabras

¿Qué ocurre en caso de escritura?
¿Se actualiza M.C. solo, M.C. y M.P, ...?

Preguntas sobre la memoria caché...



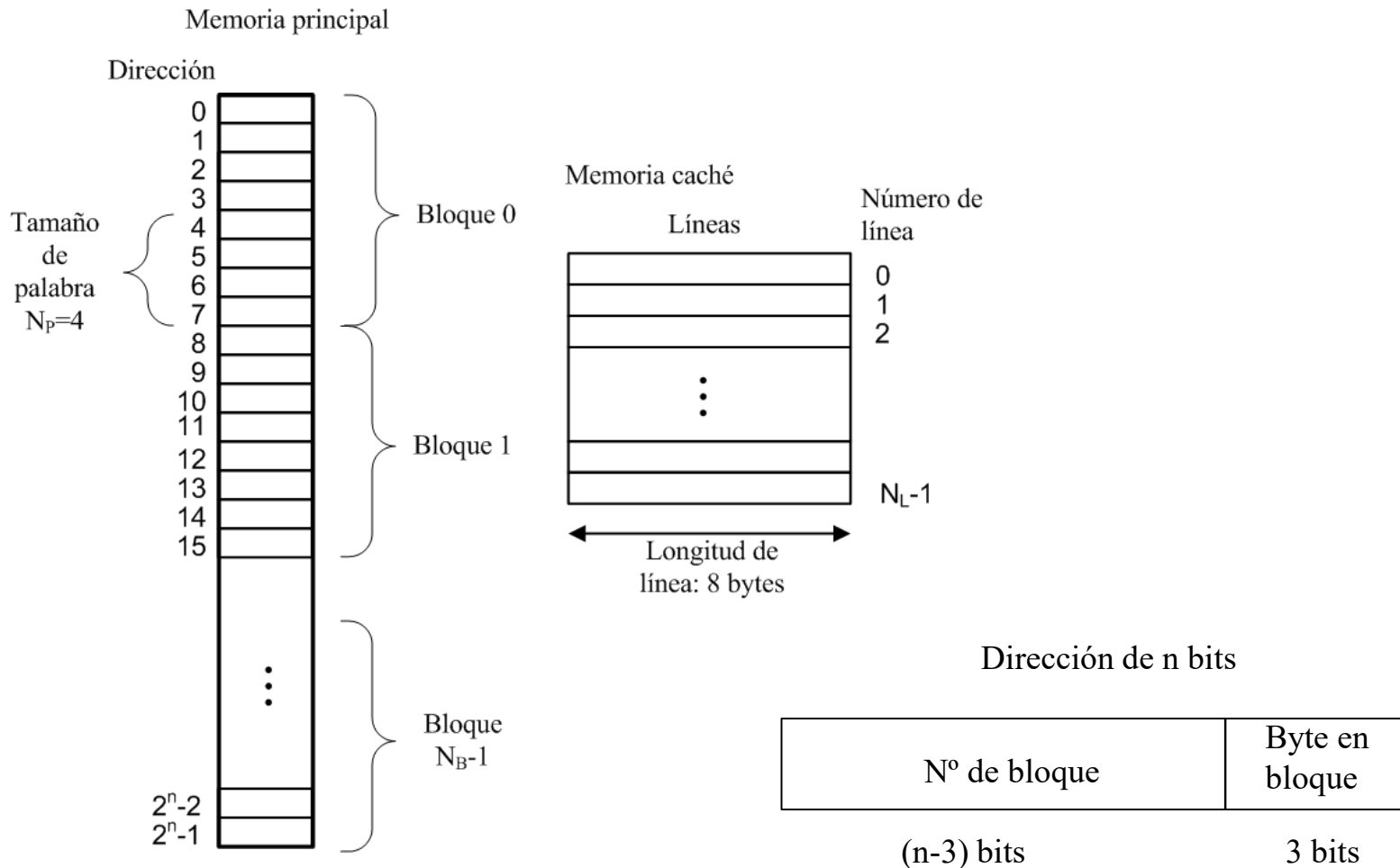
- ▶ Se divide la M.P. y la M.C. (de forma lógica) en bloques de igual tamaño
 - ▶ El bloque en caché se le llama **línea**
- ▶ A cada bloque de M.P. le corresponderá **una línea de M.C.** (bloque en caché)
- ▶ El tamaño de la M.C. es menor:
 - ▶ El número de bloques en la memoria caché es pequeño.

1. **¿Dónde se ubica un bloque de M.P.?**
 - ▶ Ubicación de bloque
2. **¿Cómo se localiza un bloque de M.P.?**
 - ▶ Identificación de bloque
3. **Si hay remplazo en un fallo... ¿qué bloque usar?**
 - ▶ Remplazo de bloque
4. **En escritura... ¿qué se actualiza? ¿M.C., M.P. y M.C.?**
 - ▶ Política de escritura

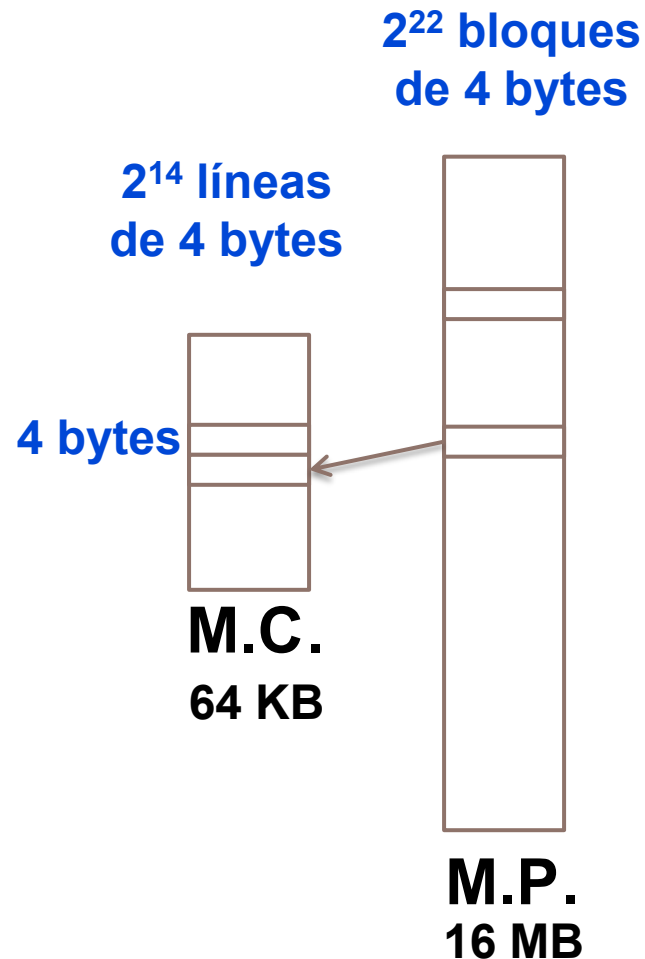
Contenidos

1. Tipos de memoria
2. Jerarquía de memoria
3. Memoria principal
4. Memoria caché
 1. Introducción
 2. Estructura de la memoria caché
 3. Diseño y organización de la memoria caché
5. Memoria virtual

Ejemplo de organización de la memoria caché

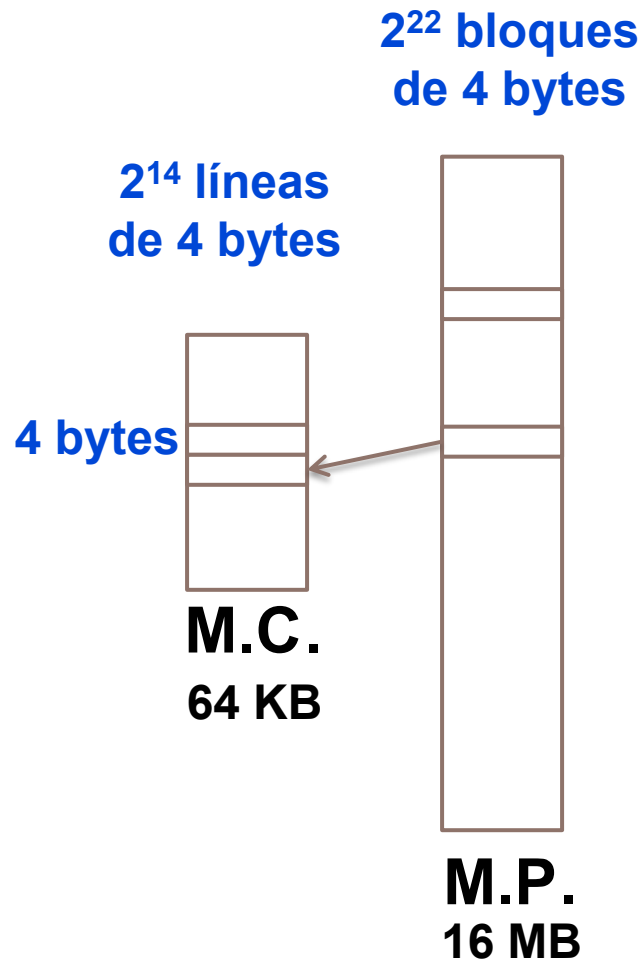


Estructura y diseño de la cache



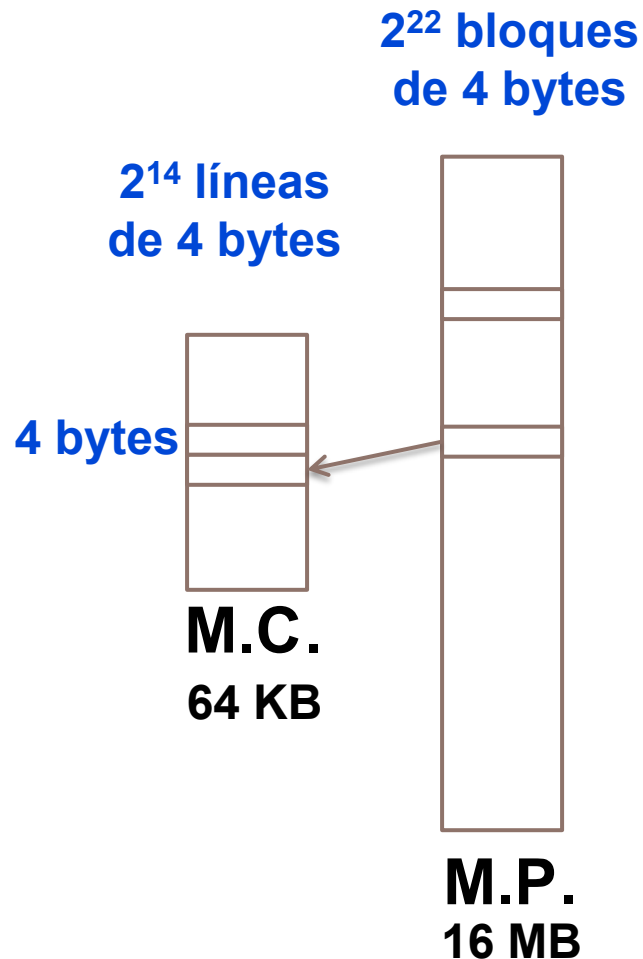
- ▶ Se divide la M.P. y la M.C. en bloques de igual tamaño
- ▶ A cada bloque de M.P. le corresponderá **una línea de M.C.** (bloque en caché)

Estructura y diseño de la cache



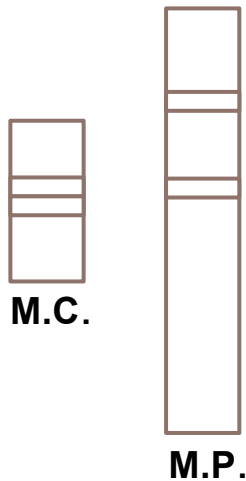
- ▶ Se divide la M.P. y la M.C. en bloques de igual tamaño
- ▶ A cada bloque de M.P. le corresponderá **una línea de M.C.** (bloque en caché)
- ▶ En el diseño se determina:
 - ▶ Tamaño
 - ▶ Función de correspondencia
 - ▶ Algoritmo de sustitución
 - ▶ Política de escritura
- ▶ Es habitual distintos diseños para L1, L2, ...

Estructura y diseño de la cache



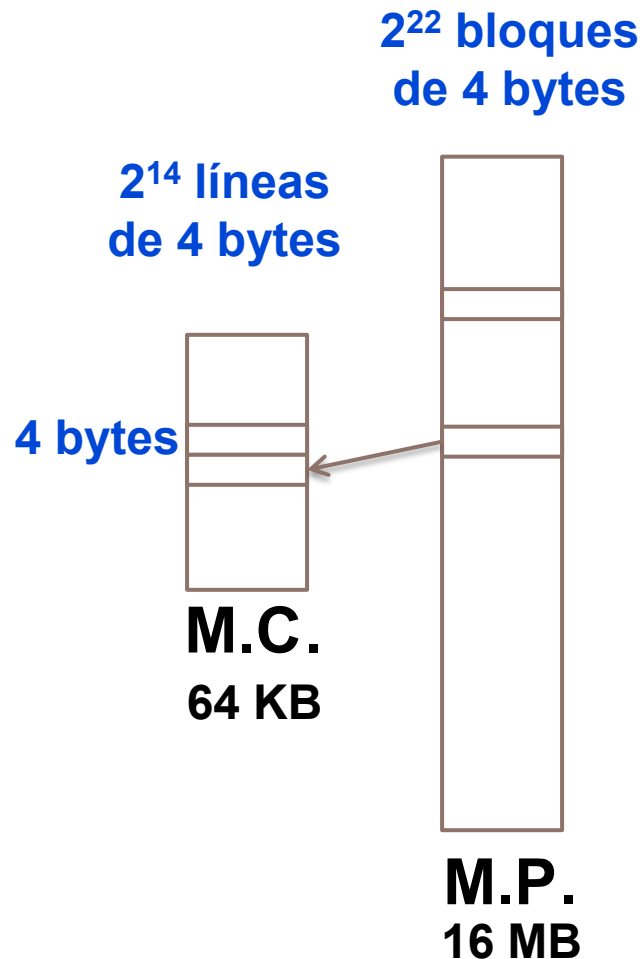
- ▶ Se divide la M.P. y la M.C. en bloques de igual tamaño
- ▶ A cada bloque de M.P. le corresponderá **una línea de M.C.** (bloque en caché)
- ▶ En el diseño se determina:
 - ▶ **Tamaño**
 - ▶ Función de correspondencia
 - ▶ Algoritmo de sustitución
 - ▶ Política de escritura
- ▶ Es habitual distintos diseños para L1, L2, ...

Tamaño de caché



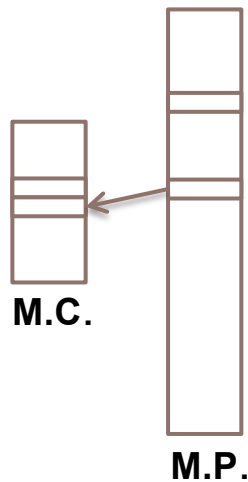
- ▶ **Tamaños:**
 - ▶ Tamaño total de la memoria caché.
 - ▶ Tamaño de la línea en dicha caché.
- ▶ Se determina por estudios sobre códigos muy usados

Estructura y diseño de la cache



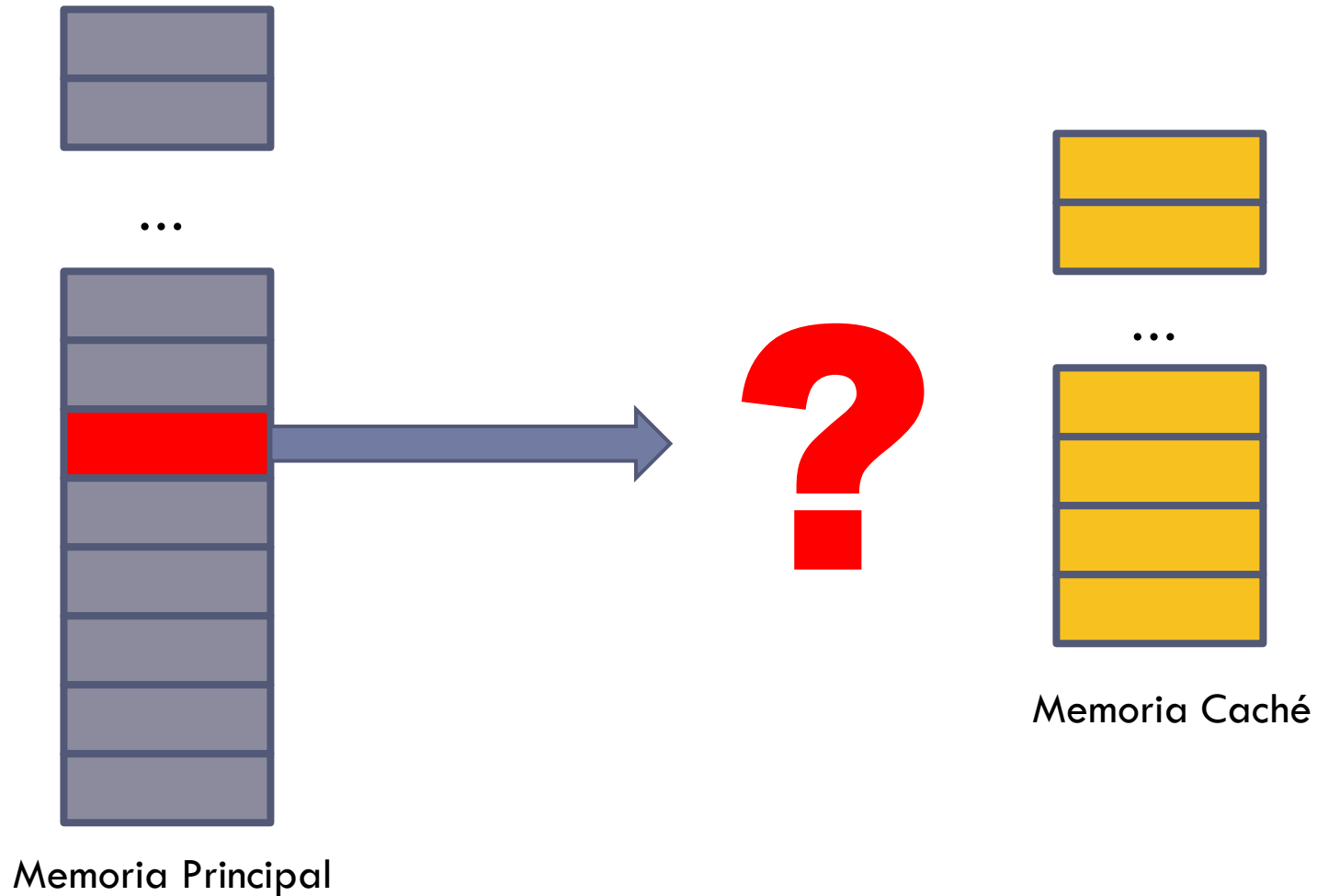
- ▶ Se divide la M.P. y la M.C. en bloques de igual tamaño
- ▶ A cada bloque de M.P. le corresponderá **una línea de M.C.** (bloque en caché)
- ▶ En el diseño se determina:
 - ▶ Tamaño
 - ▶ **Función de correspondencia**
 - ▶ Algoritmo de sustitución
 - ▶ Política de escritura
- ▶ Es habitual distintos diseños para L1, L2, ...

Función de correspondencia

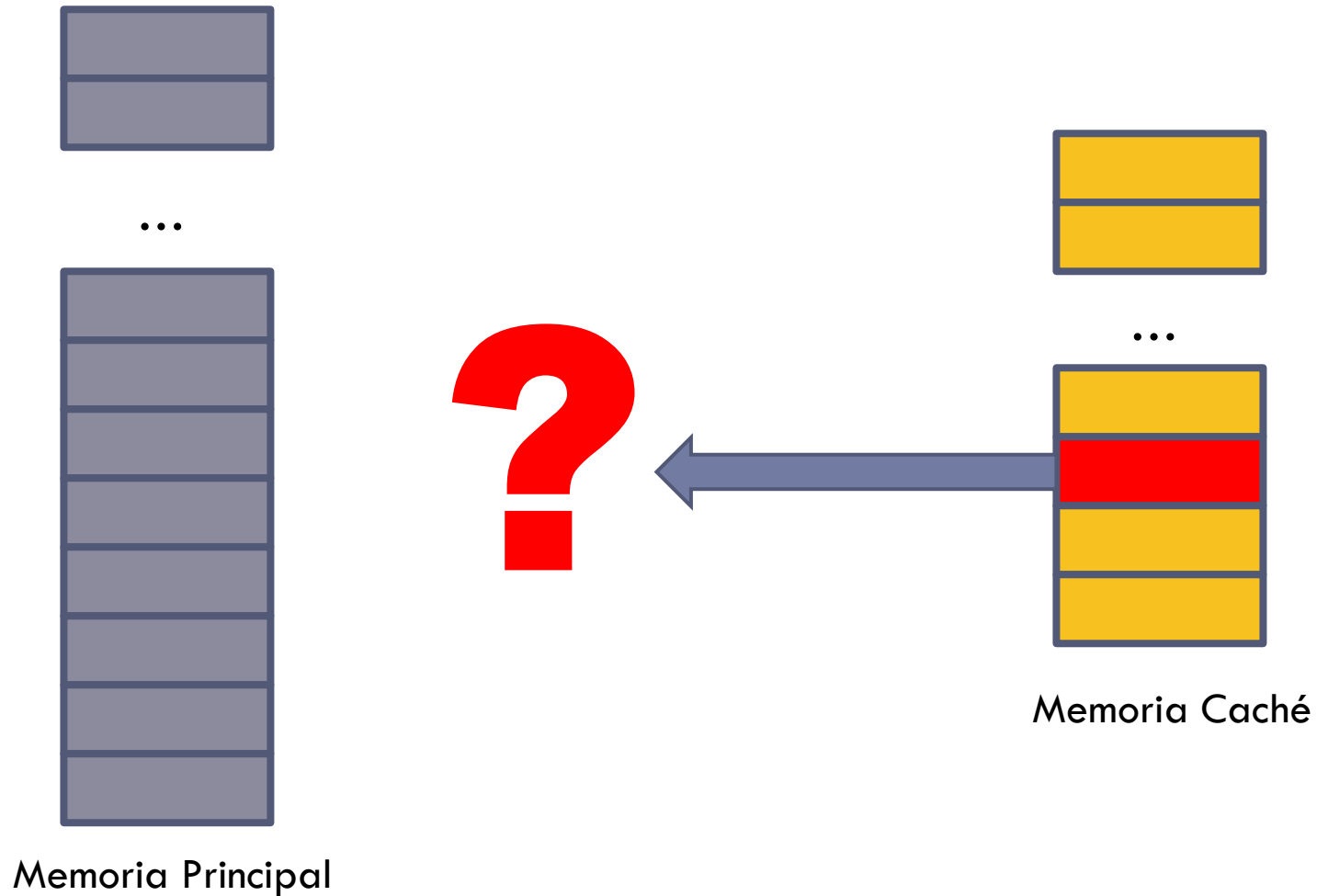


- ▶ Algoritmo que determina, para un bloque concreto de la memoria principal, en qué lugar o lugares de la memoria caché se puede almacenar.
- ▶ Un mecanismo que permita saber para una línea de la memoria caché, qué bloque concreto de memoria principal está en ese momento (o si está libre).
 - ▶ Se asocian a las líneas etiquetas.
 - ▶ Las etiquetas se basan en la dirección de comienzo de la línea.

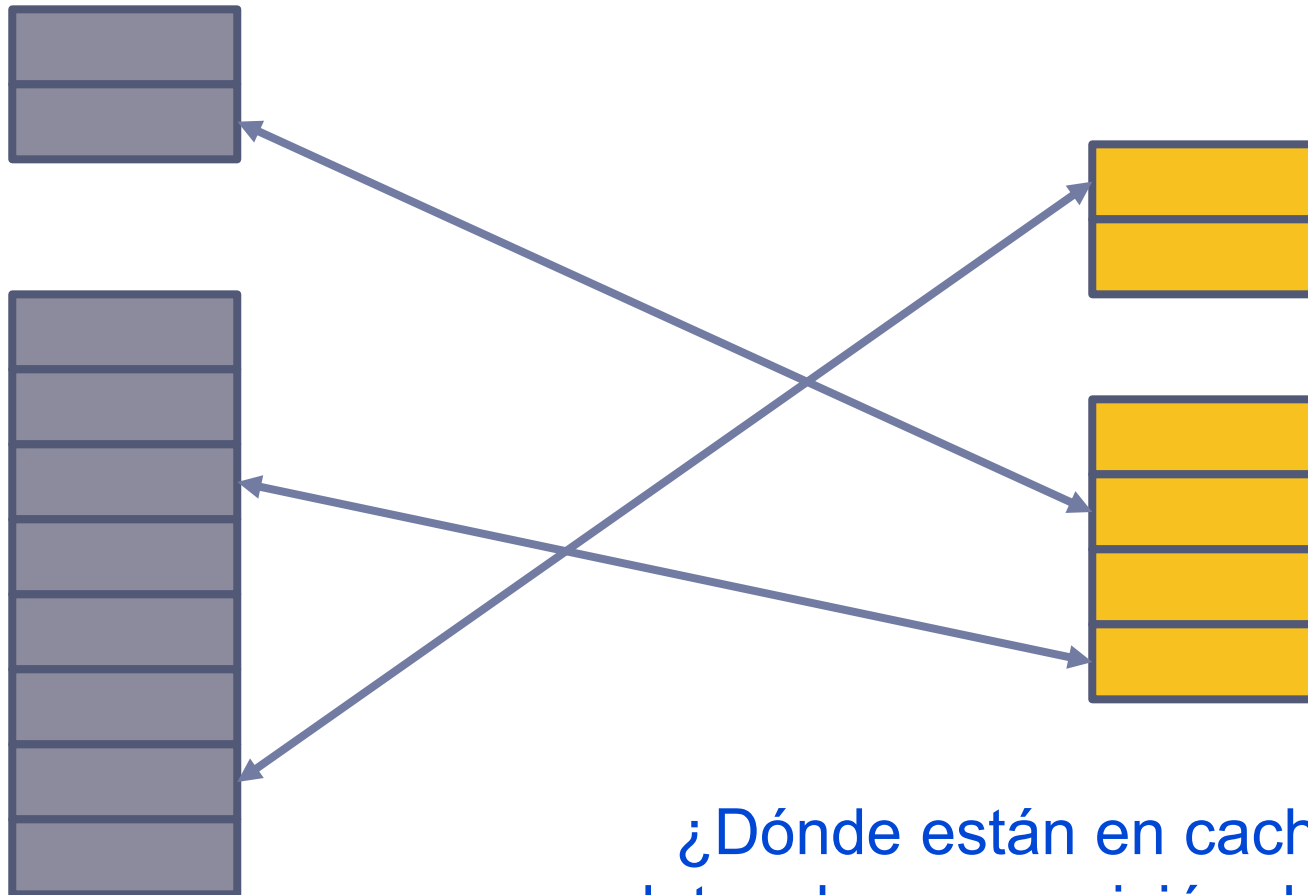
Ubicación en caché



Ubicación en memoria principal



Función de correspondencia



¿Dónde están en caché los datos de una posición de M.P.?

Funciones de correspondencia

- ▶ Función de correspondencia **directa**
- ▶ Función de correspondencia **asociativa**
- ▶ Función de correspondencia **asociativa por conjuntos**

Correspondencia **directa**

00		
01		
10		
11		

Memoria caché
Tamaño: 32 bytes
4 líneas
2 palabras por línea

Memoria principal

Palabra de 32 bits

[illegible]

Correspondencia directa

Bloque 0 - línea 0

00		
01		
10		
11		

Memoria caché
Tamaño: 32 bytes
4 líneas
2 palabras por línea

Memoria principal

Palabra de 32 bits

[illegible]

Correspondencia directa

Bloque 1 - línea 1

00		
01		
10		
11		

Memoria caché
Tamaño: 32 bytes
4 líneas
2 palabras por línea

Memoria principal

Palabra de 32 bits

[illegible]

Correspondencia directa

Bloque 2 - línea 2

00		
01		
10		
11		

Memoria caché
Tamaño: 32 bytes
4 líneas
2 palabras por línea

Memoria principal

Palabra de 32 bits

0
4
8
12
16
20
24
28
32
36
40
44
48
52
56
60
64
.
.
.

Correspondencia directa

Bloque 3 - línea 3

00		
01		
10		
11		

Memoria caché
Tamaño: 32 bytes
4 líneas
2 palabras por línea

Memoria principal

Palabra de 32 bits

0
4
8
12
16
20
24
28
32
36
40
44
48
52
56
60
64
.
.
.

Correspondencia directa

Bloque 4 - línea 0

00		
01		
10		
11		

Memoria caché
Tamaño: 32 bytes
4 líneas
2 palabras por línea

Memoria principal

Palabra de 32 bits

0
4
8
12
16
20
24
28
32
36
40
44
48
52
56
60
64
.
.
.

Correspondencia directa

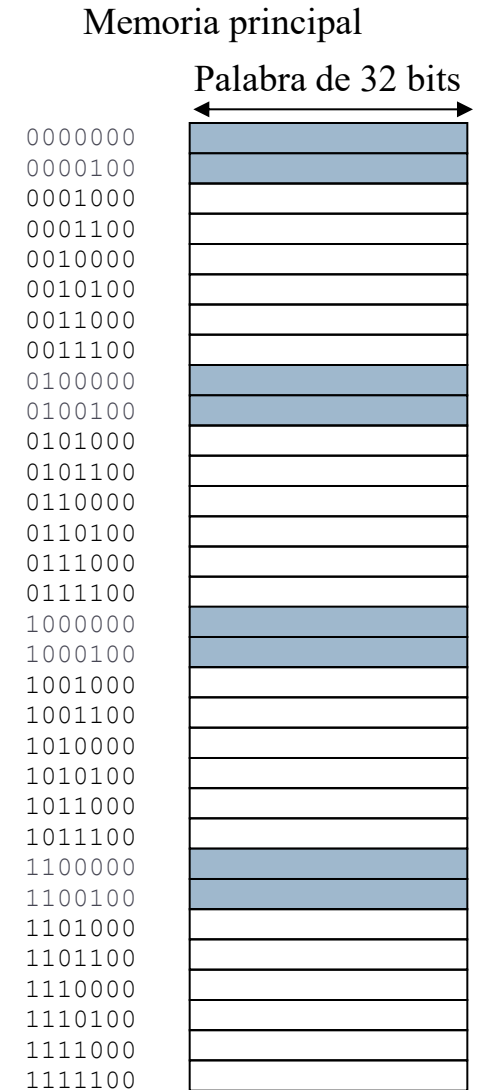
► En general:

- En una memoria caché con NL número de líneas, el bloque de memoria K se almacena en la línea:

$$K \bmod NL$$

00		
01		
10		
11		

Memoria caché
Tamaño: 32 bytes
4 líneas
2 palabras por línea



Correspondencia directa

► En general:

- En una memoria caché con NL número de líneas, el bloque de memoria K se almacena en la línea:

$$K \bmod NL$$

00		
01		
10		
11		

Memoria caché
Tamaño: 32 bytes
4 líneas
2 palabras por línea

¡ Varios bloques en la misma línea !

Memoria principal

	Palabra de 32 bits
0000000	
0000100	
0001000	
0001100	
0010000	
0010100	
0011000	
0011100	
0100000	
0100100	
0101000	
0101100	
0110000	
0110100	
0111000	
0111100	
1000000	
1000100	
1001000	
1001100	
1010000	
1010100	
1011000	
1011100	
1100000	
1100100	
1101000	
1101100	
1110000	
1110100	
1111000	
1111100	

Correspondencia directa

00		
01		
10		
11		

Memoria caché
Tamaño: 32 bytes
4 líneas
2 palabras por línea

¿Cómo se sabe **qué** bloque de memoria se encuentra una determinada línea?
Ejemplo: la dirección 0100100

Memoria principal	
	Palabra de 32 bits
0000000	
0000100	
0001000	
0001100	
0010000	
0010100	
0011000	
0011100	
0100000	
0100100	
0101000	
0101100	
0110000	
0110100	
0111000	
0111100	
1000000	
1000100	
1001000	
1001100	
1010000	
1010100	
1011000	
1011100	
1100000	
1100100	
1101000	
1101100	
1110000	
1110100	
1111000	
1111100	

Correspondencia directa

00			tag
01			tag
10			tag
11			tag

Memoria caché
Tamaño: 32 bytes
4 líneas
2 palabras por línea

¿Cómo se sabe **qué** bloque de memoria se encuentra una determinada línea?

Ejemplo: la dirección 0100100

Se añade a cada línea una **etiqueta**

Memoria principal	
	Palabra de 32 bits
0000000	
0000100	
0001000	
0001100	
0010000	
0010100	
0011000	
0011100	
0100000	
0100100	
0101000	
0101100	
0110000	
0110100	
0111000	
0111100	
1000000	
1000100	
1001000	
1001100	
1010000	
1010100	
1011000	
1011100	
1100000	
1100100	
1101000	
1101100	
1110000	
1110100	
1111000	
1111100	

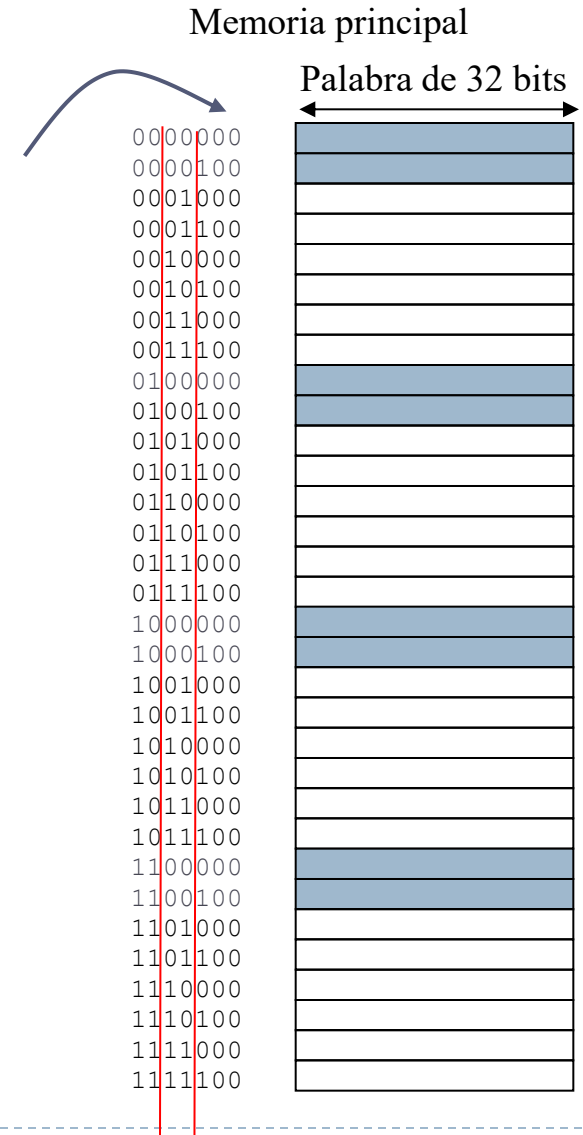
Correspondencia directa

Idea

¿qué byte dentro de la línea?
Líneas de 8 bytes

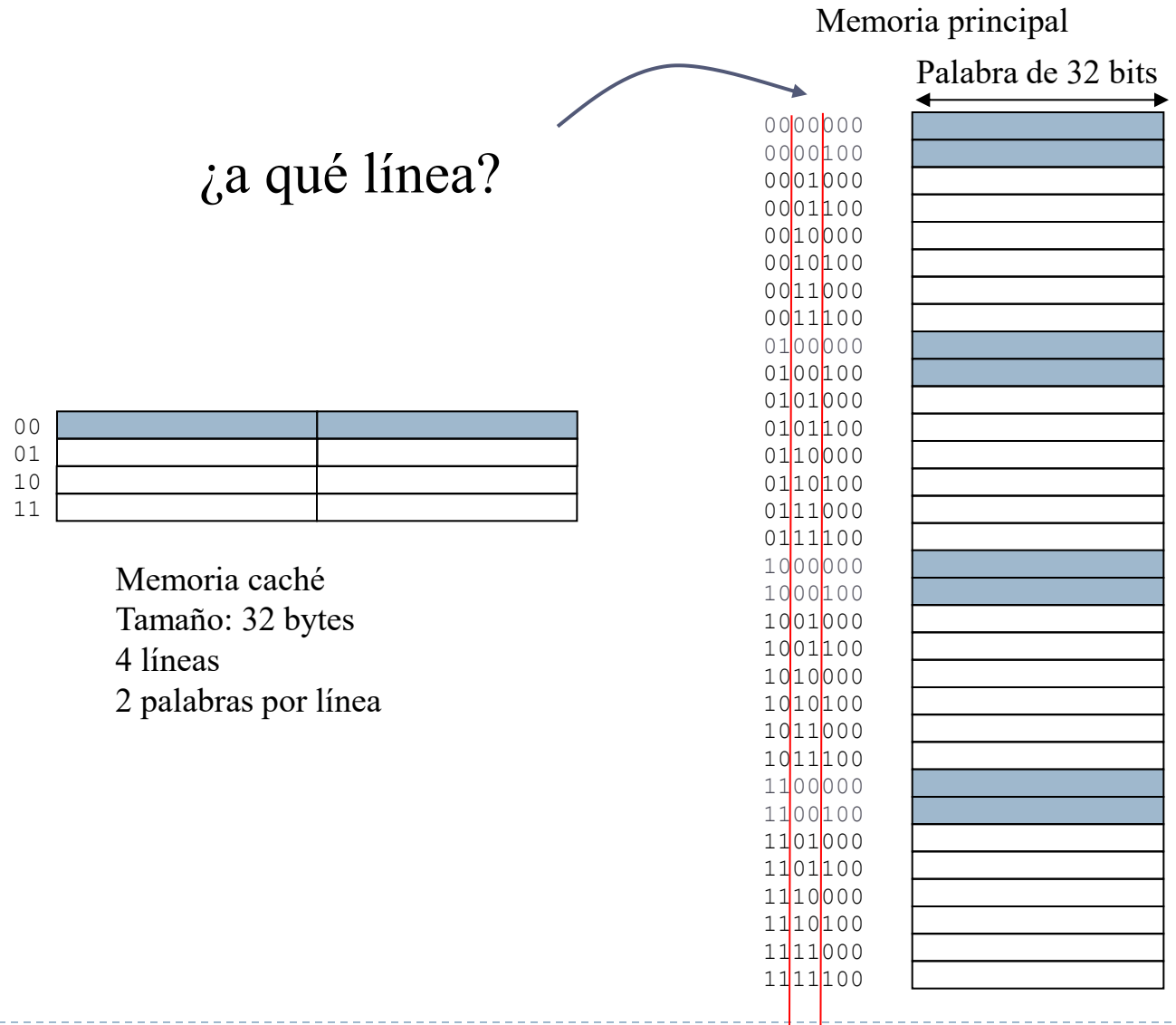
00		
01		
10		
11		

Memoria caché
Tamaño: 32 bytes
4 líneas
2 palabras por línea



Correspondencia directa

Idea



Correspondencia directa

Idea

etiqueta asociada a la línea
que diferencia los bloques
que van a la misma línea

00			
01			
10			
11			

Memoria caché
Tamaño: 32 bytes
4 líneas
2 palabras por línea

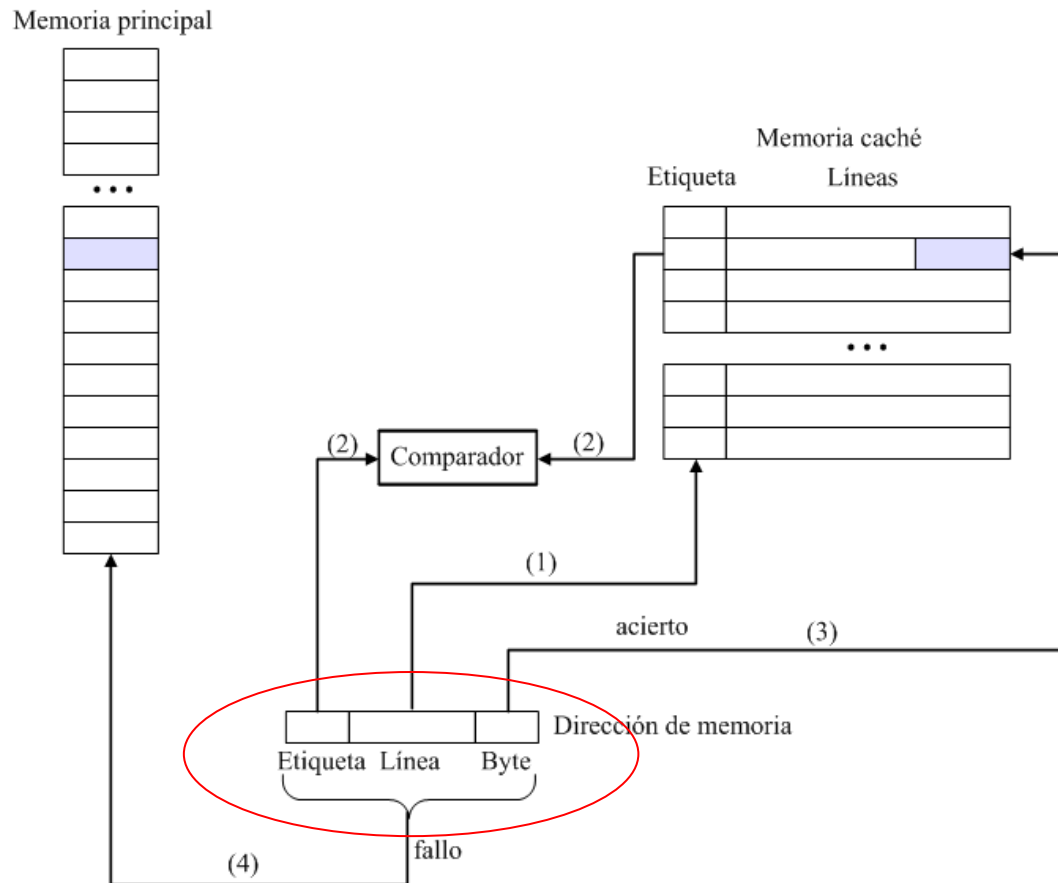
La etiqueta (**bits superiores de la dirección**)
se almacena también en la memoria caché

Memoria principal

Palabra de 32 bits

00	00000	
00	00100	
00	01000	
00	01100	
00	10000	
00	10100	
00	11000	
00	11100	
01	00000	
01	00100	
01	01000	
01	01100	
01	10000	
01	10100	
01	11000	
01	11100	
10	00000	
10	00100	
10	01000	
10	01100	
10	10000	
10	10100	
10	11000	
10	11100	
11	00000	
11	00100	
11	01000	
11	01100	
11	10000	
11	10100	
11	11000	
11	11100	

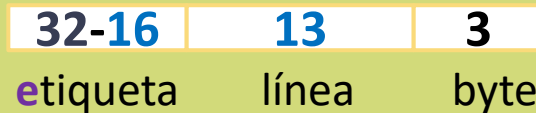
Organización de una memoria caché con correspondencia directa



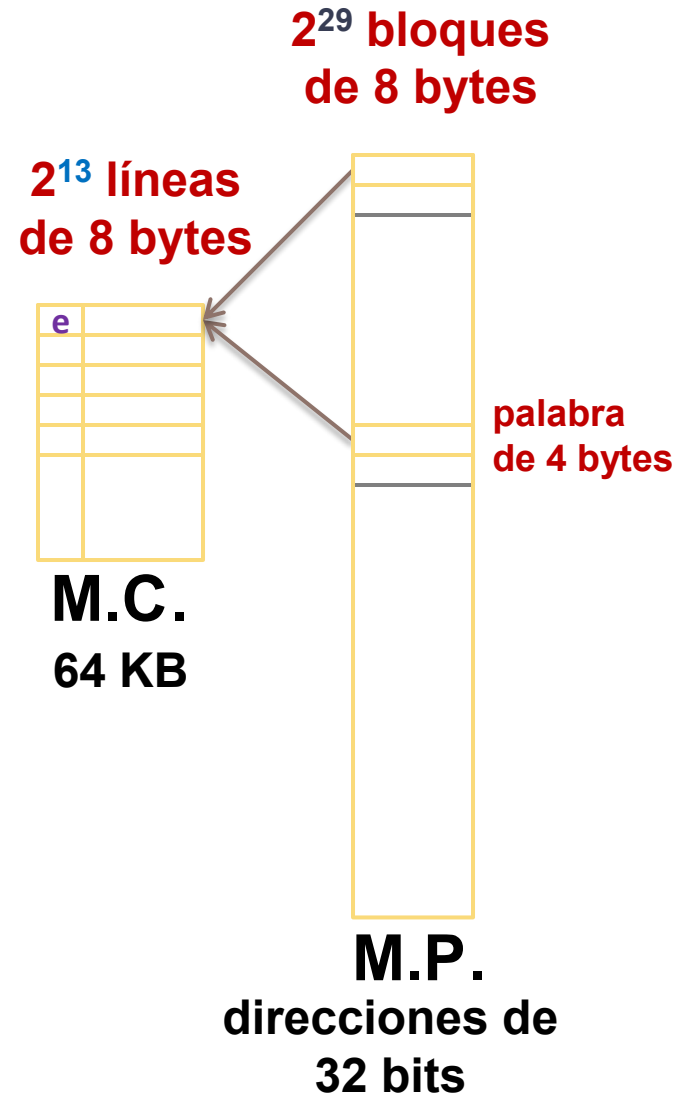
Función de correspondencia directa

Ejemplo

- ▶ Cada bloque de M.P. le corresponde una sola línea de caché (siempre la misma)
- ▶ La dirección de M.P. la determina:



- ▶ Si la etiqueta de 'línea' es 'etiqueta', entonces está el bloque en caché
- ▶ Simple, poco costosa, pero puede provocar muchos fallos dependiendo del patrón de accesos

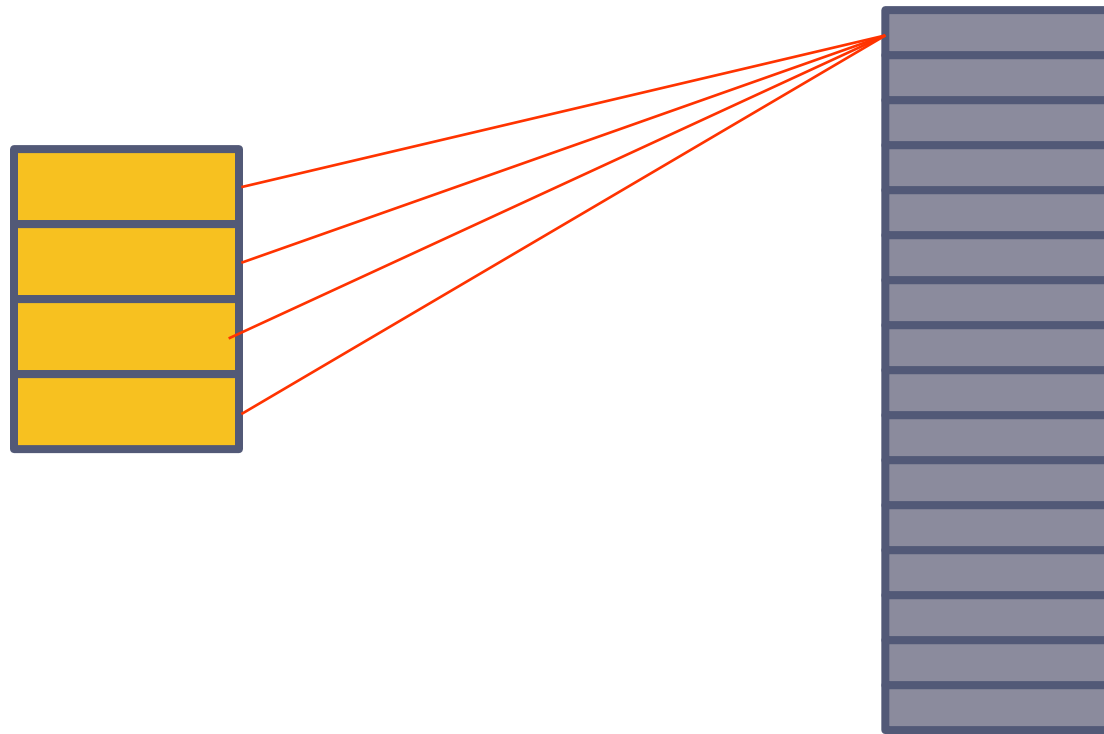


Ejercicio

- ▶ Dado un computador de 32 bits con una memoria caché de 64 KB y bloques de 32 bytes. Si se utiliza correspondencia directa
 - ▶ ¿En qué línea de la memoria caché se almacena la palabra de la dirección 0x0000408A?
 - ▶ ¿Cómo se puede obtener rápidamente?
 - ▶ ¿En qué línea de la memoria caché se almacena la palabra de la dirección 0x1000408A?
 - ▶ ¿Cómo sabe la caché si la palabra almacenada en esa línea corresponde a la palabra de la dirección 0x0000408A o a la palabra de la dirección 0x1000408A?

Correspondencia **asociativa**

- ▶ Cada bloque de M.P. puede almacenarse en cualquier línea de la caché



Correspondencia asociativa

Idea

etiqueta asociada a la línea
que diferencia los bloques
que van a la misma línea

00			
01			
10			
11			

Memoria caché
Tamaño: 32 bytes
4 líneas
2 palabras por línea

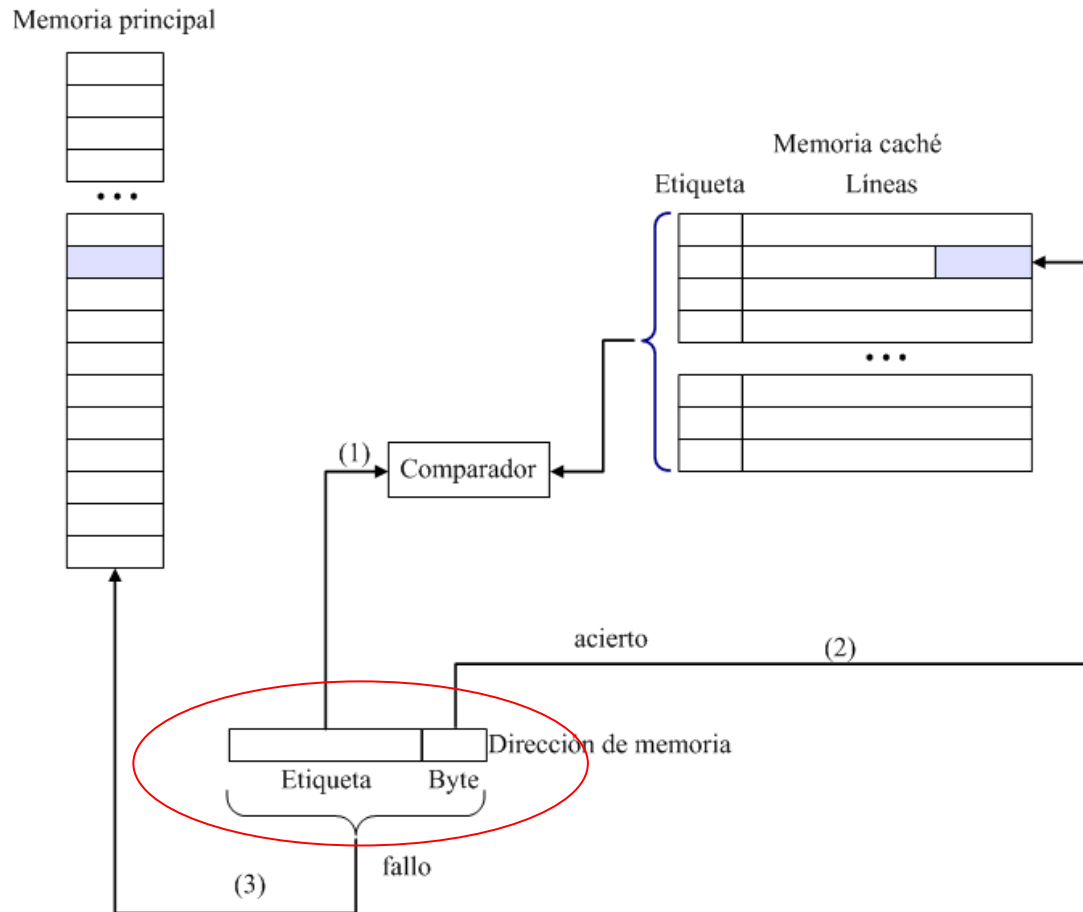
La etiqueta (bits superiores de la dirección)
se almacena también en la memoria caché

Memoria principal

Palabra de 32 bits

0000	000	
0000	100	
0001	000	
0001	100	
0010	000	
0010	100	
0011	000	
0011	100	
0100	000	
0100	100	
0101	000	
0101	100	
0110	000	
0110	100	
0111	000	
0111	100	
1000	000	
1000	100	
1001	000	
1001	100	
1010	000	
1010	100	
1011	000	
1011	100	
1100	000	
1100	100	
1101	000	
1101	100	
1110	000	
1110	100	
1111	000	
1111	100	

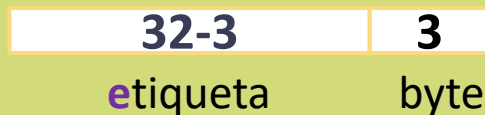
Organización de una memoria caché con correspondencia asociativa



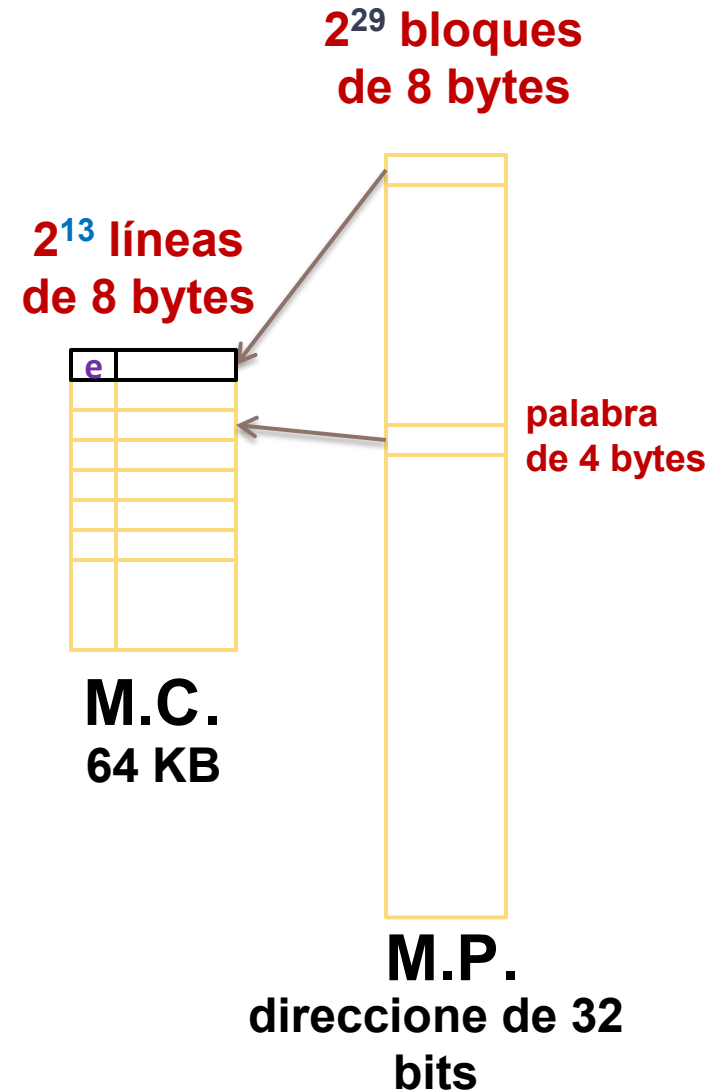
Función de correspondencia asociativa

Ejemplo

- ▶ Un bloque de M.P. puede cargarse en cualquier línea de caché
- ▶ La dir. de M.P. se interpreta como:



- ▶ Si hay una línea con 'etiqueta' en la caché, está allí el bloque
- ▶ Independiente del patrón de acceso, búsqueda costosa
- ▶ Etiquetas más grandes: cachés más grandes



Correspondencia **asociativa por conjuntos**

- ▶ La memoria se organiza en **conjuntos** de líneas
 - ▶ Las líneas en un conjunto se le denominan **vías**.
- ▶ Una memoria **caché asociativa por conjunto de K vías**:
 - ▶ Cada conjunto almacena K líneas
- ▶ Cada bloque siempre se almacena en el mismo conjunto...
 - ▶ El bloque B se almacena en el conjunto:
 - ▶ $B \bmod$ número de conjuntos
- ▶ ...dentro de un conjunto el bloque se puede almacenar en cualquiera de las líneas de ese conjunto

Correspondencia asociativa por conjuntos

Número de conjunto

Conjunto 0
Conjunto 1

Número de línea

00		
01		
10		
11		

Memoria caché

Tamaño: 32 bytes

Asociativa por conjunto de 2 vías

2 líneas por conjunto

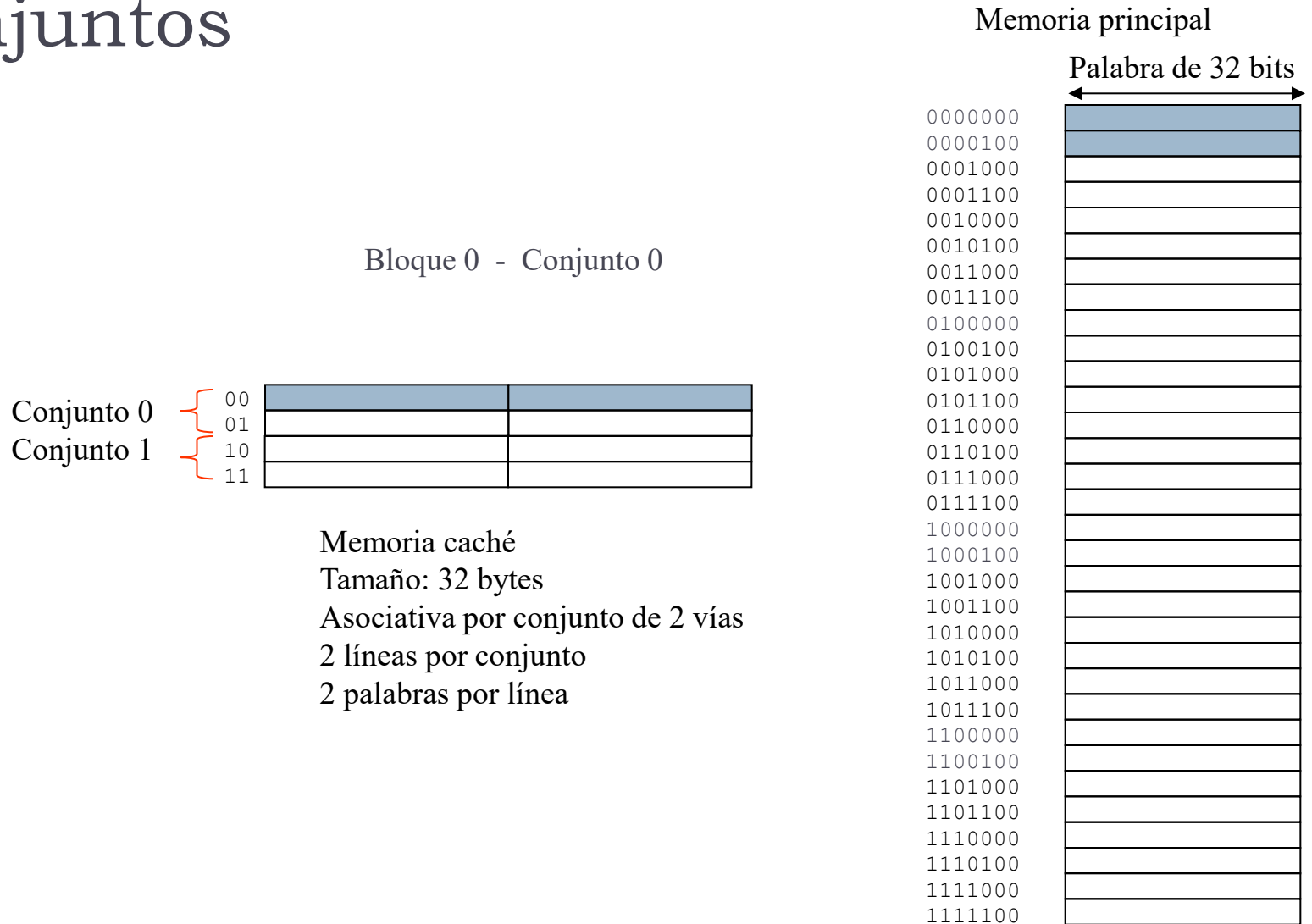
2 palabras por línea

Memoria principal

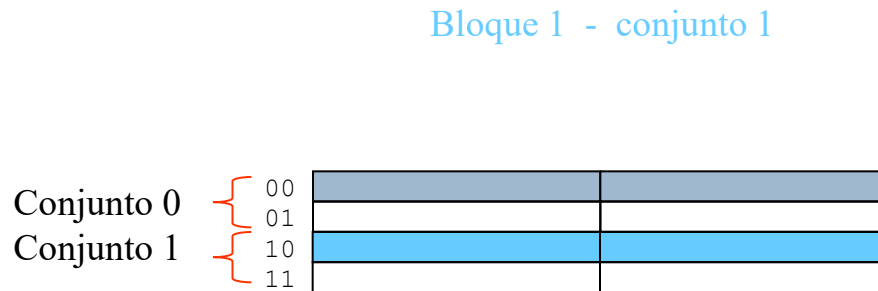
Palabra de 32 bits

0000000	
0000100	
0001000	
0001100	
0010000	
0010100	
0011000	
0011100	
0100000	
0100100	
0101000	
0101100	
0110000	
0110100	
0111000	
0111100	
1000000	
1000100	
1001000	
1001100	
1010000	
1010100	
1011000	
1011100	
1100000	
1100100	
1101000	
1101100	
1110000	
1110100	
1111000	
1111100	

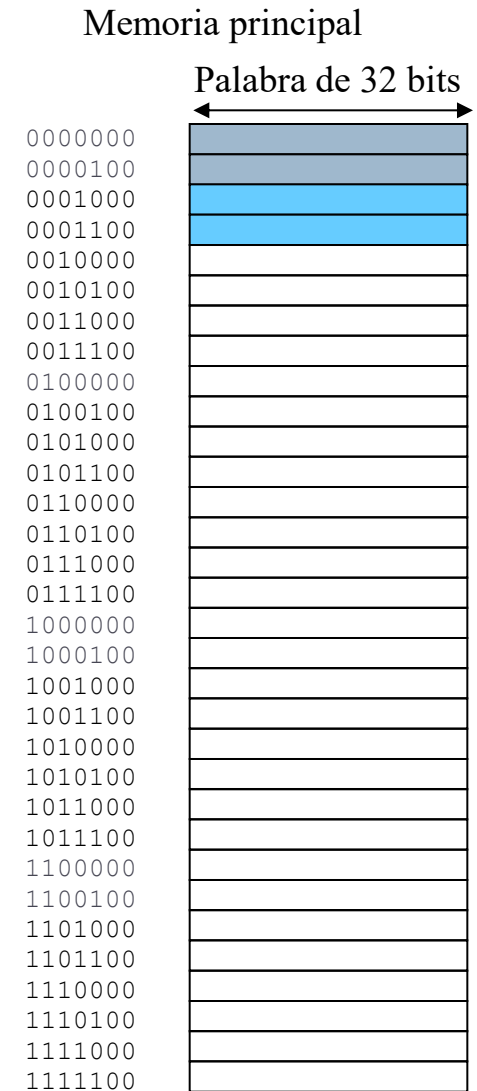
Correspondencia asociativa por conjuntos



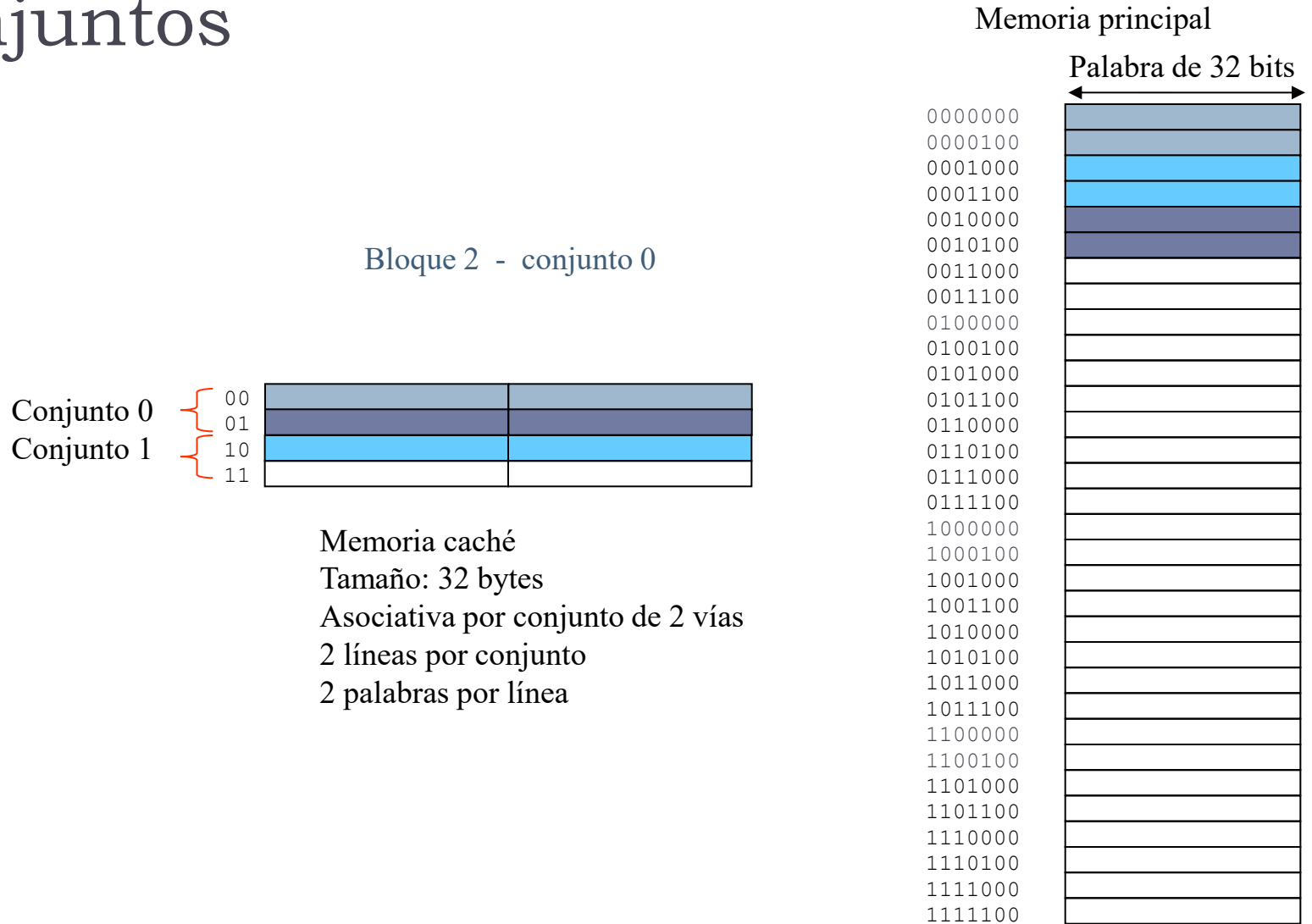
Correspondencia asociativa por conjuntos



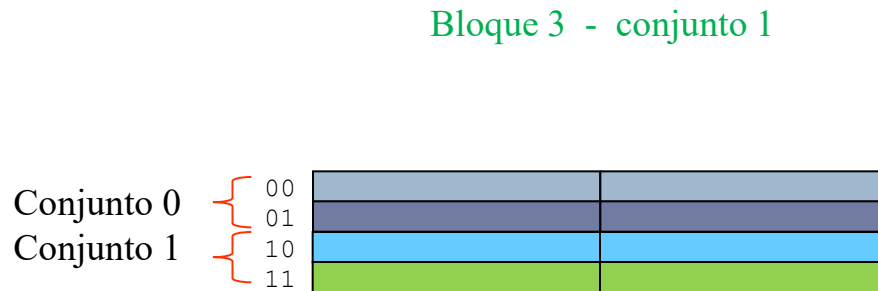
Memoria caché
 Tamaño: 32 bytes
 Asociativa por conjunto de 2 vías
 2 líneas por conjunto
 2 palabras por línea



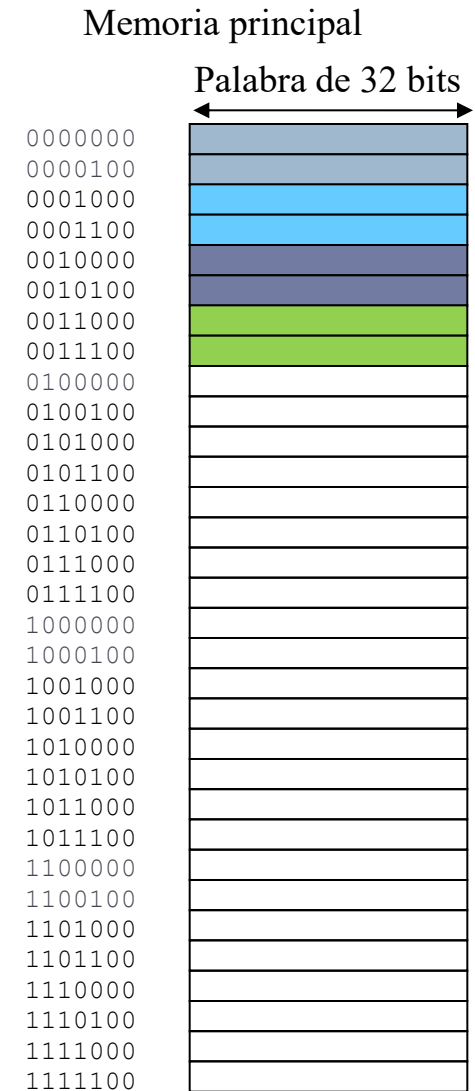
Correspondencia asociativa por conjuntos



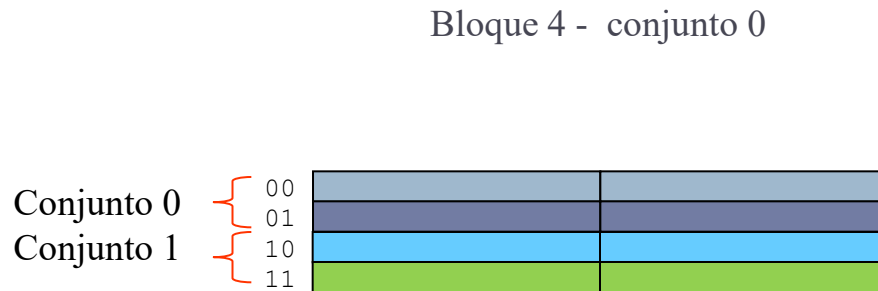
Correspondencia asociativa por conjuntos



Memoria caché
Tamaño: 32 bytes
Asociativa por conjunto de 2 vías
2 líneas por conjunto
2 palabras por línea

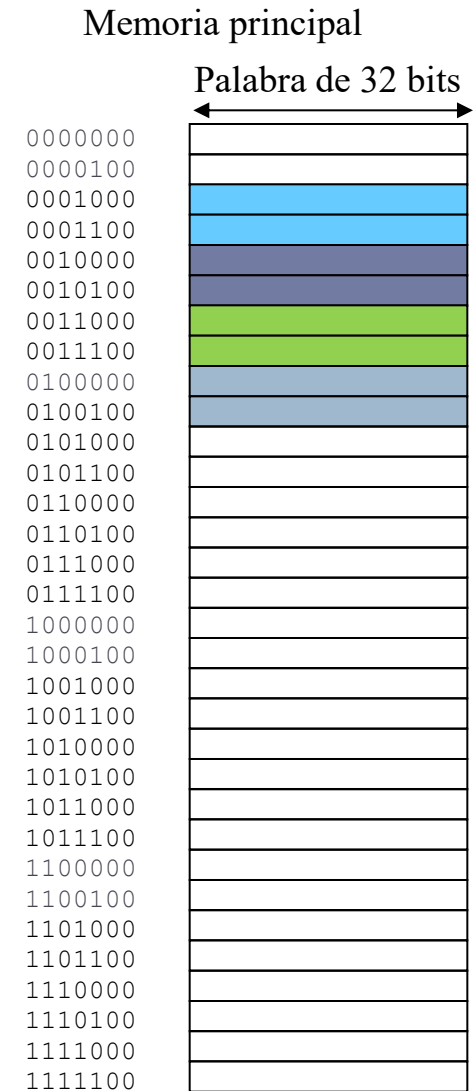


Correspondencia asociativa por conjuntos



Memoria caché
Tamaño: 32 bytes
Asociativa por conjunto de 2 vías
2 líneas por conjunto
2 palabras por línea

Habría que eliminar la línea que estaba antes

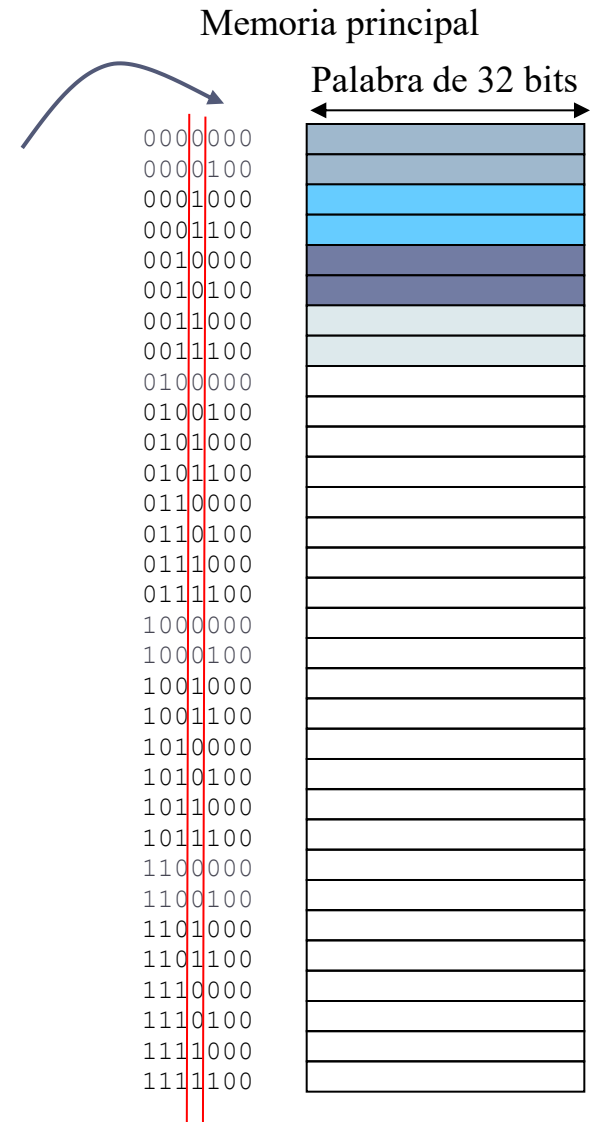


Correspondencia asociativa por conjuntos

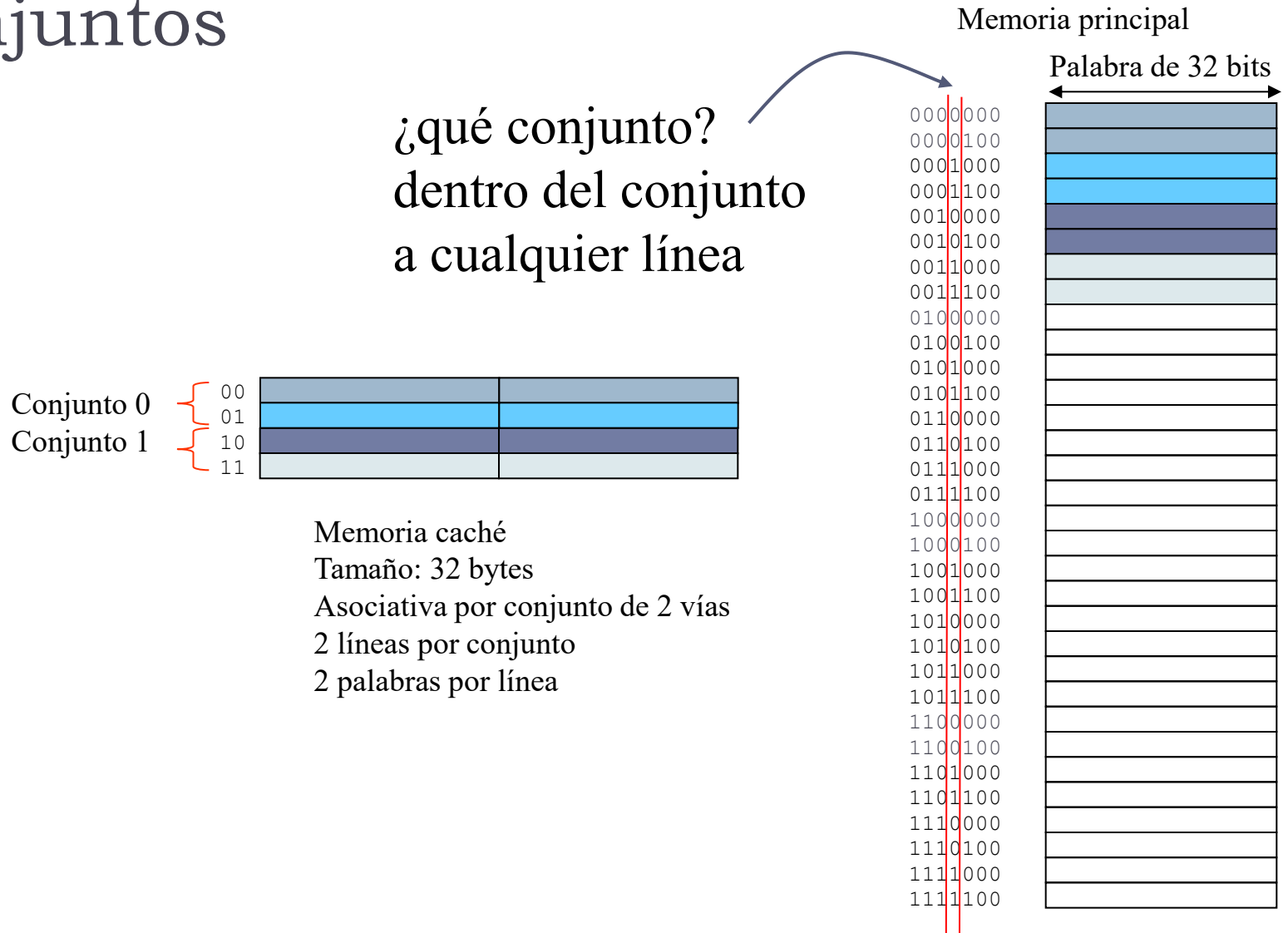
¿qué byte dentro de la línea?
Líneas de 8 bytes



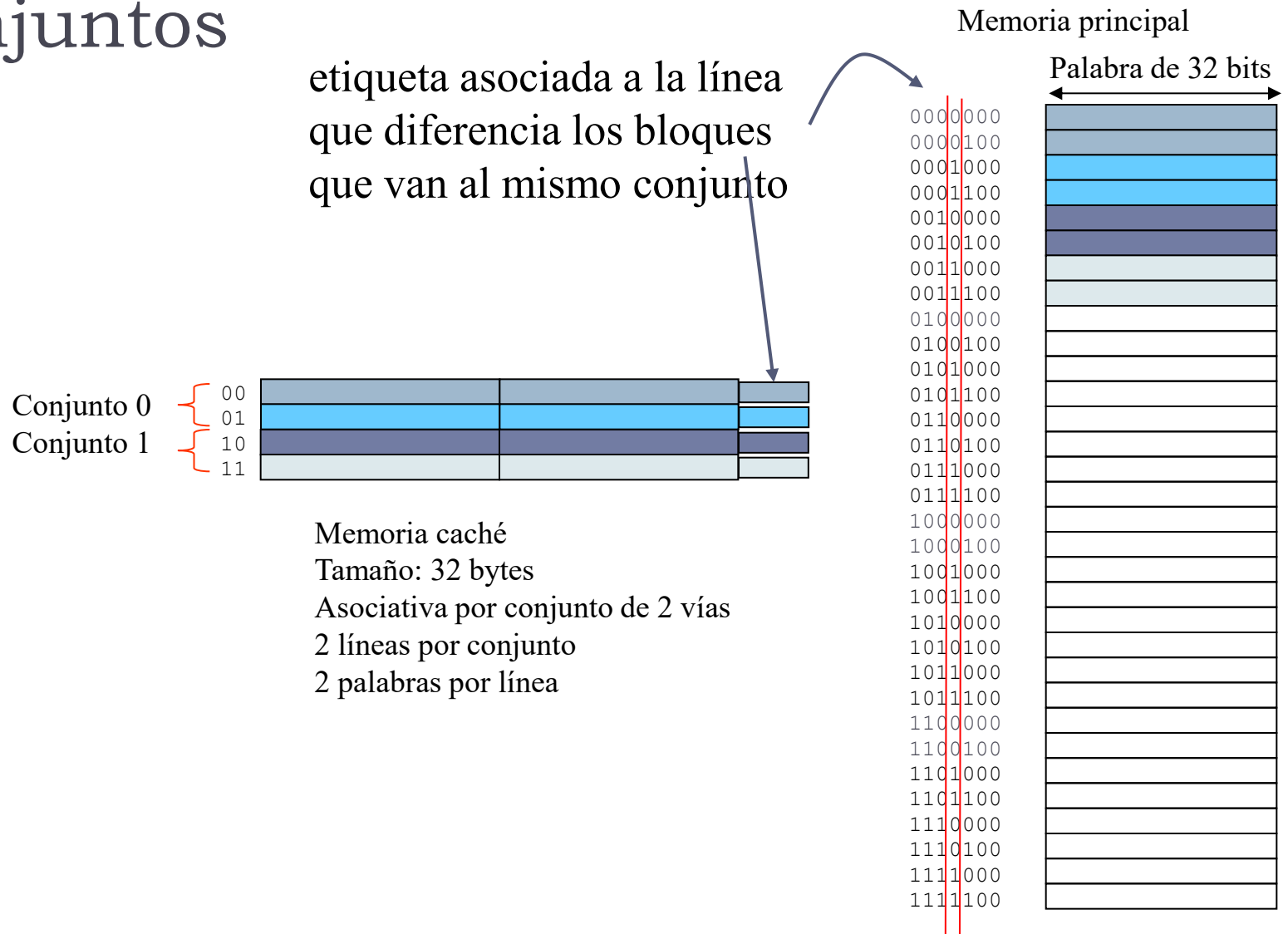
Memoria caché
Tamaño: 32 bytes
Asociativa por conjunto de 2 vías
2 líneas por conjunto
2 palabras por línea



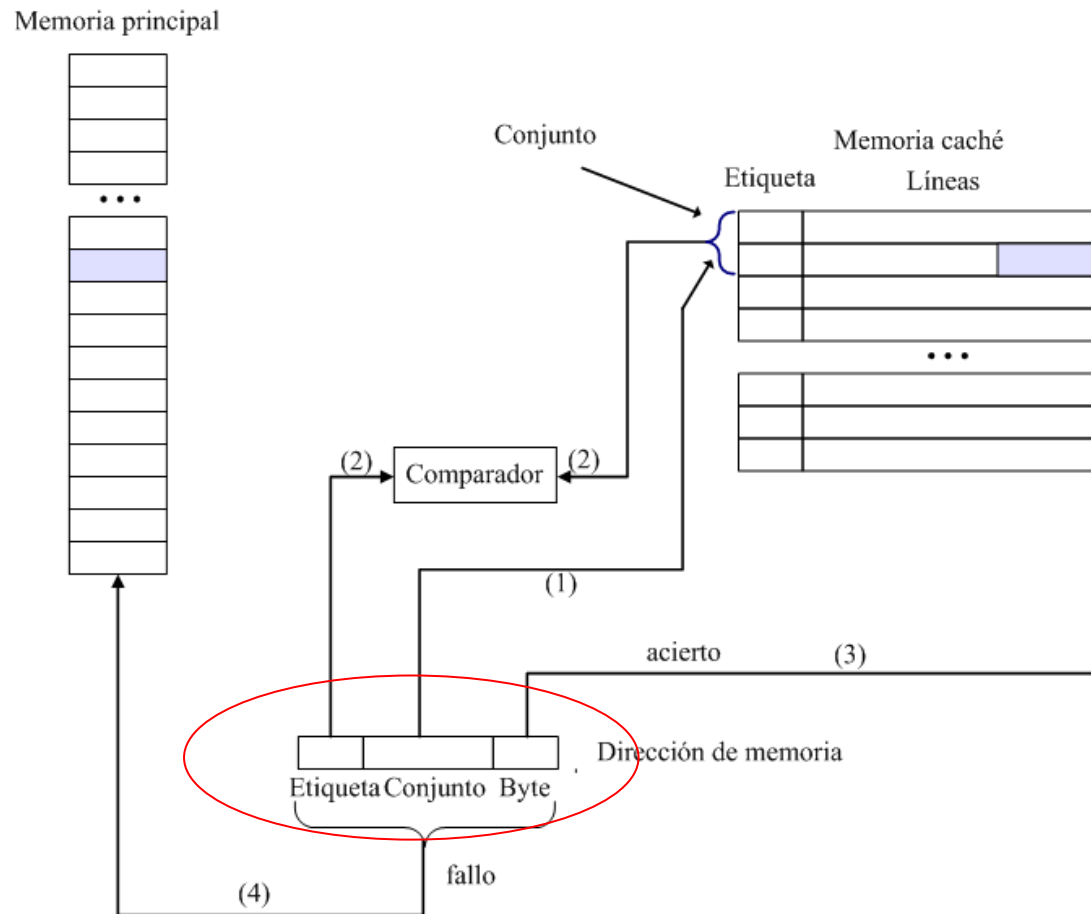
Correspondencia asociativa por conjuntos



Correspondencia asociativa por conjuntos



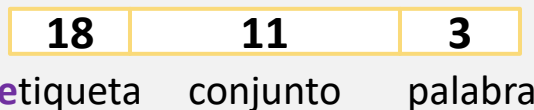
Organización de una memoria caché asociativa por conjuntos



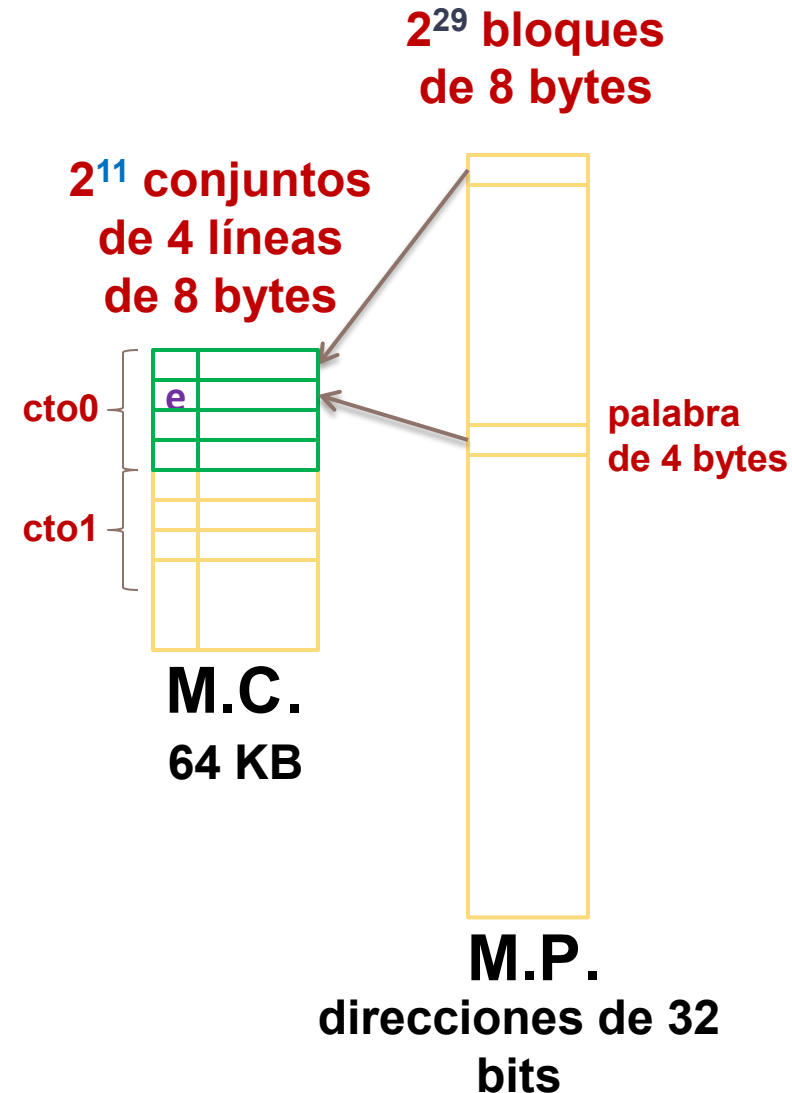
Función de correspondencia asociativa por conjuntos. Ejemplo

► Asociativa por conjuntos:

- Un bloque de M.P. puede cargarse en cualquier línea de caché (vía) de un conjunto determinado
- La dir. de M.P. se interpreta como:



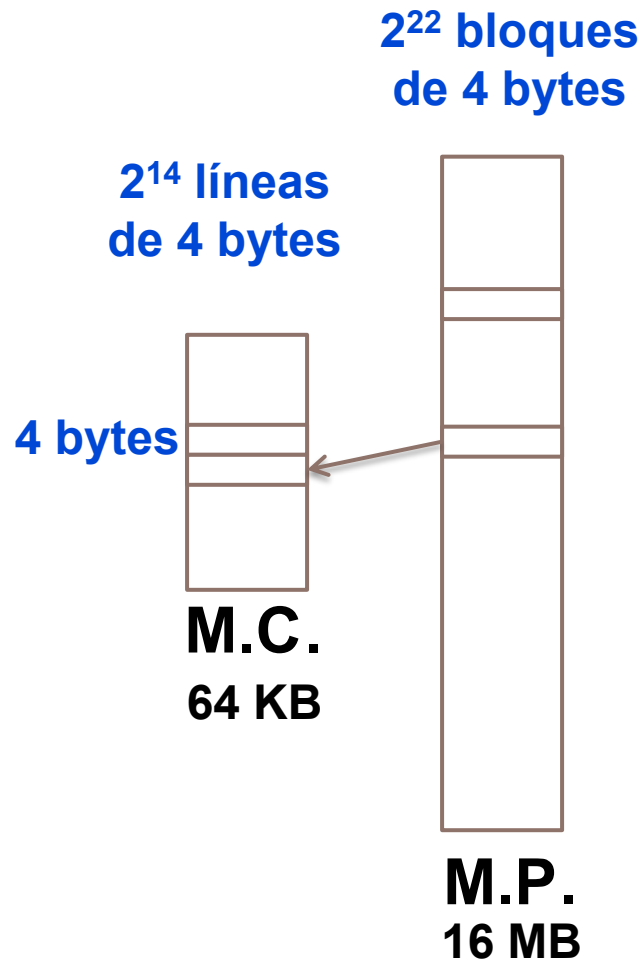
- Si hay una línea con 'etiqueta' en el conjunto 'conjunto', está allí el bloque en caché
- [V] lo mejor de directa y asociativa
[I] búsqueda menos costosa



Correspondencia asociativa por conjuntos

- ▶ **Establece un compromiso entre flexibilidad y coste:**
 - ▶ Es más flexible que la correspondencia directa.
 - ▶ Es menos costosa que la correspondencia asociativa.
- ▶ **Otras políticas pueden ser vistas como casos particulares:**
 - ▶ Directa: asociativa por conjuntos con un conjunto por línea.
 - ▶ Totalmente asociativa: asociativa por conjuntos con un solo conjunto.

Estructura y diseño de la cache



- ▶ Se divide la M.P. y la M.C. en bloques de igual tamaño
- ▶ A cada bloque de M.P. le corresponderá **una línea de M.C.** (bloque en caché)
- ▶ En el diseño se determina:
 - ▶ Tamaño
 - ▶ Función de correspondencia
 - ▶ **Algoritmo de sustitución**
 - ▶ Política de escritura
- ▶ Es habitual distintos diseños para L1, L2, ...

Sustitución de bloques

- ▶ Cuando todas las entradas de la caché contienen bloques de memoria principal:
 - ▶ Hace falta seleccionar una línea que hay que dejar libre para traer un bloque de la M.P.
 - ▶ Directa: no hay posible elección
 - ▶ Asociativa: seleccionar una línea de la caché.
 - ▶ Asociativa por conjuntos: seleccionar una línea del conjunto seleccionado.
- ▶ Existen diversos algoritmos para seleccionar la línea de la caché que hay que liberar (para asociativa y asociativa por conjuntos)

Algoritmos de sustitución

- ▶ **FIFO**

- ▶ *First-In-First-Out*
- ▶ Sustituye la línea que lleva más tiempo en la caché.

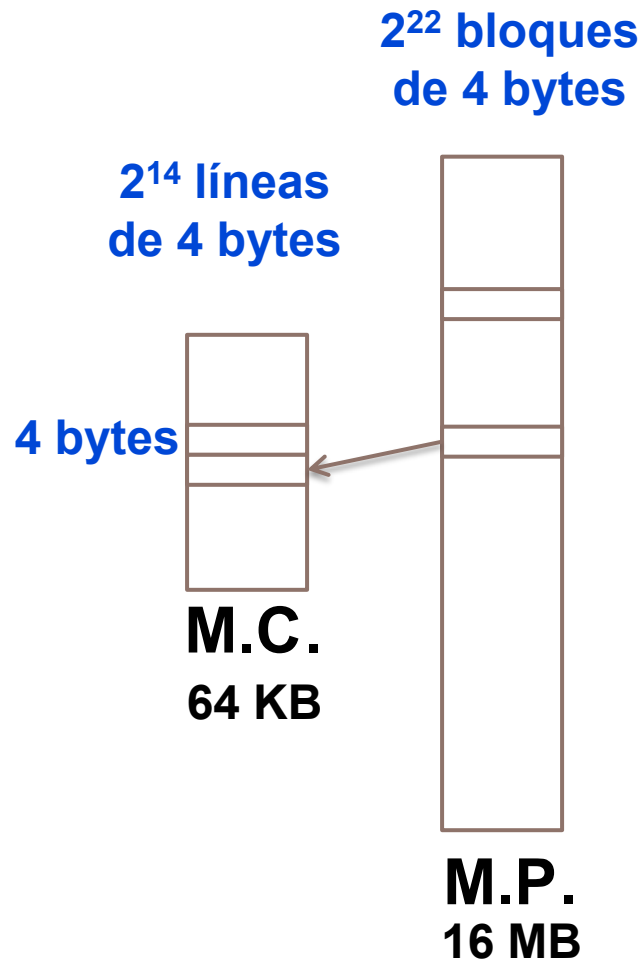
- ▶ **LRU:**

- ▶ *Least Recently Used*
- ▶ Sustituye la línea que lleva más tiempo sin usarse.

- ▶ **LFU:**

- ▶ *Least Frequently Used*
- ▶ Sustituye la línea que se ha usado menos veces.

Estructura y diseño de la cache



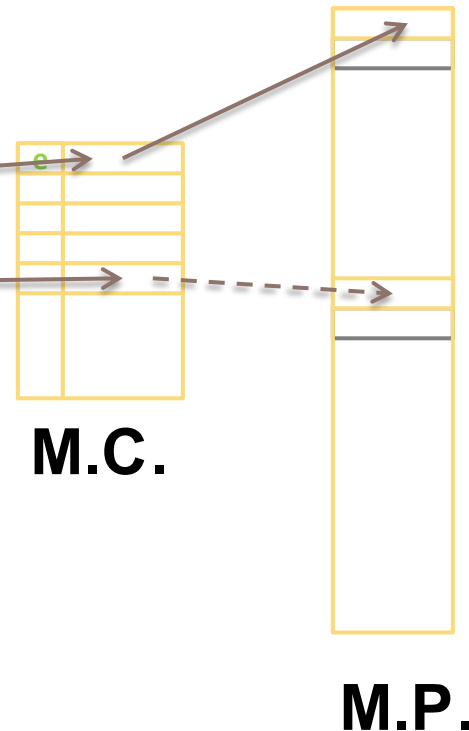
- ▶ Se divide la M.P. y la M.C. en bloques de igual tamaño
- ▶ A cada bloque de M.P. le corresponderá **una línea de M.C.** (bloque en caché)
- ▶ En el diseño se determina:
 - ▶ Tamaño
 - ▶ Función de correspondencia
 - ▶ Algoritmo de sustitución
 - ▶ **Política de escritura**
- ▶ Es habitual distintos diseños para L1, L2, ...

Políticas de escritura

- ▶ Cuando se modifica un dato en memoria caché, **en caso de acierto**, hay que actualizar en algún momento la memoria principal

- ▶ Alternativas:

- ▶ Escritura inmediata (*write through*).
- ▶ Post-escritura (*write back*).



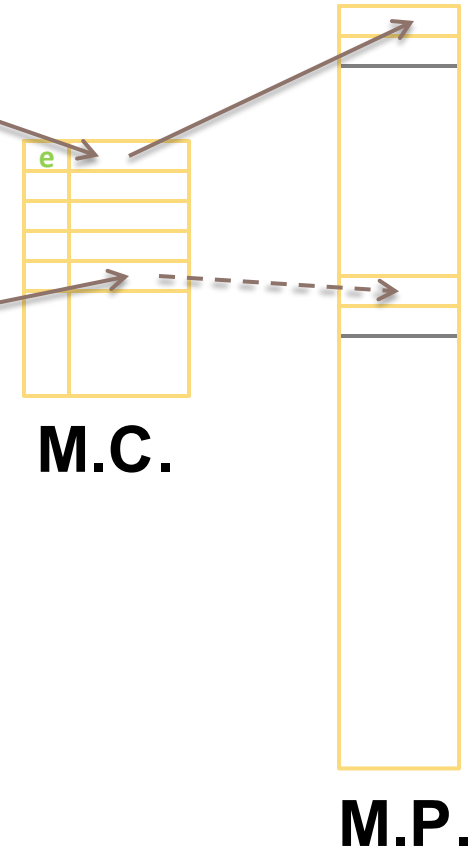
Políticas de escritura

► Escritura **inmediata**:

- La escritura se hace tanto en M.P. como en cache
- [V] Coherencia
- [I] Mucho tráfico
- [I] Escrituras lentas

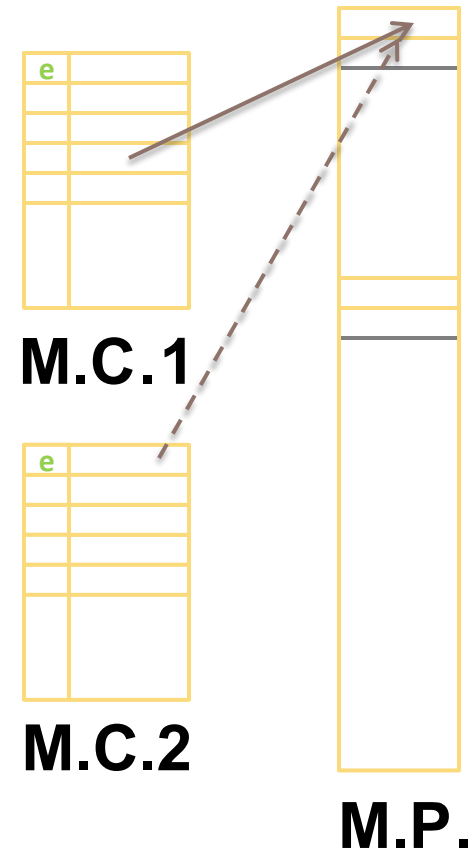
► Escritura **diferida**:

- La escritura solo se hace en la caché, indicando en un bit que no está volcada la línea en M.P.
- Al sustituir el bloque (o cuando ↓ tráfico con M.P.) se escribe en M.P.
- [V] Velocidad
- [I] Coherencia + inconsistencia



Políticas de escritura

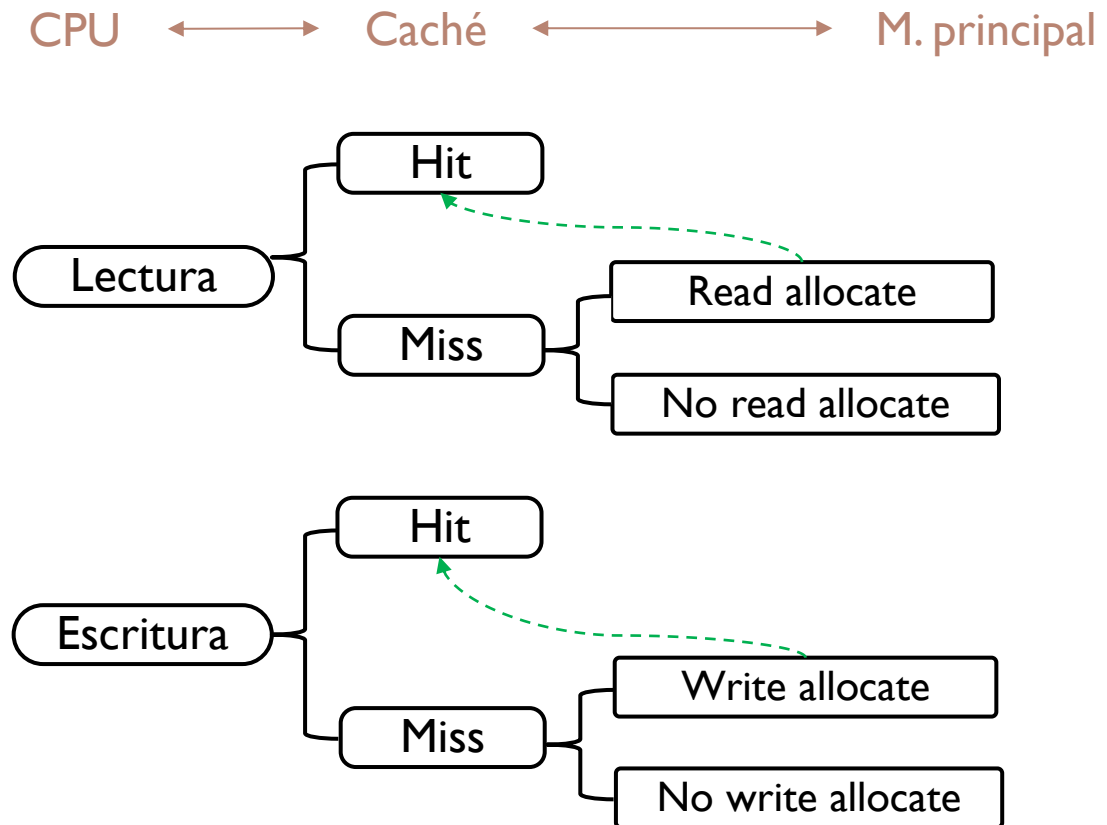
- ▶ Ej.: CPU multicore con caché por core
 - ▶ Las escrituras en caché solo son vistas por un core
 - ▶ Si cada core escribe sobre una misma palabra, ¿cuál es el resultado final?
- ▶ Ej.: módulo de E/S con acceso directo a M.P.
 - ▶ Actualización por DMA puede no ser coherente
- ▶ Porcentaje de referencias a memoria para escritura del orden del 15%.



Políticas de escritura

- ▶ Cuando se modifica un dato en memoria caché, en caso de **fallo**, hay que traer en algún momento el bloque de memoria principal
- ▶ Alternativas:
 - ▶ **Write-allocate**: traer bloque a caché y actualizar en caché.
 - ▶ Bueno si siguientes lecturas/escrituras al mismo bloque
 - ▶ **No-write-allocate**: escribir directamente en memoria
 - ▶ Bueno si siguientes operaciones no al mismo bloque

Políticas de escritura



- ▶ Típico:
 - ▶ write-through + no-write-allocate
 - ▶ write-back + write-allocate

Tipos de fallos en caché

- ▶ *Compulsory* (obligatorios)

- ▶ La primera vez que se accede a una palabra de una línea que no está
- ▶ Los fallos en una caché infinita

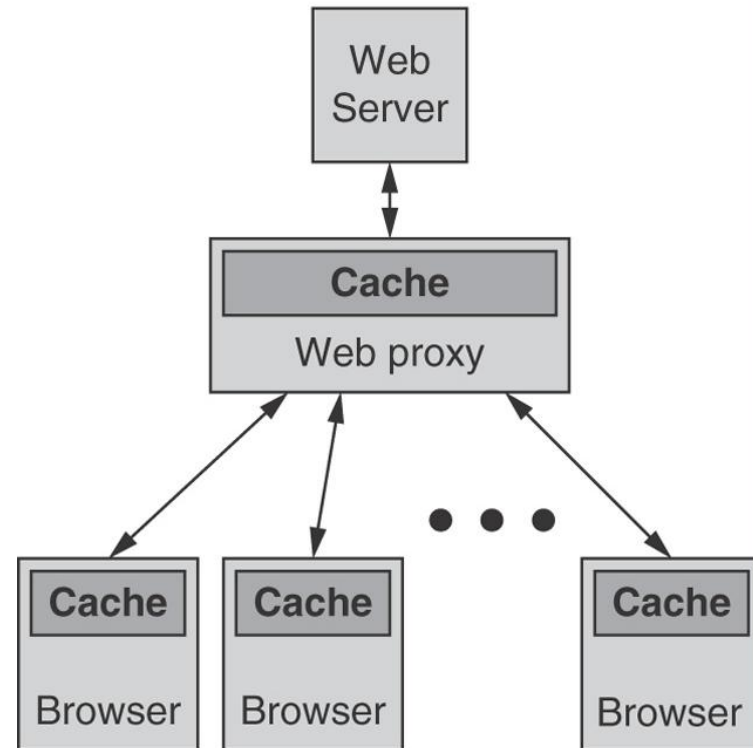
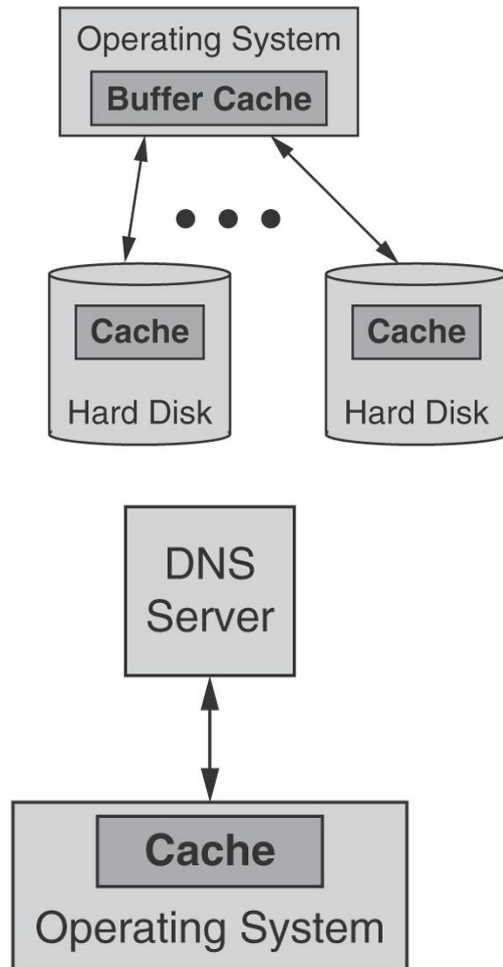
- ▶ *Capacity* (capacidad)

- ▶ Cuando se acceden a muchas otras palabras antes de volver a usar una palabra inicial.
- ▶ Los fallos en una caché totalmente asociativa

- ▶ *Conflict* (conflictos)

- ▶ Dos palabras van a la misma línea de caché
- ▶ Los fallos en correspondencia directa por accesos distintos a misma línea.

Ejemplos de cachés en otros sistemas



Memory Systems
Cache, DRAM, Disk
Bruce Jacob, Spencer Ng, David Wang
Elsevier

Grupo ARCOS

uc3m | Universidad **Carlos III** de Madrid

Tema 5: Jerarquía de Memoria (II)

Estructura de Computadores

Grado en Ingeniería Informática

Grado en Matemática aplicada y Computación

Doble Grado en Ingeniería Informática y Administración de Empresas

